

基于 Reinforce Tactics 的策略类强化学习学习指南

目录

- 基于 Reinforce Tactics 的策略类强化学习学习指南
 - 从游戏机制、数学基础到 PPO、分层强化学习与 AlphaZero
- 第 00 章 序言、学习目标与实践路线
 - 0.1 本书要解决的问题
 - 0.2 读者需要具备什么
 - 0.3 项目在本书中的角色
 - 0.4 五层知识结构
 - 0.5 环境安装与验证
 - 0.6 如何理解 “冒烟实验”
 - 0.7 代码阅读方法
 - 0.8 学习检查
- 第 01 章 游戏规则与策略问题
 - 1.1 棋盘与一局游戏
 - 1.2 八类单位
 - 1.3 建筑、收入与占领
 - 1.4 玩家回合与环境步
 - 1.5 合法动作
 - 1.6 胜利与失败
 - 1.7 策略不只是战斗
 - 1.8 对手是环境的一部分
 - 1.9 一个手工决策例
 - 1.10 本章练习
- 第 02 章 智能体、环境与马尔可夫决策过程
 - 2.1 强化学习闭环
 - 2.2 MDP 五元组
 - 2.3 状态与观察
 - 2.4 转移概率从何而来
 - 2.5 策略
 - 2.6 为什么本项目训练接口是单智能体 1v1
 - 2.7 微动作 MDP 的时间尺度
 - 2.8 终止与截断属于模型定义
 - 2.9 MDP 建模检查表
 - 2.10 本章练习
- 第 03 章 概率与统计基础
 - 3.1 为什么强化学习离不开概率

- 3.2 样本空间、事件与概率
- 3.3 随机变量
- 3.4 期望
- 3.5 方差与标准差
- 3.6 样本均值与标准误
- 3.7 胜率的二项近似
- 3.8 条件概率
- 3.9 独立性与伪重复
- 3.10 多随机种子
- 3.11 本章练习
- 第 04 章 微积分、梯度与神经网络
 - 4.1 从函数开始
 - 4.2 标量、向量、矩阵和张量
 - 4.3 导数表示局部变化率
 - 4.4 偏导数与梯度
 - 4.5 链式法则
 - 4.6 激活函数
 - 4.7 对数与对数概率
 - 4.8 神经网络的一次训练
 - 4.9 卷积与空间结构
 - 4.10 常见误解
 - 4.11 本章练习
- 第 05 章 强化学习基础
 - 5.1 即时奖励与回报
 - 5.2 折扣与有效时间范围
 - 5.3 状态价值函数
 - 5.4 动作价值函数
 - 5.5 优势函数
 - 5.6 Bellman 递推
 - 5.7 Monte Carlo 与 TD
 - 5.8 n 步回报与 GAE
 - 5.9 探索与利用
 - 5.10 信用分配
 - 5.11 On-policy 与 Off-policy
 - 5.12 本章练习
- 第 06 章 用于强化学习的 PyTorch
 - 6.1 张量与形状
 - 6.2 自动求导
 - 6.3 `nn.Module`

- 6.4 训练模式与评估模式
- 6.5 优化器
- 6.6 概率分布
- 6.7 动作掩码在 logits 层生效
- 6.8 项目的空间特征提取
- 6.9 数值稳定性
- 6.10 CPU 冒烟
- 6.11 本章练习
- 第 07 章 Gymnasium 环境契约
 - 7.1 环境的最小接口
 - 7.2 `reset`
 - 7.3 `step`
 - 7.4 观察空间
 - 7.5 动作空间
 - 7.6 终止与截断
 - 7.7 `info` 的作用
 - 7.8 Wrapper
 - 7.9 向量环境
 - 7.10 PettingZoo 边界
 - 7.11 验证命令
 - 7.12 本章练习
- 第 08 章 Stable-Baselines3 训练框架
 - 8.1 框架负责什么
 - 8.2 模型构造
 - 8.3 `learn`
 - 8.4 `predict`
 - 8.5 On-policy rollout buffer
 - 8.6 DQN replay buffer
 - 8.7 Monitor 与日志
 - 8.8 Callback
 - 8.9 模型保存与环境兼容
 - 8.10 `sb3-contrib`
 - 8.11 配置优先级
 - 8.12 最小接口检查
 - 8.13 本章练习
- 第 09 章 项目运行时与数据流
 - 9.1 分层结构
 - 9.2 CLI 的四种模式

- 9.3 GameState 的职责
- 9.4 StrategyGameEnv.__init__
- 9.5 reset 数据流
- 9.6 step 数据流
- 9.7 对手回合
- 9.8 自对弈工厂
- 9.9 配置流
- 9.10 数据形状穿过各层
- 9.11 失败归属示例
- 9.12 本章练习
- 第 10 章 观察编码与空间特征
 - 10.1 观察设计决定可学信息
 - 10.2 grid
 - 10.3 units
 - 10.4 global_features
 - 10.5 玩家相对视角
 - 10.6 战争迷雾
 - 10.7 跨地图 padding
 - 10.8 默认展平的代价
 - 10.9 SpatialFeatureExtractor
 - 10.10 观察检查
 - 10.11 历史问题：空间盲性
 - 10.12 本章练习
- 第 11 章 动作空间与动作掩码
 - 11.1 六字段动作
 - 11.2 组合空间
 - 11.3 逐维掩码
 - 11.4 历史基准证据
 - 11.5 Flat Discrete 精确掩码
 - 11.6 上限与截断
 - 11.7 自回归动作头
 - 11.8 掩码概率
 - 11.9 掩码必须进入整条链
 - 11.10 动手观察
 - 11.11 选择建议
 - 11.12 本章练习
- 第 12 章 奖励塑形、终止与截断
 - 12.1 奖励不是评价报告

- 12.2 稀疏奖励
- 12.3 直接稠密奖励的风险
- 12.4 势能塑形
- 12.5 势函数组成
- 12.6 终局的势能关闭
- 12.7 杀敌刷分
- 12.8 永不结束回合
- 12.9 和棋与截断
- 12.10 奖励分解
- 12.11 奖励量级检查
- 12.12 设计流程
- 12.13 本章练习
- 第 13 章 DQN: 从动作价值到深度 Q 网络
 - 13.1 DQN 解决什么问题
 - 13.2 从 Q-learning 开始
 - 13.3 经验回放
 - 13.4 目标网络
 - 13.5 ϵ -greedy
 - 13.6 与策略梯度的区别
 - 13.7 本项目的兼容性事实
 - 13.8 为什么项目不以 DQN 为主线
 - 13.9 常见失败
 - 13.10 CPU 兼容性验证
 - 13.11 本章练习
- 第 14 章 A2C: 同步优势 Actor-Critic
 - 14.1 从策略梯度到 Actor-Critic
 - 14.2 A2C 的含义
 - 14.3 n 步目标
 - 14.4 联合损失
 - 14.5 与 PPO 的差别
 - 14.6 项目实现路径
 - 14.7 为什么保留 A2C
 - 14.8 实验设计
 - 14.9 常见问题
 - 14.10 本章练习
- 第 15 章 PPO: 受约束的策略更新
 - 15.1 PPO 要解决的稳定性问题
 - 15.2 新旧策略概率比
 - 15.3 未裁剪代理目标

- 15.4 裁剪目标
 - 正优势数字例
 - 负优势数字例
- 15.5 GAE
- 15.6 PPO 总损失
- 15.7 一次 PPO 更新
- 15.8 超参数解释
- 15.9 项目为何选择 PPO
- 15.10 历史困难
- 15.11 监控指标
- 15.12 本章练习
- 第 16 章 MaskablePPO 与第一次可验证训练
 - 16.1 MaskablePPO 改变什么
 - 16.2 环境构造
 - 16.3 模型构造
 - 16.4 训练与预测
 - 16.5 可直接运行的 CPU 冒烟
 - 16.6 第一次正式训练应怎样定义
 - 管线层
 - 学习信号层
 - 能力层
 - 16.7 为什么不是直接运行基础 CLI
 - 16.8 MultiDiscrete 与 Flat 选择
 - 16.9 向量环境注意
 - 16.10 失败诊断
 - 16.11 结果记录模板
 - 16.12 本章练习
- 第 17 章 课程学习与 Bootstrap
 - 17.1 冷启动问题
 - 17.2 课程阶段
 - 17.3 训练状态机
 - 17.4 晋级噪声
 - 17.5 MixedBot 难度桥
 - 17.6 阶段交接：峰值而非末值
 - 17.7 早期失败：确定性 noop 教师
 - 17.8 地图迁移与价值错位
 - 17.9 课程漂移与错误修复链
 - 现象
 - 证据

- 排除过程
 - 已落地改进
 - 当前开放问题
- 17.10 `max_timesteps` 是安全阀
- 17.11 最小管线实验
- 17.12 本章练习
- 第 18 章 行为克隆
 - 18.1 用专家动作初始化策略
 - 18.2 数据如何采集
 - 18.3 交叉熵的数字例
 - 18.4 多字段动作的损失
 - 18.5 类别不平衡
 - 18.6 分布偏移
 - 18.7 历史失败 A: 确定性 Bot 产生重复轨迹
 - 18.8 历史失败 B: 测量环境漏参数
 - 18.9 历史失败 C: 好看的 loss 不等于策略强
 - 18.10 BC → PPO
 - 18.11 项目入口
 - 18.12 本章练习
- 第 19 章 自对弈
 - 19.1 为什么需要自对弈
 - 19.2 非平稳性
 - 19.3 快照池
 - 19.4 为什么历史对手重要
 - 19.5 对手快照与权重安全
 - 19.6 玩家视角与座次
 - 19.7 固定评估锚
 - 19.8 混合训练
 - 19.9 与课程和 BC 的组合
 - 19.10 项目实现
 - 19.11 RNG 与推理热路径
 - 19.12 常见失败
 - 19.13 本章练习
- 第 20 章 Feudal RL: 用层级决策分解长期战略
 - 20.1 为什么需要层级
 - 20.2 从时间抽象理解 Manager 与 Worker
 - 20.3 项目中的目标表示
 - 目标条件不是命令执行器
 - 20.4 内在奖励: 怎样告诉 Worker 目标是否有进展

- 20.5 Worker 奖励：必须以实际代码为准
 - 数值例子
- 20.6 Manager 奖励与段级回报
- 20.7 两套 Worker 动作头
 - 20.7.1 传统多头 Worker
 - 20.7.2 自回归 Worker
- 20.8 Manager 和 Worker 都使用 PPO
- 20.9 为什么项目尝试 Feudal RL
- 20.10 历史困难的因果整理
 - 当时现象
 - 调查证据
 - 已排除的简单解释
 - 已落地修复
 - 当前仍开放的问题
- 20.11 最小实操
- 20.12 常见误解
- 20.13 本章小结
- 20.14 练习
- 第 21 章 蒙特卡洛树搜索：在行动之前进行推演
 - 21.1 从反应式策略到显式搜索
 - 21.2 搜索树中的统计量
 - 数值例子
 - 21.3 Selection: PUCT 选择
 - 玩家视角的符号
 - 21.4 Expansion: 只扩展合法动作
 - 惰性复制
 - 21.5 Evaluation: 策略头与价值头
 - 21.6 Backup: 把叶节点证据传回根
 - 21.7 根节点狄利克雷噪声
 - 21.8 从访问次数得到动作分布
 - 21.9 当前代码的完整调用路径
 - 21.10 计算成本
 - 21.11 失败模式与诊断
 - 全零访问分布
 - 所有价值都接近零
 - 搜索偏爱非法动作
 - 加大 simulation 数却没有提升
 - 评估结果波动
 - 21.12 为什么 MCTS 适合又不完全适合本项目

- 21.13 最小实操
- 21.14 本章小结
- 21.15 练习
- 第 22 章 AlphaZero: 让搜索生成更强的学习目标
 - 22.1 核心闭环
 - 22.2 训练样本 (s, π, z)
 - 22.3 项目的双头网络
 - 策略头
 - 价值头
 - 为什么使用残差块
 - 22.4 联合损失
 - 数值例子
 - 22.5 搜索为何可看作策略改进
 - 22.6 Replay Buffer 与数据时效
 - 22.7 自对弈中的探索日程
 - 22.8 候选模型晋级
 - 22.9 当前实现状态
 - 22.10 历史困难的因果整理
 - 当时现象
 - 调查证据
 - 已落地措施
 - 当前开放问题
 - 22.11 最小训练
 - 22.12 结果应怎样阅读
 - 22.13 与 PPO、自对弈 PPO 的区别
 - 22.14 常见误解
 - 22.15 本章小结
 - 22.16 练习
- 第 23 章 实验设计、统计误差与可复现性
 - 23.1 训练成功不是单次曲线上升
 - 23.2 先写假设, 再运行
 - 23.3 随机种子控制了什么
 - 23.4 均值、方差与标准误
 - 23.5 胜率是二项比例
 - 23.6 和局与截断不能随意并入失败
 - 23.7 训练曲线不是独立样本
 - 23.8 消融实验
 - 23.9 配对评估
 - 23.10 多重尝试与选择偏差

- 23.11 可复现实验记录
- 23.12 本项目的诊断指标
- 23.13 一个完整实验模板
- 23.14 本章小结
- 23.15 练习
- 第 24 章 规则 Bot、训练对手与游戏平衡
 - 24.1 对手是训练分布的一部分
 - 24.2 NoopBot: 最低层的因果探针
 - 24.3 RandomBot 与动作吞吐量
 - 24.4 BalancedRandomBot: 随军队规模增长的随机压力
 - 24.5 SimpleBot: 局部启发式
 - 24.6 Medium、Advanced 与 Master
 - 24.7 MixedBot: 平滑难度边界
 - 24.8 确定性与随机 tie-break
 - 24.9 训练对手选择的三个风险
 - 过弱
 - 过强
 - 过窄
 - 24.10 游戏平衡与学习难度
 - 经济滚雪球
 - 单位耐久与和局
 - 地图距离
 - 动作分支
 - 24.11 Bot 行为指标
 - 24.12 平衡实验范式
 - 24.13 最小实操
 - 24.14 本章小结
 - 24.15 练习
- 第 25 章 模型评估、Elo、循环赛与回放
 - 25.1 评估回答的不是“训练奖励有多高”
 - 25.2 `evaluate_model` 的调用契约
 - 25.3 平均回报与胜率的分工
 - 25.4 Elo 的基本公式
 - 25.5 Elo 的局限
 - 25.6 循环赛调度
 - 25.7 TournamentConfig 的重要字段
 - 25.8 模型公平比较
 - 25.9 回放是行为证据
 - Trace 与完整 replay

- 25.10 模型比较的分层报告
 - 25.11 最小实操
 - 25.12 典型误判
 - 25.13 本章小结
 - 25.14 练习
- 第 26 章 LLM Bot: 提示、状态序列化与安全执行
 - 26.1 LLM Bot 不是强化学习训练
 - 26.2 状态为什么要序列化
 - 26.3 玩家视角与信息边界
 - 26.4 提示由规则、任务与格式组成
 - 26.5 两阶段规划
 - 26.6 结构化动作
 - 26.7 JSON 提取不是合法性验证
 - 26.8 逐动作重新校验
 - 26.9 失败恢复
 - 26.10 状态历史与 token
 - 26.11 延迟与成本模型
 - 26.12 安全边界
 - 26.13 Mock 实操
 - 26.14 LLM 与 RL 的可比性
 - 26.15 本章小结
 - 26.16 练习
- 第 27 章 DEV 工具链与综合研究实践
 - 27.1 从“能运行”到“可研究”
 - 27.2 测试金字塔
 - 单元测试
 - 集成测试
 - 冒烟测试
 - 统计回归
 - 27.3 质量检查命令
 - 27.4 配置优先级
 - 27.5 诊断顺序
 - 规则层
 - 环境契约
 - 观察与动作
 - 奖励与终止
 - 算法数值
 - 对手与课程
 - 统计评估

- 27.6 最小可复现问题
- 27.7 日志与 checkpoint
- 27.8 综合实践：从短训练到研究报告
 - 阶段 A：环境验收
 - 阶段 B：接口边界
 - 阶段 C：第一条训练曲线
 - 阶段 D：课程训练
 - 阶段 E：进阶路径
 - 阶段 F：正式研究问题
 - 阶段 G：报告
- 27.9 教材构建与验证
- 27.10 DEV 中的常见错误
- 27.11 研究伦理与结果表述
- 27.12 全书实践终点
- 27.13 本章小结
- 27.14 练习
- 附录 A 数学符号、公式与术语速查
 - A.1 记号约定
 - A.2 集合、下标与条件
 - A.3 概率与统计
 - 概率分布
 - 期望
 - 方差与标准差
 - 协方差
 - 样本均值
 - 样本方差
 - 标准误
 - 置信区间
 - A.4 向量与矩阵
 - A.5 导数、梯度与链式法则
 - A.6 激活、softmax 与对数概率
 - A.7 监督学习损失
 - A.8 回报与折扣
 - A.9 价值、Q 与优势
 - A.10 Bellman 方程
 - A.11 MC、TD 与 GAE
 - A.12 策略梯度
 - A.13 熵与 KL
 - A.14 DQN 公式

- A.15 A2C 公式
- A.16 PPO 公式
- A.17 动作掩码
- A.18 奖励塑形
- A.19 BC
- A.20 自对弈
- A.21 Feudal
- A.22 MCTS
- A.23 AlphaZero
- A.24 项目术语
- 附录 B 命令、配置与参数参考
 - B.1 统一执行环境
 - B.2 教材构建
 - B.3 测试命令
 - B.4 环境构造参数
 - B.5 PPO 与 MaskablePPO 参数
 - B.6 MaskablePPO 最小配置
 - B.7 Bootstrap
 - B.8 行为克隆
 - B.9 自对弈
 - B.10 Feudal
 - B.11 AlphaZero
 - B.12 评估
 - B.13 Tournament
 - B.14 TensorBoard
 - B.15 配置覆盖与审计
- 附录 C 历史实验索引与因果证据表
 - C.1 怎样阅读本附录
 - C.2 v15–v23: 课程门槛与熵调度阶段
 - 这一阶段最重要的方法论结论
 - C.3 v24–v28: 复现与奖励项隔离
 - v24: 看似忠实、实际不忠实的复现
 - v25: random_10 二分诊断
 - v26: 真正对齐奖励后的复现
 - v27a/b/c: 单项回加
 - v28: 生产奖励修正版
 - C.4 v29–v32: 时间上限、warm start 与晋级噪声
 - C.5 v33: BC warm start
 - C.6 v34–v40: 单一单位构成吸引子

- v34: 深进度不等于策略多样
- v35–v37: 四类杠杆
- v38 与 v39: 结构削弱和成本削弱
- v40: 跳过 starter
- C.7 v41–v46: 击杀刷分、和局与对手多样性
 - v41: kill-farm draw plateau
 - v42: 去掉战斗刷分
 - v43a/b: 对手多样性与容量配对
 - v44–v46
- C.8 v48–v54: 动作空间、对称战斗、截断与可占结构
 - v49 的奖励几何
 - v52a 与 v52b
 - v53 系列
 - v54
- C.9 2026-07-12 训练审查
- C.10 2026-07-24 全管线审查
- C.11 跨版本因果链
 - 奖励吸引子链
 - 课程交接链
 - 单位构成链
 - 对手分布链
- C.12 当前应如何复现历史结论
- C.13 版本总索引
- 附录 D 章节练习参考答案
 - D.00 第 0 章
 - D.01 第 1 章
 - D.02 第 2 章
 - D.03 第 3 章
 - D.04 第 4 章
 - D.05 第 5 章
 - D.06 第 6 章
 - D.07 第 7 章
 - D.08 第 8 章
 - D.09 第 9 章
 - D.10 第 10 章
 - D.11 第 11 章
 - D.12 第 12 章
 - D.13 第 13 章
 - D.14 第 14 章

- D.15 第 15 章
- D.16 第 16 章
- D.17 第 17 章
- D.18 第 18 章
- D.19 第 19 章
- D.20 第 20 章
- D.21 第 21 章
- D.22 第 22 章
- D.23 第 23 章
- D.24 第 24 章
- D.25 第 25 章
- D.26 第 26 章
- D.27 第 27 章
- 附录 E 源码符号索引与原始文献
 - E.1 索引使用原则
 - E.2 游戏领域层
 - `reinforcetactics/core/game_state.py`
 - `reinforcetactics/constants.py`
 - `reinforcetactics/game/mechanics.py`
 - E.3 规则 Bot
 - `reinforcetactics/game/bot_base.py`
 - `reinforcetactics/game/bot.py`
 - `reinforcetactics/game/model_bot.py`
 - `reinforcetactics/game/alphazero_bot.py`
 - E.4 Gymnasium 环境
 - `reinforcetactics/rl/gym_env.py`
 - `reinforcetactics/rl/observation.py`
 - `reinforcetactics/rl/masking.py`
 - E.5 特征提取与策略网络
 - E.6 配置
 - `reinforcetactics/rl/config.py`
 - `reinforcetactics/utils/run_config.py`
 - E.7 Bootstrap
 - `reinforcetactics/rl/bootstrap.py`
 - `scripts/train/train_bootstrap.py`
 - E.8 行为克隆
 - `reinforcetactics/rl/imitation.py`
 - `scripts/build_bc_warmstart.py`

- E.9 自对弈
 - `reinforcetactics/rl/self_play.py`
 - `scripts/train/train_self_play.py`
- E.10 Feudal RL
 - `reinforcetactics/rl/feudal_rl.py`
 - `scripts/train/train_feudal_rl.py`
- E.11 MCTS 与 AlphaZero
 - `reinforcetactics/rl/alphazero_net.py`
 - `reinforcetactics/rl/mcts.py`
 - `reinforcetactics/rl/alphazero_trainer.py`
 - `scripts/train/train_alphazero.py`
- E.12 评估、Tournament 与 Replay
 - `reinforcetactics/rl/evaluation.py`
 - `reinforcetactics/tournament/`
 - `reinforcetactics/utils/replay_actions.py`
 - `reinforcetactics/utils/replay_player.py`
- E.13 LLM Bot
 - `reinforcetactics/game/llm_bot.py`
 - `reinforcetactics/game/llm_prompts.py`
- E.14 DEV 与测试索引
- E.15 原始论文与教材
 - 强化学习基础
 - DQN
 - Actor-Critic、GAE 与 PPO
 - 奖励塑形
 - 行为克隆与数据聚合
 - 层级强化学习
 - MCTS 与 AlphaZero
 - 无效动作掩码
- E.16 框架官方文档
 - PyTorch
 - Gymnasium
 - Stable-Baselines3
 - sb3-contrib
 - PettingZoo
- E.17 历史材料与当前事实的分界
- E.18 建议阅读顺序

第 00 章 序言、学习目标与实践路线

0.1 本书要解决的问题

强化学习教材常从马尔可夫决策过程或 Bellman 方程开始。这样的安排在数学上严谨，却容易使初学者把“公式”“框架接口”和“真实游戏行为”视为三件互不相干的事情。本书采用相反的路线：先观察一局可运行的策略游戏，再逐层回答以下问题。

1. 游戏中的局面怎样变成神经网络能够接收的数值？
2. 建造、移动、攻击和结束回合怎样变成动作空间？
3. 胜负很晚才发生时，算法如何判断早先哪一步有贡献？
4. 为什么一个语法正确的奖励函数仍可能教出拖延、刷伤害或只造单一兵种的策略？
5. PPO、DQN、A2C、MCTS 和 AlphaZero 分别解决什么问题，哪些真正适合本项目？
6. 训练曲线好看是否表示模型真的更会玩游戏？

读完全书后，读者应能独立完成“检查环境—选择动作表示—设计奖励—训练—评估—诊断—复现实验”的闭环，而不只是复制一条训练命令。



0.2 读者需要具备什么

本书假定读者：

- 能阅读 Python 函数、类、列表、字典和异常；
- 理解循环、条件分支和基本调试；
- 知道程序会从文件读取配置并产生输出；
- 不要求学过概率论、微积分、线性代数或机器学习。

第三至第五章会补齐后续公式真正需要的数学。这里的“补齐”不是回避推导，而是把推导拆成可以逐步核验的小问题。每个核心公式都会说明计算对象、每个符号、数值例子、代码位置和易错理解。

0.3 项目在本书中的角色

Reinforce Tactics 是一个回合制策略游戏，同时提供可无界面运行的游戏引擎和 Gymnasium 环境。它适合作为学习载体，原因不是“规则简单”，而是它包含许多现实强化学习系统共有的困难：

项目现象	强化学习问题
一回合可连续操作多支单位	环境步与领域回合不一致
动作由类型、单位、起点和终点组成	结构化动作与组合爆炸
大多数动作组合违反游戏规则	非法动作掩码
赢棋要经过造兵、行军、战斗和占领	长期信用分配
对手也在学习或变化	非平稳训练分布
地图尺寸不同	观察与动作空间兼容
中间奖励可被反复获取	奖励投机
胜率来自有限局数	统计不确定性

因此，本项目既能演示标准接口，也能说明“算法公式正确”为什么不等于“完整系统正确”。

0.4 五层知识结构

本书把一个强化学习项目分成五层。

1. **领域层**：游戏规则、状态、合法动作和胜负。
2. **环境层**：观察、动作编码、奖励、终止与截断。
3. **算法层**：价值估计、策略更新、搜索与模仿。
4. **训练层**：课程、对手池、回调、日志和 checkpoint。
5. **证据层**：评估设计、置信区间、回放和可复现记录。

排查问题时必须先确定错误属于哪一层。例如，模型持续输出无效动作可能来自环境掩码，而不是 PPO 学习率；胜率突然下降可能来自评估对手变化，而不是模型权重退化。

0.5 环境安装与验证

本机约定使用 Conda 环境 `reinforce-tactics`。从仓库根目录执行：

```
conda activate reinforce-tactics

python -c "import sys, gymnasium, stable_baselines3, sb3_contrib, torch;
print(sys.version); print(gymnasium.__version__, stable_baselines3.__version__,
sb3_contrib.__version__, torch.__version__)"
```

本书验证输出的主要版本为 Python 3.12.13、Gymnasium 1.3.0、Stable-Baselines3 2.9.0、sb3-contrib 2.9.0 和 PyTorch 2.13.0 CPU。若第一行的 Python 不是 Conda 环境中的解释器，后续出现 `ModuleNotFoundError` 时应先修正环境，而不是重复安装到系统 Python。

检查包和项目导入：

```
python -c "from reinforcetactics.rl.gym_env import StrategyGameEnv;
env=StrategyGameEnv(map_file='maps/1v1/starter.csv', opponent='noop');
print(env.observation_space); print(env.action_space); env.close()"
```

这条命令不训练模型，只确认项目、地图、观察空间和动作空间能够创建。

0.6 如何理解“冒烟实验”

强化学习具有随机性，短跑只能验证接口，不能验证最终能力。本书使用三类证据，标记方式如下。

标记	含义	可以得出的结论
CPU 冒烟	数十至数千步	接口可运行、张量形状正确、损失可计算
历史长训	仓库保存的长期运行记录	在特定版本、配置和随机种子下出现过某结果
算法性质	公式、论文或实现逻辑	在相应假设下成立的理论或结构事实

“64 步训练成功” 只表示训练循环完成，不表示策略学会获胜。反过来，单次长训成功也不证明参数普遍有效；需要多随机种子和固定评估协议。

0.7 代码阅读方法

阅读本书中的代码时，建议保持三个问题：

- 1. 输入和输出的形状是什么？
- 2. 这个函数改变了什么状态？
- 3. 这个数值是游戏规则、训练信号还是评估指标？

例如 `reward` 是训练信号，不是 GUI 中的游戏积分；`step` 是一次环境微动作，不一定是一整个玩家回合；`done` 在现代 Gymnasium 中应拆成 `terminated` 与 `truncated`。

0.8 学习检查

完成本章后，应能回答：

- 1. 为什么本项目比单步分类任务更适合展示信用分配？
- 2. 为什么训练短跑不能证明模型已经学会游戏？
- 3. 环境创建失败时，为什么先检查 Python 解释器比调算法超参数更合理？
- 4. 领域层、环境层和算法层分别负责什么？

下一章先建立领域层：游戏究竟要求智能体做什么。

第 01 章 游戏规则与策略问题

1.1 棋盘与一局游戏

Reinforce Tactics 在方格地图上进行。地图包含可通行地形、阻挡地形以及总部、塔楼等建筑。每名玩家拥有金币、单位和建筑控制权。玩家在自己的回合中可以执行多个微动作，最后显式结束回合。对局通常通过占领敌方总部、消灭满足规则条件的敌方力量，或在最大回合数处形成和局而结束。



Beginner 地图对局

游戏逻辑的单一真源是 `reinforcetactics/core/game_state.py` 中的 `GameState`。GUI、规则 Bot、LLM Bot、强化学习环境和锦标赛都应通过同一规则层改变状态。这个设计非常重要：如果训练环境另写一套简化规则，模型可能在训练器里合法、进入 GUI 后却无法行动。

1.2 八类单位

项目包含八类单位。精确数值由常量和可能的 `engine_overrides` 决定，因此本书以稳定的功能角色为主。

单位	代码	主要角色	学习上的意义
Warrior	W	低成本近战基础单位	容易形成单兵种局部最优
Mage	M	法术攻击	需要选择高价值目标
Cleric	C	治疗或净化	奖励常延迟体现为存活率
Archer	A	远程攻击	位置和射程关系重要
Knight	K	高机动或冲击	行动价值依赖路径与时机
Rogue	R	灵活战术单位	增加动作组合和反制关系
Sorcerer	S	增益、减益和控制	动作目标依赖更复杂
Barbarian	B	高攻击近战	成本、伤害与生存的权衡

单位类型不是独立标签。选择造什么兵取决于金币、建筑、地图距离、敌方组成和未来数个回合。仅按“当前伤害最大”行动会忽略经济与占领目标。

1.3 建筑、收入与占领

建筑同时承担经济、生产和胜负功能。控制建筑通常会在回合转换时产生收入；某些建筑允许创建单位；敌方总部是核心战略目标。占领不是瞬时视觉效果，而是由单位在合法位置执行 `seize` 一类动作改变控制进度或归属。

经济系统制造了强化学习中的延迟收益：

- 花费金币造兵会使当前资源减少；
- 新单位可能数回合后才参与战斗；
- 占领中立建筑的直接收益有限，但之后每回合增加收入；
- 过度储蓄在数值上安全，却可能失去地图控制。

因此，一步奖励不能只按金币余额增加。否则智能体可能拒绝合理消费。

1.4 玩家回合与环境步

游戏回合不是强化学习环境中的一个 `step`。智能体的一回合可表示为：

创建单位 → 移动单位 A → 攻击 → 移动单位 B → 占领 → 结束回合

上述每个箭头前的操作通常各占一个环境步。只有执行 `end_turn` 后，环境才让规则 Bot 完成其整个回合，再把控制权交回智能体。

概念	作用域	典型代码
环境步	一次微动作	<code>StrategyGameEnv.step(action)</code>
玩家回合	当前玩家的一组动作	<code>GameState.current_player</code>
完整轮次	双方都行动一次	由游戏回合计数体现
一局 episode	从初始状态到终止或截断	<code>reset</code> 到下一次 <code>reset</code>

这个区别影响折扣：若一局有 80 个游戏回合，但每回合平均有 8 个微动作，终局奖励可能距离开局约 640 个环境步。

1.5 合法动作

`GameState.get_legal_actions(player)` 根据当前状态生成合法操作。合法性至少依赖：

- 当前玩家及单位所有权；
- 单位是否已行动、是否受到状态效果；
- 起点、终点、射程和路径；
- 目标位置是否被占用；
- 金币是否足够、建筑是否允许生产；
- 技能类型与目标类型是否匹配；
- 游戏是否已结束。

合法动作集合会在每一步后变化。移动一个单位可能使另一个单位的路径失效；购买会改变金币；攻击可能消灭目标；占领会改变建筑归属。因此，动作掩码不能只在 `reset` 时计算一次。

1.6 胜利与失败

强化学习环境关心的是终局语义，而不是 GUI 是否显示结束画面。项目的 `info["end_reason"]` 可区分总部占领、消灭、最大回合和最大环境步等原因。需要特别区分：

- **规则终止**：游戏规则确认胜、负或和，例如最大游戏回合导致和局；

- **外部截断**：环境达到 `max_steps`，但规则内的游戏仍可能继续。

终止意味着该轨迹在 MDP 内真正结束；截断通常意味着数据收集器提前停止。二者对价值函数是否继续 bootstrap 的处理不同，第七和第十二章会严格展开。

1.7 策略不只是战斗

一个完整策略至少包含四个互相制约的层面。

1. **经济**：何时消费、生产何种单位。
2. **机动**：如何分配单位、选择路径和保持阵形。
3. **战斗**：目标优先级、射程、治疗、控制与风险。
4. **目标**：占领中立建筑、防守总部、推进敌方总部并结束游戏。

历史训练中出现“只造 Warrior”并不表示模型完全没有学习。它可能学到了一个在当前价格、对手和奖励下足够稳定的局部最优。要判断问题来自算法还是平衡，需要将单位组成、对局结果和地图目标一起分析。

1.8 对手是环境的一部分

从智能体的接口看，执行 `end_turn` 后发生的手对手行动属于环境转移。若对手是固定脚本，转移分布相对稳定；若对手从池中随机抽取，转移带有额外随机性；若对手不断换成当前策略的最新快照，环境变得非平稳。

同一策略面对不同对手可能呈现不同胜率。因此，“模型强度”不是脱离对手定义的单一数值。课程学习需要明确每一阶段的对手，最终评估需要固定锚点。

1.9 一个手工决策例

假设当前有足够金币建造一个 Warrior 或保留资金，前方有中立塔楼：

- 立即造兵：当前金币下降，但下一回合多一个可推进单位；
- 保留金币：当前资源指标更高，但可能让对手先占塔；
- 移动现有单位：短期没有伤害奖励，但缩短占领距离；
- 攻击附近敌人：获得即时战斗信号，却可能偏离总部目标。

这个例子说明奖励函数必须表达“最终赢棋优先”，中间信号只能充当路标。若击伤奖励可以无限获取而占领只奖励一次，智能体就可能主动维持战斗而不结束游戏。

1.10 本章练习

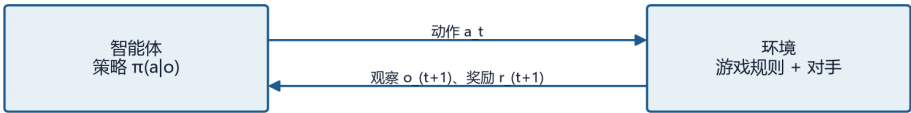
1. 给出一个“当前金币减少但长期价值增加”的动作。
2. 为什么 `end_turn` 必须是一个可学习的动作，而不能总由环境自动执行？
3. 若评估时换了更强 Bot，胜率下降是否足以证明模型退化？
4. 为什么合法动作必须在每个环境步重新计算？

第 02 章 智能体、环境与马尔可夫决策过程

2.1 强化学习闭环

强化学习研究连续决策。智能体观察环境，选择动作；环境执行规则，返回新观察和奖励；这一过程重复到 episode 结束。

强化学习的闭环



智能体与环境闭环

形式化地，一条轨迹写作：

$$\tau = (S_0, A_0, R_1, S_1, A_1, R_2, ..., S_T)$$

符号	含义	本项目实例
S_t	时刻 t 的完整状态	棋盘、双方单位、金币、回合等
A_t	在状态 S_t 选择的动作	创建、移动、攻击、占领或结束回合
R_{t+1}	动作后得到的即时奖励	终局、占领、势能变化等
T	episode 最后一步	胜负、规则和局或外部截断

奖励下标写成 R_{t+1} ，强调它是在执行 A_t 后才得到。不同资料也会写成 r_t ；只要定义一致，两种记法都可使用。

2.2 MDP 五元组

马尔可夫决策过程（MDP）通常定义为：

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$$

- $\mathcal{S}$ ：所有可能状态的集合；
- $\mathcal{A}$ ：动作集合；
- $P(s'|s, a)$ ：在状态 s 执行动作 a 后转移到 s' 的概率；
- $R(s, a, s')$ ：转移对应的奖励；
- γ ：折扣因子，决定未来奖励的当前权重。

“马尔可夫”不是说未来与历史毫无关系，而是说：给定当前完整状态后，预测下一状态不再需要更早历史。

$$P(S_{t+1}|S_t, A_t, S_{t-1}, A_{t-1}, \dots) = P(S_{t+1}|S_t, A_t)$$

在本项目中，若 `GameState` 包含单位状态、金币、当前玩家、回合、控制权和所有会影响规则的计数器，它可以近似视为马尔可夫状态。若漏掉“单位本回合是否行动”等变量，相同棋盘外观可能产生不同合法动作，观察便不再满足马尔可夫性。

2.3 状态与观察

算法通常不直接接收 `GameState` 对象，而接收数值观察 O_t ：

$$O_t = f(S_t)$$

函数 f 把复杂状态编码成 `grid`、`units` 和 `global_features`。若编码保留了决策所需的全部信息，可以把观察近似当作状态；若启用战争迷雾，智能体只能看见局部信息，问题更接近部分可观测 MDP (POMDP)。

POMDP 额外考虑隐藏状态和观察生成：

$$O_t \sim \Omega(\cdot | S_t)$$

此时同一个观察可能对应多个真实局面。无记忆策略只能按当前可见信息行动；带循环网络、历史堆叠或显式信念状态的策略才可能利用过去推断隐藏单位。本项目主线策略没有完整的信念状态建模，因此战争迷雾应被视为显著增加难度的实验条件。

2.4 转移概率从何而来

游戏规则本身大多是确定的，但整体转移仍可能随机：

- Bot 的随机选择或随机决胜规则；
- 地图随机生成；

- 初始状态或玩家座次；
- 自对弈对手池抽样；
- 随机种子未统一时的多个随机数生成器。

即使环境完全确定，随机策略也会产生不同轨迹。训练中的随机性至少来自策略采样、环境随机和优化器的小批次。

2.5 策略

策略 π 描述在观察下如何选择动作。随机策略写作：

$$\pi_{\theta}(a|o) = P(A_t = a | O_t = o; \theta)$$

θ 是神经网络参数。确定性评估通常选择概率最大的合法动作；随机训练则按分布采样，以获得探索。

策略不是“把局面映射为唯一动作”的表。对 PPO 而言，它是可微的概率分布；对 DQN 而言，常先估计每个动作的 Q 值，再用 ϵ -greedy 选择；对 MCTS 而言，最终动作分布来自搜索访问次数。

2.6 为什么本项目训练接口是单智能体 1v1

游戏引擎支持多种玩家布局，但 `StrategyGameEnv` 的 RL 观察采用“自己/对手”相对通道，且以 `opp = 3 - player` 的方式确定唯一对手。这一编码天然面向两名玩家。

训练时，学习智能体控制一方；另一方 Bot 在智能体结束回合后由环境内部执行。因此，从 Gymnasium API 看只有一个学习者。这样可以直接使用 Stable-Baselines3 的单智能体算法。

真正的多智能体环境需要明确：

- 每个智能体分别看到什么；
- 同时还是轮流行动；
- 奖励是共享、零和还是个体化；
- 智能体退出后环境怎样继续；
- 队友与敌人的相对编码。

`pyproject.toml` 虽列出 `PettingZoo` 依赖，但当前主训练路径没有实现 `PettingZoo` 的 AEC 或 Parallel 环境。不能因为依赖存在就声称项目已经具备通用多智能体训练接口。

2.7 微动作 MDP 的时间尺度

设一个玩家回合平均含 m 个微动作，一局含 H 个玩家回合，则 episode 大约含 mH 个智能体环境步，再加上对手在环境内部执行的动作。折扣按环境步生效：

$$\text{终局权重} \approx \gamma^{mH}$$

若 $\gamma = 0.99$ 、 $mH = 500$ ：

$$0.99^{500} \approx 0.0066$$

开局看到的终局奖励只剩原始量级的约 0.66%。这不是说 PPO 完全忽略终局，而是说明长微动作轨迹会加剧信用分配困难。提高 γ 、缩短 episode、合理塑形、课程学习和分层决策都在不同层面缓解该问题。

2.8 终止与截断属于模型定义

若总部被占领，后续状态没有意义，属于 `terminated=True`。若达到外部 `max_steps`，游戏规则仍可继续，属于 `truncated=True`。对价值目标：

$$y_t = \begin{cases} r_{t+1}, & \text{规则终止} \\ r_{t+1} + \gamma V(s_{t+1}), & \text{时间截断且下一状态可估值} \end{cases}$$

把两者都当作终止会在截断边界错误地把未来价值设为零；反过来，把真正失败也 bootstrap 会让价值穿过不应存在的终局。

2.9 MDP 建模检查表

建立新地图或新规则实验时，应检查：

1. 观察是否包含影响合法动作和奖励的状态变量？
2. 动作编码是否能唯一还原领域动作？
3. 奖励是否与最终目标一致？
4. `terminated` 和 `truncated` 是否语义正确？
5. 对手、随机地图和座次是否进入种子控制？
6. 不同地图的空间形状是否与同一模型兼容？

2.10 本章练习

1. 指出一个在 GUI 中可见但可能不应直接作为奖励的量。
2. 战争迷雾为何使问题从近似 MDP 变成 POMDP？
3. 计算 $\gamma = 0.99$ 时距离 200 步的终局奖励权重。
4. 为什么“项目依赖 PettingZoo”不等于“当前 RL 环境是 PettingZoo 多智能体环境”？

第 03 章 概率与统计基础

3.1 为什么强化学习离不开概率

同一个模型在同一地图上重复对局，结果仍可能不同。原因包括策略采样、随机对手、随机座次和环境随机。训练日志记录的是随机变量的样本，不是永远不变的真值。

理解概率的目标不是手算复杂积分，而是能够区分：

- 一局结果与长期平均；
- 观测到的胜率与真实胜率；
- 策略给出的动作概率与动作价值；
- 随机波动与系统性退化。

3.2 样本空间、事件与概率

一次评估对局的样本空间可简化为：

$$\Omega = \{\text{胜, 负, 和}\}$$

事件“未输”包含胜与和。概率满足 $0 \leq P(E) \leq 1$ ，所有互斥完备结果的概率和为 1。

若固定模型和评估协议，重复对局可视为从某个未知分布采样。改变对手、地图、座次或确定性参数，会改变分布；此时不能把两组结果直接合并成同一个胜率。

3.3 随机变量

随机变量把随机结果映射成数值。例如定义得分：

$$X = \begin{cases} 1, & \text{胜} \\ 0.5, & \text{和} \\ 0, & \text{负} \end{cases}$$

这样平均得分可以同时反映胜、和、负。若只计算 `wins / games`，和局与失败都会计为 0；这种定义可以使用，但必须在报告中写清楚。

回报 G 、episode 长度、单位数量和伤害也都是随机变量。

3.4 期望

离散随机变量的期望是概率加权平均：

$$\mathbb{E}[X] = \sum_x P(X = x)x$$

若胜、和、负概率分别为 0.6、0.2、0.2，则上面的期望得分为：

$$0.6 \times 1 + 0.2 \times 0.5 + 0.2 \times 0 = 0.7$$

期望 0.7 不表示某一局会得到 0.7，而表示大量同分布对局的长期平均趋近 0.7。

强化学习的目标通常写成：

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)]$$

它表示按当前策略产生轨迹时，轨迹回报的期望。训练无法枚举所有轨迹，只能用有限批轨迹估计。

3.5 方差与标准差

期望描述中心，方差描述波动：

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2]$$

标准差是方差的平方根：

$$\sigma_X = \sqrt{\text{Var}(X)}$$

两个策略平均回报相同，但一个每局接近平均，另一个在大胜与惨败之间跳动，它们的稳定性不同。训练中高方差会使优势估计噪声增大；评估中高方差需要更多对局。

数字例：样本 2, 4, 6 的均值为 4。与均值的差为 -2, 0, 2，平方为 4, 0, 4。若按总体方差除以 3：

$$\text{Var}(X) = \frac{4 + 0 + 4}{3} = \frac{8}{3}, \quad \sigma \approx 1.63$$

3.6 样本均值与标准误

评估 n 局得到样本 $X_1, \dots, X_n$ ，样本均值为：

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$$

样本均值本身也会随机变化。其标准误近似为：

$$SE(\bar{X}) = \frac{s}{\sqrt{n}}$$

局数扩大 4 倍，标准误约减半，而不是缩小到四分之一。这说明从 20 局提高到 80 局很有价值，但继续从 800 局提高到 860 局的边际收益很小。

3.7 胜率的二项近似

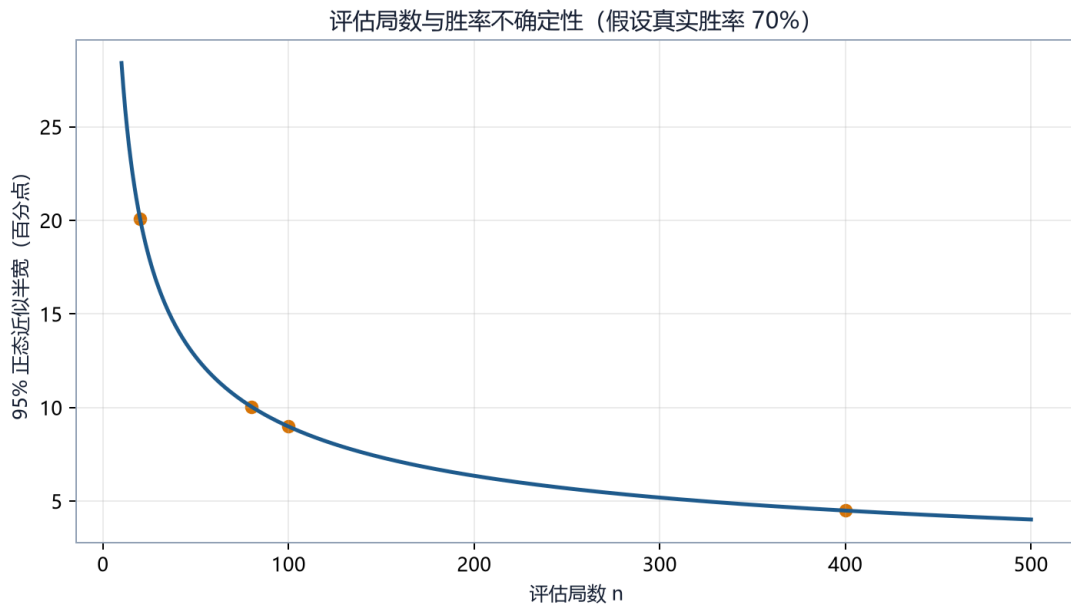
暂时忽略和局，将胜负视为 Bernoulli 变量。真实胜率为 p ，评估 n 局的胜率估计 $\hat{p}$ 的标准误近似：

$$SE(\hat{p}) = \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}}$$

若 $\hat{p} = 0.70, n = 80$ ：

$$SE \approx \sqrt{\frac{0.7 \times 0.3}{80}} \approx 0.051$$

粗略 95% 区间用 $\hat{p} \pm 1.96 SE$ ，约为 0.70 ± 0.10 。因此观测 70% 不代表真实胜率精确等于 70%。



评估局数与不确定性

在局数很少或胜率接近 0/1 时，正态近似可能不可靠。正式报告可使用 Wilson 区间或 bootstrap，但不能只显示一个百分比。

3.8 条件概率

条件概率表示已知条件后的概率：

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

例如“总体胜率 70%”可能掩盖：

条件	胜率
先手	85%
后手	55%

若训练与评估座次比例不同，总体胜率变化可能只是条件分布改变。分析时应按地图、对手、座次和终局原因分组。

3.9 独立性与伪重复

如果确定性 Bot、固定地图和固定种子产生完全相同的轨迹，重复 100 次不等于获得 100 个独立样本。样本数看似增大，实际有效信息没有增加。

历史平衡实验曾遇到重复轨迹：相同开局和确定性决胜规则使多局结果完全一致。解决方式不是只增加局数，而是引入受控随机性、交换座次并检查轨迹哈希或行为统计。

3.10 多随机种子

神经网络训练结果受初始化、采样和并行调度影响。比较算法时至少应报告多个训练种子：

$$\bar{m} = \frac{1}{K} \sum_{k=1}^K m_k$$

这里 m_k 是第 k 个训练种子的最终指标。评估局数增加只能降低“对局采样误差”，不能消除“训练种子差异”。二者必须分开。

3.11 本章练习

- 1. 策略 A 在 20 局中赢 14 局，计算胜率和近似标准误。
- 2. 为什么把相同确定性轨迹重复 100 次不能提供 100 个独立样本？
- 3. 平均回报相同的两个模型，为什么还要比较方差和终局类型？
- 4. 将评估局数从 25 提高到 100，标准误约缩小为原来的多少？

第 04 章 微积分、梯度与神经网络

4.1 从函数开始

函数把输入映射成输出：

$$y = f(x)$$

例如，线性函数 $f(x) = 2x + 1$ 在 $x = 3$ 时输出 7。神经网络也是函数，只是输入、参数和输出都可能是高维数组。

在强化学习中常见三类函数：

- 策略函数：观察 o 映射为动作概率；
- 价值函数：观察 o 映射为期望回报；
- Q 函数：观察与动作 (o, a) 映射为动作价值。

训练的本质是调整函数的参数，使损失减小或期望回报增大。

4.2 标量、向量、矩阵和张量

对象	例子	程序中的典型形状
标量	奖励 $r = 1.5$	<code>()</code>
向量	5 个全局特征	<code>(5,)</code>
矩阵	高 × 宽的地图	<code>(H, W)</code>
三阶张量	带通道的地图	<code>(H, W, C)</code>
批张量	一批地图	<code>(B, H, W, C)</code>

“维数”可能指向量长度，也可能指张量轴数。阅读代码时应直接写形状，避免只说“高维”。

向量点积：

$$\mathbf{w}^T \mathbf{x} = \sum_{i=1}^d w_i x_i$$

若 $\mathbf{x} = [2, 3]$ 、 $\mathbf{w} = [0.5, -1]$ ，点积为：

$$0.5 \times 2 + (-1) \times 3 = -2$$

神经网络的全连接层对一批输入执行矩阵乘法并加偏置：

$$\mathbf{y} = W\mathbf{x} + \mathbf{b}$$

4.3 导数表示局部变化率

导数回答：“输入稍微变化，输出怎样变化？”

$$f'(x) = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

对 $f(x) = x^2$ ，导数为 $f'(x) = 2x$ 。在 $x = 3$ 处导数为 6，表示输入增加一个很小的量 Δx 时，输出约增加 $6\Delta x$ 。

训练不要求手算极限，但需要理解导数的符号：

- 正导数：参数增大时目标局部增大；
- 负导数：参数增大时目标局部减小；
- 接近零：局部变化不敏感，可能接近极值或进入平坦区。

4.4 偏导数与梯度

损失通常依赖许多参数：

$$L = L(\theta_1, \theta_2, \dots, \theta_n)$$

对每个参数分别求偏导，组成梯度：

$$\nabla_{\theta} L = \left[\frac{\partial L}{\partial \theta_1}, \frac{\partial L}{\partial \theta_2}, \dots, \frac{\partial L}{\partial \theta_n} \right]$$

梯度指向损失局部增长最快的方向。要减小损失，梯度下降更新为：

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L$$

其中 α 是学习率。若参数 $\theta = 2$ 、梯度为 3、学习率为 0.1，则新参数为 $2 - 0.1 \times 3 = 1.7$ 。

学习率太大可能越过低点甚至发散；太小则训练缓慢。梯度是局部信息，不保证一步到达全局最优。

4.5 链式法则

神经网络由多层函数复合而成：

$$y = f(g(x))$$

链式法则：

$$\frac{dy}{dx} = \frac{df}{dg} \frac{dg}{dx}$$

数字例： $g(x) = 2x$, $f(g) = g^2$, 则 $y = (2x)^2 = 4x^2$ 。

$$\frac{df}{dg} = 2g, \quad \frac{dg}{dx} = 2, \quad \frac{dy}{dx} = 4g = 8x$$

反向传播就是从损失向前层反向应用链式法则。PyTorch 自动记录运算图并计算这些梯度。

4.6 激活函数

如果连续多层都只有线性变换，它们仍等价于一个线性变换。激活函数引入非线性。

ReLU：

$$ReLU(x) = \max(0, x)$$

双曲正切：

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Softmax 把 logits 转为离散概率：

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

若 logits 为 $[2, 1, 0]$ ，指数近似为 $[7.39, 2.72, 1]$ ，概率约为 $[0.665, 0.245, 0.090]$ 。logit 增大不等于概率等量增加，因为概率还受其他动作影响。

4.7 对数与对数概率

策略梯度使用 $\log \pi_\theta(a|o)$ 。对数有两个重要性质：

$$\log(xy) = \log x + \log y, \quad \frac{d}{dx} \log x = \frac{1}{x}$$

轨迹概率是多个动作概率的乘积，取对数后变成和，数值更稳定。对小概率动作， $\log p$ 是较大的负数：

p	$\log p$
0.9	-0.105
0.5	-0.693
0.1	-2.303
0.01	-4.605

负对数似然损失 $-\log p$ 会在正确动作概率很小时产生较大惩罚。

4.8 神经网络的一次训练

以监督目标 y 为例：

1. 前向： $\hat{y} = f_{\theta}(x)$;
2. 损失： $L(\hat{y}, y)$;
3. 清空旧梯度；
4. 反向：计算 $\nabla_{\theta} L$;
5. 优化器更新参数。

```
optimizer.zero_grad()

prediction = network(x)

loss = loss_fn(prediction, target)

loss.backward()

optimizer.step()
```

强化学习的不同之处主要在“目标从哪里来”。行为克隆有专家标签；DQN 的标签来自奖励和目标网络；PPO 的目标来自轨迹、价值估计和新旧策略概率比。

4.9 卷积与空间结构

棋盘相邻关系具有局部性。卷积层用一个小核在地图上滑动，在不同位置共享参数。相同的“附近有敌人”检测器可应用于地图各处。

展平输入会把相邻格变成向量中相邻或不相邻的索引，网络需要重新学习空间关系；卷积直接编码局部连接和平移共享。项目的 `SpatialFeatureExtractor` 将 `grid` 与 `units` 通道组合后使用卷积，并可加入坐标通道与掩码平均池化。

4.10 常见误解

- 梯度不是“正确答案”，只是当前参数附近的改进方向。
- 损失下降不保证游戏胜率上升；损失只反映训练目标。
- Softmax 概率高不表示动作合法，合法性仍需掩码。
- 网络更大不必然更强；数据分布、动作表示和奖励错误会限制上限。
- 反向传播不是强化学习特有技术，RL 只是用不同方式构造损失。

4.11 本章练习

1. 计算 $[1, 2, 3]$ 与 $[2, 0, -1]$ 的点积。
2. 若梯度为 -4 、学习率为 0.05 ，梯度下降对参数增加还是减少多少？
3. 为什么多个线性层之间必须加入非线性激活？
4. 为什么棋盘观察通常比纯展平输入更适合卷积？

第 05 章 强化学习基础

5.1 即时奖励与回报

即时奖励 R_{t+1} 只描述一次转移。智能体要最大化的是从当前开始的长期回报：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

也可写成：

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$$

符号	含义	范围
G_t	从时刻 t 开始的折扣回报	由奖励尺度决定
γ	折扣因子	通常 $0 \leq \gamma \leq 1$
k	距离当前的步数	$0, 1, 2, \dots$
T	episode 结束时刻	正整数

若奖励为 $1, 1, 1$, $\gamma = 0.9$ ：

$$G_0 = 1 + 0.9 + 0.9^2 = 2.71$$

回报不等于日志里的“episode reward” 在所有实现中的含义。监控器常记录未折扣奖励之和，而算法内部计算折扣目标。

5.2 折扣与有效时间范围

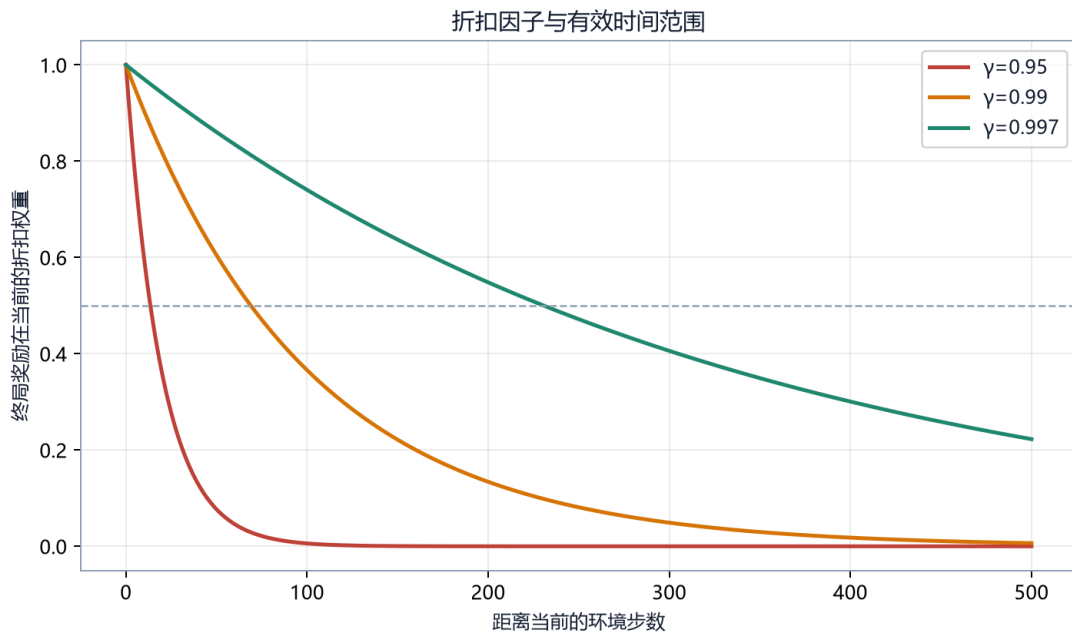
折扣有三种作用：

- 表达更早收益通常更确定；
- 控制无限序列的数学收敛；
- 决定信用能向多远传播。

常用近似有效范围：

$$H_{eff} \approx \frac{1}{1-\gamma}$$

$\gamma = 0.99$ 时约 100 步, $\gamma = 0.997$ 时约 333 步。它不是硬边界, 而是量级判断。



折扣因子与时间范围

项目一局可能包含数百微动作, 因此固定使用 0.99 会使早期状态对远端终局的直接权重很低。提高 γ 也会增加价值估计方差, 不能只看“越长期越好”。

5.3 状态价值函数

策略 π 下的状态价值:

$$V^\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$$

它表示从状态 s 出发继续按策略 π 行动的期望回报。价值是期望, 不是胜负确定预测。

若某局面有 60% 概率得到 +10、40% 概率得到 -10:

$$V^\pi(s) = 0.6 \times 10 + 0.4 \times (-10) = 2$$

实际一局不会得到 2, 但长期平均为 2。

5.4 动作价值函数

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a]$$

它表示在 s 先执行 a ，以后再按 π 行动的期望回报。DQN 直接近似 Q ；Actor-Critic 通常分别表示策略和 V 。

5.5 优势函数

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

优势衡量动作相对当前状态下策略平均水平的好坏。若 $V(s) = 5$ 、 $Q(s, a) = 8$ ，优势为 $+3$ ；若 $Q = 2$ ，优势为 -3 。

使用优势而不是原始回报可以减少“本来就处于好局面”造成的共同偏移。策略梯度据此提高正优势动作的概率、降低负优势动作的概率。

5.6 Bellman 递推

回报可拆成当前奖励与下一状态回报：

$$G_t = R_{t+1} + \gamma G_{t+1}$$

对条件期望取平均：

$$V^\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma V^\pi(S_{t+1}) | S_t = s]$$

这就是 Bellman 期望方程。它的关键不是形式复杂，而是允许用“一步奖励 + 下一状态估值”学习长期价值。

Q 函数对应：

$$Q^\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma \mathbb{E}_{a' \sim \pi} Q^\pi(S_{t+1}, a')]$$

最优 Q 的 Bellman 方程将下一个动作替换为最大值：

$$Q^*(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} Q^*(S_{t+1}, a')]$$

5.7 Monte Carlo 与 TD

Monte Carlo (MC) 等 episode 结束后用实际回报：

$$\text{target}_{MC} = G_t$$

优点是目标不依赖当前价值估计，缺点是必须等待终局且方差高。

一步 Temporal Difference (TD) 使用：

$$\text{target}_{TD} = R_{t+1} + \gamma V(S_{t+1})$$

TD 可以在线更新，方差较低，但目标依赖不完美的 V ，引入偏差。TD 误差：

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

5.8 n 步回报与 GAE

n 步目标在 MC 与一步 TD 之间折中：

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

GAE 用参数 λ 对不同长度的 TD 信息加权：

$$\widehat{A}_t^{GAE} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$$

λ 越接近 0，越接近短程 TD；越接近 1，越依赖长程实际奖励。PPO 常见 $\gamma = 0.99, \lambda = 0.95$ ，但长微动作任务需要结合 episode 长度验证。

5.9 探索与利用

利用选择当前估计最好的动作，**探索**尝试不确定动作。只利用可能过早锁定局部最优；只探索则无法稳定执行已学策略。

不同算法使用不同机制：

- DQN: ϵ -greedy;
- PPO/A2C: 从随机策略分布采样，并可加入熵奖励；
- AlphaZero: MCTS 访问分布与根节点 Dirichlet 噪声；
- 自对弈：改变对手分布形成战略探索。

策略熵：

$$\mathcal{H}(\pi) = - \sum_a \pi(a|s) \log \pi(a|s)$$

均匀分布熵高，单一动作概率接近 1 时熵低。熵系数太高会妨碍收敛，太低则容易过早确定化。

5.10 信用分配

信用分配回答：最终胜利应归因于哪些早期动作。策略游戏的困难来自：

- 因果链长；
- 同一动作在不同局面作用不同；

- 对手行动介于自己的两次决策之间；
- 奖励塑形可能提供错误捷径。

解决方案不是单一算法开关，而是组合：

- 正确的观察与动作表示；
- 合理 γ 与 GAE；
- 价值函数；
- 势能塑形；
- 课程学习；
- 分层目标；
- 高质量评估。

5.11 On-policy 与 Off-policy

On-policy 算法主要使用当前策略刚产生的数据。PPO、A2C 属于此类；策略更新后旧数据很快失效，样本利用率较低但更新定义清晰。

Off-policy 算法可以重复使用旧策略数据。DQN 使用经验回放；样本利用率较高，但旧数据分布和当前策略不同，需要目标网络等稳定机制。

5.12 本章练习

1. 计算奖励 2, 0, 3、 $\gamma = 0.9$ 时的 G_0 。
2. 若 $V = 7, Q = 4$ ，优势是多少，策略应倾向提高还是降低该动作概率？
3. 说明 MC 与一步 TD 的偏差—方差差别。
4. 为什么提高 γ 不能自动解决长局训练？

第 06 章 用于强化学习的 PyTorch

6.1 张量与形状

PyTorch 的核心对象是 `torch.Tensor`。张量同时保存数值、数据类型、设备和可选梯度信息。

```
import torch

x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])

print(x.shape)    # torch.Size([2, 2])

print(x.dtype)    # torch.float32

print(x.device)   # cpu
```

强化学习中最常见的错误不是公式，而是形状：

- 环境返回 `(H,W,C)`，CNN 需要 `(B,C,H,W)`；
- 单个动作是 `(6,)`，批动作是 `(B,6)`；
- 布尔掩码长度必须与对应 logits 完全一致；
- 价值输出通常是 `(B,1)`，优势可能是 `(B,)`。

项目特征提取器会把批维和通道维整理为卷积所需格式。阅读 `permute`、`reshape`、`unsqueeze` 时，应在注释旁写出变换前后的形状。

6.2 自动求导

```
x = torch.tensor(3.0, requires_grad=True)

y = x**2

y.backward()

print(x.grad)    # 6
```

PyTorch 在前向计算时记录运算图；`backward()` 从输出向参数应用链式法则。只有参与图且 `requires_grad=True` 的叶张量会积累梯度。

梯度默认累积，因此训练循环必须在每次更新前清零：

```
optimizer.zero_grad()

loss.backward()

optimizer.step()
```

忘记清零会把多批梯度意外相加；在有意做梯度累积时才保留。

6.3 nn.Module

网络继承 `torch.nn.Module`：

```
from torch import nn

class ValueNet(nn.Module):

    def __init__(self, input_dim: int):

        super().__init__()

        self.net = nn.Sequential(

            nn.Linear(input_dim, 128),

            nn.ReLU(),

            nn.Linear(128, 1),

        )

    def forward(self, x):

        return self.net(x)
```

`model.parameters()` 返回优化器要更新的参数；`state_dict()` 保存具名权重。checkpoint 若只保存 `state_dict`，可能遗漏优化器、训练步数、地图尺寸和架构超参数。本项目的 Feudal 与 AlphaZero checkpoint 因此还要保存 `hyperparameters` 和训练状态。

6.4 训练模式与评估模式

```
model.train()

# 更新参数

model.eval()

with torch.no_grad():
    prediction = model(x)
```

`eval()` 会改变 dropout、batch normalization 等层的行为；`torch.no_grad()` 则关闭梯度记录以减少内存。二者作用不同，评估时通常都需要。

6.5 优化器

Adam 为每个参数维护一阶和二阶矩估计，常用于深度强化学习：

```
optimizer = torch.optim.Adam(model.parameters(), lr=3e-4)
```

学习率仍是最重要的尺度之一。PPO 一次 rollout 会重复训练多个 epoch；学习率、epoch 数、batch size 和 clip range 共同决定一次采样数据引起多大变化。

梯度裁剪：

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=0.5)
```

它限制整体梯度范数，缓解异常批次造成的巨大更新，但不能修复错误奖励尺度或 NaN 输入。

6.6 概率分布

离散策略常用 `torch.distributions.Categorical`：

```
from torch.distributions import Categorical

logits = torch.tensor([[2.0, 1.0, 0.0]])

dist = Categorical(logits=logits)

action = dist.sample()

log_prob = dist.log_prob(action)

entropy = dist.entropy()
```

PPO 需要保存旧策略对已执行动作的 `log_prob`，更新时再用新策略计算同一动作的 `log_prob`。二者相减后取指数即可得到概率比：

$$\frac{\pi_{new}(a|s)}{\pi_{old}(a|s)} = \exp(\log \pi_{new} - \log \pi_{old})$$

6.7 动作掩码在 logits 层生效

非法动作应在 softmax 前赋极小 logit：

```
masked_logits = logits.masked_fill(~mask, -1e8)

dist = Categorical(logits=masked_logits)
```

这样非法动作概率近似为 0，合法动作重新归一化。若所有掩码均为 False，分布没有可选动作，因此项目保证 `end_turn` 始终可用。

逐维 MultiDiscrete 会为六个维度创建六个独立分布。即使每个维度值合法，组合仍可能非法；这是第十一章的核心问题。

6.8 项目的空间特征提取

`reinforcetactics/r1/extractors.py` 中的 `SpatialFeatureExtractor` 接收：

- `grid`：地形、建筑和归属通道；
- `units`：单位类型、归属、生命和状态通道；
- `global_features`：金币、回合和单位数量等摘要。

空间通道拼接后转为 `(B,C,H,W)`，经多层卷积提取局部关系。可选设计包括：

- `coord_conv` : 加入归一化横纵坐标, 使网络区分绝对位置;
- `extra_conv` : 扩大感受野;
- `masked_avg` : 跨地图 padding 时忽略填充格;
- 固定 `features_dim` : 向策略和价值头提供稳定维度。

普通平均池化在 6×6 地图填充到 10×12 时, 会把大量零填充纳入平均。掩码平均只统计真实地图区域, 减少阶段切换造成的尺度变化。

6.9 数值稳定性

需要重点监控:

- logits 是否出现 `inf` 或 `nan` ;
- value loss 是否被大终局奖励放大;
- `log(0)` 是否通过稳定分布实现避免;
- 观察是否归一化到合理范围;
- 掩码后是否至少有一个动作;
- 梯度范数是否持续撞上裁剪上限。

项目全局特征使用缩放和 `tanh` , 避免金币数千而单位数量个位数造成输入尺度悬殊。归一化是表示设计的一部分, 不是单纯“让网络更快”。

6.10 CPU 冒烟

```
python -c "import torch; from reinforcetactics.rl.extractors import SpatialFeatureExtractor; print(torch.__version__, torch.cuda.is_available())"
```

本书基准输出为 CPU 版本且 `cuda=False` 。所有短实验应显式允许 CPU; 生产长训才需要结合硬件选择批大小和并行环境数。

6.11 本章练习

1. (8,10,12,32) 若代表 (B,H,W,C) , 怎样变成 CNN 常用形状?
2. `model.eval()` 与 `torch.no_grad()` 有何不同?
3. 为什么梯度裁剪不能修复奖励正负号错误?
4. 为什么跨地图 padding 时掩码平均优于普通平均?

第 07 章 Gymnasium 环境契约

7.1 环境的最小接口

Gymnasium 将算法与领域规则隔离。算法只需要空间定义以及 `reset`、`step`：

```
obs, info = env.reset(seed=42)

obs, reward, terminated, truncated, info = env.step(action)
```

`StrategyGameEnv` 继承 `gymnasium.Env`。创建后应满足：

- `obs` 属于 `observation_space`；
- `action` 属于 `action_space`；
- `reward` 可转换为标量浮点数；
- `terminated`、`truncated` 是布尔值；
- `info` 是诊断字典，不应被策略依赖为隐藏观察。

7.2 reset

`reset(seed=...)` 开始新 episode，重建游戏状态、对手、计数器和奖励势能基线。第一次重置通常应调用父类种子逻辑，以创建环境自己的随机数生成器。

```
obs, info = env.reset(seed=7)
```

同一种子可帮助重现环境随机序列，但不能保证 GPU 并行训练逐位一致。项目还包含游戏引擎 RNG、Bot RNG、NumPy、Python 和 PyTorch 随机源，完整复现必须统一记录。

7.3 step

一次环境步的顺序可概括为：

1. 解码动作；
2. 按规则执行或判定无效；

3. 更新动作统计和即时奖励；
4. 若智能体结束回合，让对手完成回合；
5. 检查规则终局和外部步数上限；
6. 构造下一观察、奖励分解和诊断信息。

调用方循环：

```
obs, info = env.reset(seed=42)

while True:

    action = env.action_space.sample()

    obs, reward, terminated, truncated, info = env.step(action)

    if terminated or truncated:

        break
```

随机采样没有使用合法动作掩码，因此可能产生无效动作；它只能验证接口，不是合理基线。

7.4 观察空间

默认观察是 `spaces.Dict`：

```
grid            (H, W, GRID_CHANNELS)
units           (H, W, UNIT_CHANNELS)
global_features (5,)
visibility       (H, W) # 仅战争迷雾
```

空间声明是契约。若 `Box` 声明范围 `[0,1]`，实际观察就不应出现未缩放金币 5000。
`gymnasium.utils.env_checker.check_env` 可以发现形状、dtype 和返回值错误。

7.5 动作空间

默认：

```
MultiDiscrete([10, 8, W, H, W, H])
```

六个字段依次表示动作类型、单位类型、起点横纵坐标、目标横纵坐标。

Flat Discrete 模式：

```
Discrete(max_flat_actions)
```

每个索引在当前步骤映射到一条完整合法动作。两种模式的网络输出形状不同，模型不可互换。

7.6 终止与截断

Gymnasium 明确区分：

- `terminated=True`：任务定义内的终局；
- `truncated=True`：任务外条件提前停止，常见为时间限制。

价值目标在规则终止处不 bootstrap，在纯截断处通常要 bootstrap。官方 Gymnasium 文档专门强调这一差别，因为旧版 `done` 无法直接表达。

项目中：

原因	标志	含义
总部占领、消灭	<code>terminated</code>	规则终局
<code>max_turns</code> 和局	<code>terminated</code>	规则定义的和局
<code>max_steps</code>	<code>truncated</code>	收集器截断

将 `max_steps` 截断再额外施加 `draw` 惩罚，可能与算法的时间限制 bootstrap 同时作用，形成双重边界效应。当前环境把截断奖励设为显式可选项。

7.7 info 的作用

`info` 应保存诊断信息，例如：

- `winner`、`game_over`、`end_reason`；
- `reward_breakdown`；
- `n_legal_actions`；
- 单位创建、伤害、治疗和占领统计；
- episode 长度和累计奖励。

训练策略不应把只存在于 `info` 的隐藏敌方信息作为输入，否则训练和部署接口不一致。评估器可以使用它分解结果。

7.8 Wrapper

Wrapper 在不改领域环境的情况下增加行为。项目中的 `ActionMaskedEnv` 和自对弈环境都是 wrapper 或类似包装层。常见用途：

- 统一动作掩码接口；
- 记录 episode 指标；
- 向量化多个环境；
- 翻转玩家相对观察；
- 注入对手快照。

Wrapper 必须正确转发属性和 `reset/step`。多层 wrapper 下直接访问 `env.game_state` 可能失败，应通过框架提供的属性转发或 `unwrapped` 谨慎访问。

7.9 向量环境

`VecEnv` 同时推进多个独立环境，收集形状为 `(n_envs, ...)` 的批数据。若 PPO 的 `n_steps=2048, n_envs=8`，每次 rollout 收集：

$$2048 \times 8 = 16384$$

条转移。配置中的 `eval_freq`、`callback` 调用次数和实际总环境步之间可能因此出现倍数关系。解释训练日志必须知道指标按“算法调用”还是“总环境步”计数。

7.10 PettingZoo 边界

PettingZoo 为多智能体环境定义 AEC 和 Parallel API。当前项目虽依赖该包，但主环境仍是 `Gymnasium` 单智能体包装：对手回合发生在环境内部。若未来实现真正多智能体训练，需要新环境接口，而不是仅把 `opponent="self"` 称为 PettingZoo。

7.11 验证命令

```
python -c "from gymnasium.utils.env_checker import check_env; from
reinforcetactics.rl.gym_env import StrategyGameEnv;
e=StrategyGameEnv(map_file='maps/1v1/starter.csv',opponent='noop',max_steps=16);
check_env(e); print('check_env passed'); e.close()"
```

`check_env` 通过只说明 Gymnasium 契约基本一致，不验证奖励是否合理、掩码是否精确或 Bot 是否有竞争力。

7.12 本章练习

1. 写出当前 Gymnasium `step` 的五个返回值。
2. 为什么 `max_turns` 和 `max_steps` 在项目中采用不同结束标志？
3. `n_steps=512, n_envs=4` 时每次 PPO rollout 有多少条转移？
4. 为什么环境契约通过仍不能证明训练问题建模正确？

第 08 章 Stable-Baselines3 训练框架

8.1 框架负责什么

Stable-Baselines3 (SB3) 实现常用深度强化学习算法和训练基础设施。它负责：

- 策略、价值或 Q 网络；
- rollout buffer 或 replay buffer；
- 批处理、反向传播和优化器；
- 模型保存与加载；
- callback、日志与向量环境；
- `learn` 和 `predict` 的统一接口。

项目仍需负责环境、观察、动作、奖励、对手与评估。框架不会判断奖励是否鼓励真实胜利，也不会自动修复非法动作表示。

8.2 模型构造

```
from stable_baselines3 import PPO

model = PPO(
    "MultiInputPolicy",
    env,
    learning_rate=3e-4,
    n_steps=2048,
    batch_size=64,
    n_epochs=10,
    gamma=0.99,
    gae_lambda=0.95,
    clip_range=0.2,
    verbose=1,
)
```

`MultiInputPolicy` 用于 `spaces.Dict` 观察。它不是算法名称，而是策略类选择。`policy_kwargs` 可以指定网络结构和自定义特征提取器。

8.3 learn

```
model.learn(total_timesteps=100_000, callback=callbacks)
```

对 on-policy 算法，`learn` 循环收集 rollout、计算优势、进行多轮小批优化，再收集新数据。`total_timesteps` 是目标环境步数；向量环境和固定 rollout 长度可能使最终计数略超过目标。

PPO 的每次更新数据量：

$$N_{rollout} = n_{steps} \times n_{envs}$$

若 batch size 不能整除该值，最后出现较小批次并产生警告。短实验应把 `batch_size` 设为 rollout 大小的因数。

8.4 predict

```
action, state = model.predict(obs, deterministic=True)
```

`deterministic=True` 通常选择分布众数或最大 Q 动作，适合稳定评估；训练时算法会自行采样。对于 MaskablePPO，预测还需提供当前掩码：

```
action, state = model.predict(  
    obs,  
    action_masks=env.action_masks(),  
    deterministic=True,  
)
```

若训练使用掩码而评估忘记传掩码，模型可能在评估时选择非法动作，造成“训练正常、评估很差”的测量错误。

8.5 On-policy rollout buffer

PPO/A2C 的 buffer 保存一段当前策略数据：

```
observation, action, reward, done,  
value, log_probability, advantage, return
```

更新后通常清空，继续用新策略收集。旧 `log_probability` 用于计算 PPO 新旧概率比；`value` 用于 GAE。

8.6 DQN replay buffer

DQN 保存：

```
(observation, action, reward, next_observation, done)
```

训练时随机采样旧转移，打破相邻样本相关性并重复利用数据。由于数据来自旧策略，DQN 是 off-policy。经验回放不适用于普通 PPO，因为 PPO 目标假设数据来自旧策略的已知快照且只进行受限更新。

8.7 Monitor 与日志

Monitor 记录 episode 回报、长度和额外信息。TensorBoard 展示：

- rollout/ep_rew_mean、ep_len_mean；
- train/policy_gradient_loss；
- train/value_loss；
- train/entropy_loss；
- train/approx_kl；
- train/clip_fraction；
- time/fps 和步数。

这些指标必须结合环境结果解释。价值损失下降不等于胜率上升；平均回报上涨可能来自塑形刷分；熵降低可能是合理收敛，也可能是过早坍缩。

8.8 Callback

Callback 在训练事件上执行逻辑：

- 周期评估；
- checkpoint 保存；
- 课程晋级；
- 熵系数调度；
- 指标记录；
- 早停或停滞异常。

回调频率容易受 `n_envs` 影响。应在代码中确认 `n_calls`、`num_timesteps` 和实际环境步的关系，而不是只看配置名。

项目的 `PeriodicEvalCallback` 和 `PromotionCallback` 进一步记录胜负、终局原因、动作分布和晋级状态。

8.9 模型保存与环境兼容

SB3 `.zip` 通常保存策略参数、算法超参数和空间信息。加载时环境必须兼容：

- 观察键、形状和 dtype；
- 动作空间类型与尺寸；
- 自定义特征提取器导入路径；
- 掩码接口；
- 地图 padding 与单位集合。

MultiDiscrete 模型不能直接放到 Flat Discrete 环境，因为输出头完全不同。仅文件能加载不代表环境语义相同；奖励和对手变化也会影响评估。

8.10 sb3-contrib

实验性或扩展算法位于 `sb3-contrib`。项目主线使用 `MaskablePPO`。官方接口要求环境暴露 `action_masks()`；在 `SubprocVecEnv` 下，掩码必须在环境内部实现，不能只靠主进程包装函数。

项目提供 `make_maskable_env` 和 `make_maskable_vec_env` 统一构建环境，避免不同训练脚本漏传参数。

8.11 配置优先级

项目高级训练脚本通常采用：

1. `dataclass` 默认；
2. YAML/JSON 配置；
3. CLI 显式覆盖。

最终运行配置必须落盘。只保存原始 YAML 不够，因为 CLI 可能改过值；只保存命令也不够，因为默认值可能随版本变化。`run_config` 一类工具应记录合并后的配置、版本、Git 提交、设备和种子。

8.12 最小接口检查

```
python -c "from sb3_contrib import MaskablePPO; from reinforcetactics.rl.masking
import make_maskable_env;
e=make_maskable_env(map_file='maps/1v1/starter.csv',opponent='noop',action_space_t
ype='flat_discrete',max_flat_actions=128);
m=MaskablePPO('MultiInputPolicy',e,n_steps=32,batch_size=32,n_epochs=1,device='cpu
',verbose=0); m.learn(64); print('learn passed'); e.close()"
```

这条命令验证 64 步训练循环，不评价策略强度。

8.13 本章练习

1. `MultiInputPolicy` 与 PPO 的关系是什么？
2. 为什么 PPO 不能像 DQN 一样任意重复使用很久以前的数据？
3. 训练使用动作掩码时，评估为何也必须传掩码？
4. 模型文件能加载时，还需要检查哪些环境兼容项？

第 09 章 项目运行时与数据流

9.1 分层结构

项目不是“一个训练脚本加一个环境”。主要层次如下：

层	主要职责	代表模块
入口	参数解析与模式分发	main.py 、 reinforcetactics/cli
应用	GUI 会话、输入和 Bot 工厂	reinforcetactics/app
领域	游戏状态、单位、地图和规则	reinforcetactics/core 、 game
RL 环境	观察、动作、奖励、掩码	reinforcetactics/rl/gym_env.py
训练	PPO、课程、自对弈、Feudal、AlphaZero	reinforcetactics/rl 、 scripts/train
评估	模型评估、锦标赛、Elo	rl/evaluation.py 、 tournament

从训练算法到游戏状态的数据流



项目数据流

领域层应保持为规则真源。RL 环境可以编码和计分，却不应偷偷改变移动、攻击或占领规则。

9.2 CLI 的四种模式

`main.py` 提供 `train`、`evaluate`、`play` 和 `stats`。

```
python main.py --mode play

python main.py --mode train --algorithm ppo --timesteps 100000

python main.py --mode evaluate --model models/ppo_final.zip --episodes 20

python main.py --mode stats
```

基础 CLI 适理解入口，但不是当前最可靠的生产训练路径：

- `ppo` 使用普通 PPO，不自动使用动作掩码；
- `a2c` 同样没有掩码；
- `dqn` 默认创建 `MultiDiscrete` 环境，而 `SB3 DQN` 只接受 `Discrete`，当前会在模型构造处失败；
- 课程、BC、自对弈、Feudal 和 AlphaZero 各有独立脚本。

教材不会把“参数可选”误写成“训练路径已验证”。

9.3 GameState 的职责

`GameState` 保存并改变：

- 地图与建筑；
- 单位及其生命、位置和状态；
- 当前玩家、回合和金币；
- 合法动作；
- 移动、攻击、治疗、技能和占领；
- 胜负与最大回合；
- 可选的规则常量覆盖。

GUI 人类操作、Bot 和环境最终都应调用这些领域操作。例如环境解码出 `move` 后，仍由规则层验证移动。训练环境不能假设掩码永远正确；执行时必须再次校验。

9.4 StrategyGameEnv.__init__

构造函数把游戏实例包装为 RL 任务。关键参数：

参数	作用
<code>map_file</code>	固定地图；为空时随机生成
<code>opponent</code>	规则 Bot、随机、noop、自对弈或空
<code>max_steps</code>	外部环境步上限
<code>max_turns</code>	游戏规则回合上限
<code>reward_config</code>	奖励权重覆盖
<code>enabled_units</code>	可创建兵种集合
<code>fog_of_war</code>	部分可观测模式
<code>action_space_type</code>	<code>multi_discrete</code> 或 <code>flat_discrete</code>
<code>max_flat_actions</code>	Flat 模式固定上限
<code>max_actions_per_turn</code>	防止永不结束回合
<code>gamma</code>	势能塑形折扣，应与训练器一致
<code>pad_to_size</code>	跨地图统一观察尺寸
<code>engine_overrides</code>	平衡实验覆盖

构造函数固定 RL 环境为 1v1；GUI 的多人模式不经过同一观察契约。

9.5 reset 数据流

重置时必须同时恢复领域状态和训练辅助状态：

加载初始地图

- 创建新的 GameState
- 初始化/重新绑定对手
- 确定 agent_player
- 清零步数、动作与 episode 统计
- 初始化势函数 $\Phi(s_0)$
- 构建 flat 合法动作（若启用）
- 返回观察和 info

势能塑形的 `_prev_potential` 必须设为初始状态的 $\phi(s_0)$ ，不能简单设为 0；否则 episode 第一步会凭空得到 $\gamma\phi(s_1)$ 。

9.6 step 数据流

环境每步大致执行：

输入动作

- 根据动作空间解码为六字段动作
- 执行领域动作
- 记录无效、伤害、治疗、购买、占领等事件
- 计算本方动作即时奖励
- 如果 `end_turn`：运行对手完整回合并记录对手事件
- 检查规则终局 / `max_steps`
- 计算势能变化和终局奖励
- 构建下一观察、掩码所需合法动作和 `info`

动作执行和奖励计算不能随意交换。若在对手回合前就计算完整势能差，就会漏掉对手夺取建筑和造成伤害；若终局先关闭势能而后又重复计算，奖励分解会不守恒。

9.7 对手回合

智能体选择 `end_turn` 后：

1. `GameState.end_turn()` 把控制权给对手；
2. 环境调用对手 `take_turn()`；
3. Bot 内部执行多个领域动作并结束自己的回合；
4. 控制权回到智能体。

历史实现曾出现环境在 Bot 已结束回合后再调用一次 `end_turn`，导致玩家被切换两次。此类错误会让观察与当前玩家不一致，训练表现看似随机。当前逻辑应检查 Bot 执行后 `current_player`，只在仍未归还时补救。

9.8 自对弈工厂

`opponent="self"` 本身不包含对手网络。训练脚本通过：

```
set_self_play_opponent_factory(factory)
```


注入一个函数，根据新的 `GameState` 和对手玩家创建快照 Bot。每次 `reset` 都要重新绑定到新状态，不能让 Bot 保留上一个 episode 的 `GameState` 引用。

9.9 配置流

高级训练使用 `reinforcetactics/rl/config.py` 的 `dataclass`:

```
EnvConfig
PPOConfig
FeudalConfig
SelfPlayConfig
AlphaZeroConfig
CurriculumStage / CurriculumConfig
EvalConfig / LoggingConfig
TrainingConfig
```

`load_config` 读取 YAML/JSON, `apply_overrides` 应用点号路径 CLI 覆盖。训练器最终得到一个合并后的 `TrainingConfig`。记录实验时必须保存这一最终对象，而非只保存原始 YAML。

9.10 数据形状穿过各层

```
GameState 对象
  ↓ build_observation
Dict[str, np.ndarray]
  ↓ VecEnv
Dict[str, np.ndarray], 带批维 B
  ↓ features extractor
Tensor[B, features_dim]
  ↓ 策略/价值头
动作分布 + V(o)
  ↓ sample / predict
NumPy 动作
  ↓ env.step
领域动作
```

调试时沿这条链逐层打印形状，比直接查看最终 loss 更有效。

9.11 失败归属示例

现象	优先检查层
动作索引越界	动作编码/环境
训练 loss 为 NaN	观察、奖励尺度、网络
只会结束回合	掩码、奖励、探索和对对手
训练胜率高、GUI 不能加载	空间/特征提取器/checkpoint
换地图构造失败	observation/action shape
同模型重复评估结果完全相同	对手 RNG 与种子

9.12 本章练习

1. 为什么 Bot 必须在每次 `reset` 后绑定新的 `GameState` ?
2. 势能基线若错误设为 0, 第一步奖励会出现什么偏差?
3. 基础 CLI 的 DQN 为什么不是当前可直接运行的默认路径?
4. 说明从 `GameState` 到策略 logits 的数据形状链。

第 10 章 观察编码与空间特征

10.1 观察设计决定可学信息

神经网络只能使用观察中出现的信息。奖励再合理、算法再先进，也无法弥补关键状态缺失。反过来，把无关或泄漏的信息加入观察，会让训练指标虚高、部署失败。

项目观察为字典：

$$o_t = \{\text{grid}, \text{units}, \text{global_features}, [\text{visibility}]\}$$

字典观察允许分别处理空间平面和全局向量。

10.2 grid

grid 的形状为：

`(H, W, GRID_CHANNELS)`

通道表达地形、建筑类别、相对归属和结构生命等。使用 one-hot 或归一化通道而不是把所有类别压成一个整数，有两个优点：

- 数字大小不暗示不存在的顺序关系；
- 卷积可以分别学习“水域”“总部”“敌方归属”等检测器。

若用地形编号 1、2、3，网络会把编号 3 当作比 1 “大”；one-hot 不引入这种错误几何。

10.3 units

units 也是 `(H,W,C)`，包含：

- 单位类型 one-hot；
- 自己/对手相对归属；
- 是否已行动；
- 生命比例；
- 麻痹、加速、防御和攻击增益等状态。

“是否已行动”是马尔可夫状态的重要部分。若只编码单位位置和类型，相同画面下合法动作可能不同。

10.4 global_features

五维全局摘要包括双方金币、回合与双方单位数。原始数值量级差异很大，因此构造观察时按 `gold_scale`、`turn_scale` 和 `unit_count_scale` 缩放，再通过 `tanh` 映射到稳定范围。

例如金币特征可写为：

$$x_{gold} = \tanh\left(\frac{gold}{c_{gold}}\right)$$

若缩放常数为 1000，金币 100 的输入约 0.10，金币 5000 接近饱和。缩放常数应覆盖课程中的典型经济范围；太小会让大量状态挤在 1 附近，太大则差异过小。

10.5 玩家相对视角

观察不是固定用“玩家 1/玩家 2”通道，而是“自己/对手”。这使同一策略可以控制不同座次：

```
绝对 player 1 / player 2
      ↓ perspective_player
相对 self / opponent
```

自对弈翻转视角时，还要交换金币、归属、可见性和当前玩家含义。只交换单位 `owner` 而不交换全局特征会产生内部矛盾。

10.6 战争迷雾

启用 `fog_of_war` 后增加可见性平面，并隐藏不可见敌军。需要同时检查：

- 隐藏单位是否从 `units` 移除；
- 合法动作是否泄漏隐藏目标；
- LLM 序列化是否泄漏；
- 自对弈翻转后是否用正确玩家可见性；
- `info` 中的诊断是否被策略误用。

普通无记忆策略面对同一观察无法区分不同隐藏局面。若研究战争迷雾，应考虑循环策略、观察历史或信念建模，而不是只打开布尔开关后沿用所有结论。

10.7 跨地图 padding

同一个神经网络要求固定输入形状。课程从 6×6 到 10×12 时，可把小地图补零到最大尺寸：

真实 6×6 区域 + 其余 padding

`pad_to_size=(pad_h,pad_w)` 不能小于当前地图。当前实现只对 Flat Discrete 模式完整支持，因为 MultiDiscrete 的动作维度本身包含 W, H ，仅填充观察仍不能统一动作空间。

padding 需要配套：

- 空间通道填零；
- 坐标通道按统一规则生成；
- 池化忽略填充格；
- 合法动作只指向真实地图；
- LLM 或 GUI 坐标转换保持原地图坐标。

10.8 默认展平的代价

SB3 的普通组合提取器会将空间张量展平，再送入线性层。输入维度随地图面积增长：

$$D = H \times W \times (C_{grid} + C_{units})$$

第一层参数约为 $D \times d_{hidden}$ 。地图扩大时参数数量增长，且相邻关系没有显式结构。

10.9 SpatialFeatureExtractor

空间提取器的概念流程：

```
grid + units
→ 通道拼接
→ (B,H,W,C) 转 (B,C,H,W)
→ 可选坐标通道
→ 多层 3×3 卷积
→ masked average / average / flatten
→ 拼接 global_features
→ 线性投影到 features_dim
```

坐标卷积解决“卷积平移共享但总部角落具有绝对意义”的矛盾。额外卷积扩大感受野，使高层特征能覆盖更大邻域。掩码平均使小地图的特征不被零填充稀释。

10.10 观察检查

```
python -c "from reinforcetactics.rl.gym_env import StrategyGameEnv;
e=StrategyGameEnv(map_file='maps/1v1/beginner.csv',opponent='noop');
o,_=e.reset(seed=42); print({k:(v.shape,v.dtype,float(v.min()),float(v.max())) for
k,v in o.items()}); e.close()"
```

应检查每个键的形状、dtype、最小最大值以及 `observation_space.contains(obs)`。只检查 shape 不足以发现越界或 NaN。

10.11 历史问题：空间盲性

2026-07-24 的管线审查指出，早期生产配置使用默认展平网络，策略需要从头学习地图邻接关系。后续配置引入 `SpatialFeatureExtractor`、坐标通道、更深卷积和掩码池化。

这个修复提高了表示能力，但不能单独证明胜率提升。正确实验需要保持动作空间、奖励、对手、种子和训练预算不变，只替换特征提取器，并进行多种子对比。

10.12 本章练习

1. 为什么单位“是否已行动”必须进入观察？
2. one-hot 地形相比单整数编码避免了什么错误假设？
3. 为什么 padding 观察不能自动解决 MultiDiscrete 跨地图动作空间变化？
4. 设计一个检查战争迷雾信息泄漏的测试。

第 11 章 动作空间与动作掩码

11.1 六字段动作

默认动作向量：

$$a = (t, u, x_s, y_s, x_d, y_d)$$

字段	含义	典型范围
t	动作类型	0-9
u	创建单位类型	0-7
x_s, y_s	起点	地图坐标
x_d, y_d	目标点	地图坐标

动作类型包括创建、移动、攻击、占领、治疗、结束回合以及多种技能。并非所有字段对每类动作有意义，例如 `end_turn` 不需要坐标，但固定长度便于统一接口。

11.2 组合空间

6×6 地图的完整组合数：

$$10 \times 8 \times 6 \times 6 \times 6 \times 6 = 103680$$

某一步真正合法动作常只有几十条。随机采样几乎必然无效。仅给无效动作负奖励会浪费样本，并使惩罚淹没游戏信号。

11.3 逐维掩码

MultiDiscrete MaskablePPO 为每个维度提供独立布尔掩码。例如合法动作只有：

```

move    from (1,0) to (2,0)
attack from (3,2) to (4,2)
end_turn

```

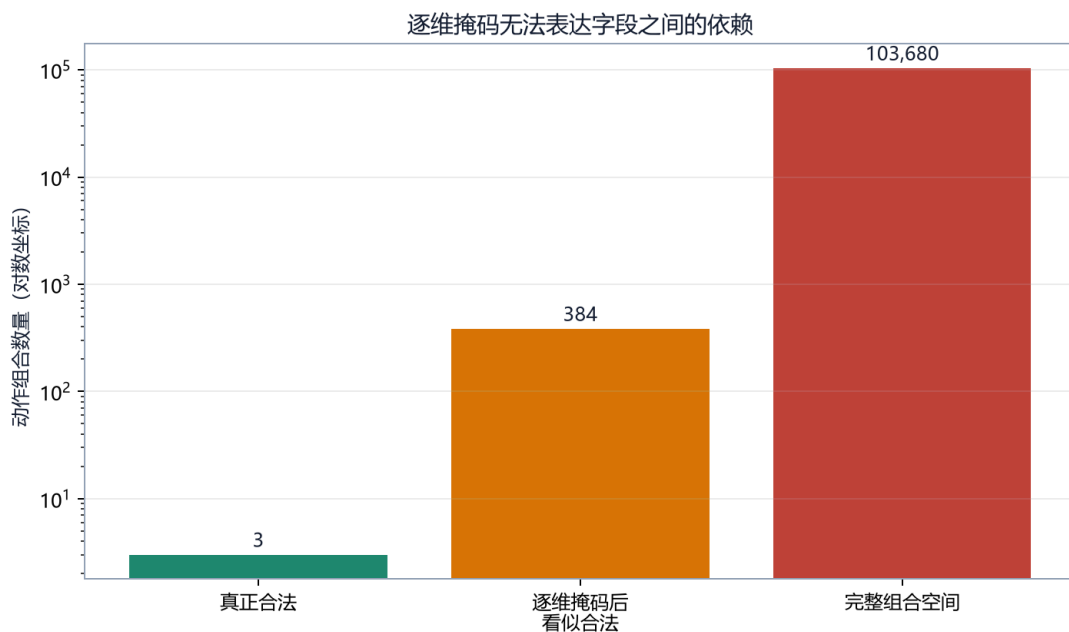
逐维掩码会分别允许：

```

type: move, attack, end_turn
from_x: 1, 3
from_y: 0, 2
to_x: 2, 4
to_y: 0, 2

```

独立分布可重组出 `move from (3,0) to (4,2)` 等从未存在的动作。掩码保证每个字段值曾在某条合法动作出现，却不能保证字段组合合法。



逐维掩码过度近似

若上述掩码还允许 8 个单位类型，近似组合为：

$$3 \times 8 \times 2 \times 2 \times 2 \times 2 = 384$$

真正合法只有 3 条，表面“通过掩码”的组合约 99.2% 仍无效。

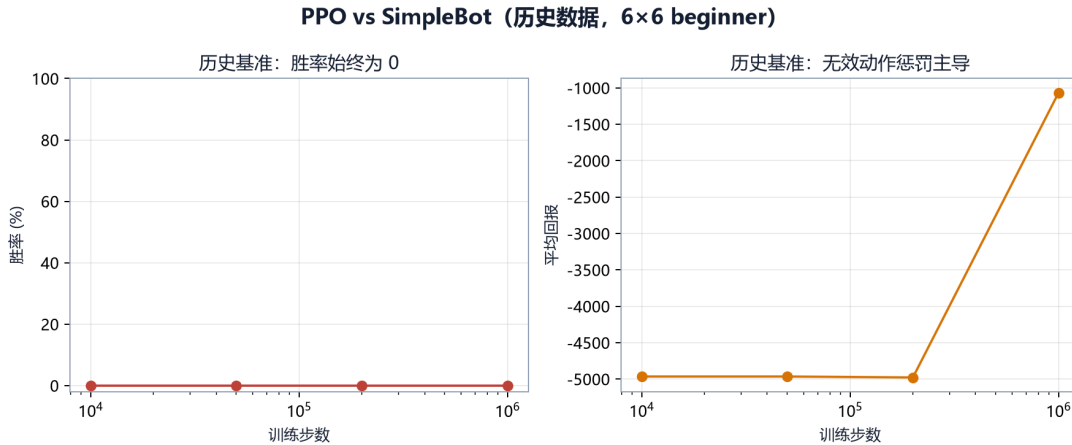
11.4 历史基准证据

历史 PPO vs SimpleBot 基准在 10K、50K、200K 和 1M 步时胜率均为 0。前 200K 步 episode 常达到 500 步截断，平均回报约 -4965；1M 步后模型学会频繁 `end_turn`，平均局长降到约 10，但全部失败。

如果每局 500 步、无效率约 99%、无效惩罚 -10：

$$500 \times 0.99 \times (-10) = -4950$$

这与观测回报接近，说明训练主要在学习避免接口惩罚，而不是游戏策略。



历史掩码失败基准

后期 `end_turn` 坍塌也有因果解释：它始终合法，可以避免 -10；但对手得到连续发展机会，模型迅速失败。

11.5 Flat Discrete 精确掩码

Flat 模式在每步枚举完整合法动作：

```
_current_actions = [
    [create, unit_type, x, y, x, y],
    [move, _, sx, sy, dx, dy],
    ...
    [end_turn, _, 0, 0, 0, 0],
]
```

动作空间固定为 `Discrete(max_flat_actions)`。前 `len(_current_actions)` 个索引对应真实动作，其余 padding 索引为 False。策略采样一个索引即可得到完整组合，因此掩码是精确的。

优势：

- 消除字段重组无效动作；
- DQN 也能使用 Discrete；
- 固定 `max_flat_actions` 可与观察 padding 共同支持跨地图。

代价：

- 索引语义随当前合法动作列表变化；
- 上限太小会截断合法动作；
- 上限太大使策略输出头变大；
- 若列表排序不稳定，同一索引在相似状态中含义变化剧烈。

2026-07-24 审查特别指出 Flat 动作索引按当前列表位置重建，策略难以形成稳定动作语义。精确合法性不等于理想表示。

11.6 上限与截断

`max_flat_actions` 必须覆盖课程最密集局面。若实际合法动作数超过上限，简单截断可能系统性丢掉排序靠后的单位或动作。

正确做法：

1. 在代表地图与高单位密度下记录 `n_legal_actions`；
2. 观察最大值和高分位数；
3. 检查截断日志；
4. 明确动作排序；
5. 必要时提高上限或改用结构化动作网络。

11.7 自回归动作头

自回归分解按条件选择：

$$p(a|s) = p(t|s) p(u|t, s) p(x_s, y_s|t, u, s) p(x_d, y_d|t, u, x_s, y_s, s)$$

先选动作类型，再根据类型生成相关字段掩码；选定起点后，目标掩码只包含该单位可执行目标。这样既保留结构语义，又能表达字段依赖。

项目 Feudal AR Worker 使用 `structured_action_masks()` 提供阶段条件掩码。代价是采样、log probability、entropy、训练评估和 batch 化都更复杂；任何阶段掩码错位都会破坏 PPO 概率比。

11.8 掩码概率

给合法指示 $m_i \in \{0, 1\}$:

$$\pi'(a = i|s) = \frac{m_i e^{z_i}}{\sum_j m_j e^{z_j}}$$

非法动作概率为 0，合法动作重新归一化。若 logits 对两个合法动作分别为 2 和 1，其他动作即使 logit 为 100 也会被掩掉，合法动作概率仍约为 0.731 与 0.269。

11.9 掩码必须进入整条链

- rollout 采样必须使用掩码；
- buffer 要保留与动作对应的掩码或能重建；
- 更新时计算 log probability 必须用相同语义掩码；
- `predict` 评估必须传掩码；
- 自对弈对手从翻转观察行动时也要用对手视角合法动作；
- SubprocVecEnv 的子环境必须原生暴露 `action_masks`。

只在训练 `sample` 时掩码、更新时不掩码，会使新旧概率不可比。

11.10 动手观察

```
python agents/learning-guide-v2/tools/smoke_labs.py
```

结果中的 `mask_cross_product` 与 `domain_legal_actions` 展示逐维掩码的过度近似比；
`action_space_compatibility` 展示 DQN 在两种空间下的构造差异。

11.11 选择建议

场景	建议
小地图、先验证可训练性	Flat Discrete + MaskablePPO
默认 MultiDiscrete 基线	可研究，但必须监控无效率
大地图、动作很多	自回归或合法动作评分网络
DQN 实验	只能使用 Discrete 路径
跨地图同一模型	Flat + padding，或重新设计固定结构动作

11.12 本章练习

1. 为什么每个维度都合法仍可能组成非法动作？
2. Flat Discrete 精确掩码解决了什么，又引入什么语义问题？
3. 为什么训练和评估必须使用一致掩码？
4. 写出自回归动作概率的条件分解。

第 12 章 奖励塑形、终止与截断

12.1 奖励不是评价报告

奖励是算法每步收到的训练信号。最终研究结论仍应依据规则胜负、固定对手和统计协议。一个模型可以获得高塑形回报却从不赢棋。

环境奖励可分为：

类别	示例	作用
终局	win、loss、draw	明确任务目标
事件	创建、伤害、击杀、占领、治疗、技能	缩短信号距离
势能	收入差、单位差、建筑控制	对状态进展计分
约束	无效动作、回合限制	抑制退化行为

12.2 稀疏奖励

只在终局给：

$$r_t = \begin{cases} +1, & \text{胜} \\ -1, & \text{负} \\ 0, & \text{其他} \end{cases}$$

优点是目标纯净；缺点是随机策略很难产生胜利，数百步中大多数奖励为 0，信用分配困难。

12.3 直接稠密奖励的风险

若伤害 +1、击杀 +5、占领 +10，算法会最大化这些数字，而不是理解设计者意图。只要战斗可反复发生，累计伤害可能超过一次终局胜利。

奖励黑客不是模型“作弊”，而是模型正确优化了错误目标。处理方式是重新审视可重复性、量级和终局主导关系。

12.4 势能塑形

选择状态势函数 $\phi(s)$ ，塑形项为：

$$F(s_t, s_{t+1}) = \gamma\phi(s_{t+1}) - \phi(s_t)$$

新奖励：

$$r'_t = r_t + F(s_t, s_{t+1})$$

在满足假设时，这种形式不会改变最优策略，只改变学习过程。直观上，状态变好时提前获得进展信号；将各步折扣求和后中间势能会相消。

三步展开：

$$\begin{aligned} F_0 + \gamma F_1 + \gamma^2 F_2 &= (\gamma\phi_1 - \phi_0) + \gamma(\gamma\phi_2 - \phi_1) + \gamma^2(\gamma\phi_3 - \phi_2) \\ &= -\phi_0 + \gamma^3\phi_3 \end{aligned}$$

中间 ϕ_1, ϕ_2 抵消，这就是“势能差”而非“每步重复发钱”的关键。

12.5 势函数组成

项目可用加权状态差构造：

$$\Phi(s) = w_I \Delta I + w_U \Delta U + w_C \Delta C$$

- ΔI ：己方与对手收入差；
- ΔU ：单位数量或价值差；
- ΔC ：建筑控制差；
- w_I, w_U, w_C ： `reward_config` 权重。

若 $\phi(s_t) = 10, \phi(s_{t+1}) = 13, \gamma = 0.99$ ：

$$F = 0.99 \times 13 - 10 = 2.87$$

若状态保持不变， $F = 0.99 \times 10 - 10 = -0.1$ 。这反映折扣下维持势能也有轻微时间成本。

训练器和环境的 γ 必须一致。否则塑形抵消关系相对于算法目标不成立。

12.6 终局的势能关闭

终局后不存在下一状态价值。若简单把 $\phi(s_T)$ 继续保留，策略可能偏好以高势能结束。项目在规则终止处对势能进行关闭处理，使塑形总和不变终局排序；在纯截断处则按普通转移保留下一状态信息。

这再次说明 `terminated` 与 `truncated` 不能合并。

12.7 杀敌刷分

历史训练出现过 kill-farm draw plateau:

1. 智能体学会交战;
2. 伤害和击杀带来正奖励;
3. 对手补充单位, 战斗可持续;
4. 占领与结束游戏的边际收益不足;
5. 双方战斗到最大回合, 塑形回报仍较高。

修复不是单独“多训一会儿”, 而是重新设计奖励几何:

- 降低可重复伤害/击杀权重;
- 对受到的伤害计负项, 使交换近似零和;
- 提高占领和终局相对量级;
- 对和局给明确代价;
- 用 `max_turns` 形成规则和局;
- 按终局原因和奖励分解验证。

12.8 永不结束回合

早期 `turn_penalty` 只在 `end_turn` 时扣分。模型发现不结束回合即可回避该负奖励, 于是反复执行合法但无进展动作, 直到 `max_steps` 截断。

当前默认 `turn_penalty=0`。推进压力可通过终局速度奖励、折扣和每回合动作上限实现。
`max_actions_per_turn` 达到上限后把掩码收缩到 `end_turn`, 从环境约束层阻止无限循环。

这个案例说明: 惩罚一个动作与惩罚一种行为并不等价。

12.9 和棋与截断

规则和棋应使用 `terminated=True` 并应用 `draw`。外部步数截断使用 `truncated=True`, 默认不自动收取 `draw`; 若要惩罚截断, 应显式配置 `truncation` 并理解 SB3 的时间限制 `bootstrap`。

把截断同时当作负和局又 `bootstrap` 下一状态, 会混合两个不同语义。实验报告必须分别统计 `max_turns_draw` 与 `max_steps_truncate`。

12.10 奖励分解

`info["reward_breakdown"]` 应满足：

$$r_{step} = \sum_k r_{component,k}$$

建议至少记录：

- 终局；
- 势能；
- 创建、伤害、受伤、击杀；
- 占领进度与完成；
- 对手夺取建筑；
- 无效动作；
- 截断或和局。

若平均回报上涨，首先查看增长来自哪一项。只有终局胜率和任务相关行为同步改善，才能把它解释为策略进步。

12.11 奖励量级检查

设计前估算一局的最大累计量：

单次奖励 × 一局可能发生次数

一次击杀 +5 看似远小于胜利 +1000；若一局可发生 300 次伤害事件且每次 +5，累计可超过终局。应使用上界、历史分位数和实际分解，而不只比较单步权重。

12.12 设计流程

1. 先定义规则胜负指标；
2. 用稀疏终局建立基准；
3. 逐个加入有因果理由的塑形项；
4. 检查是否可重复刷取；
5. 保持训练 γ 与势能 γ 一致；
6. 记录分解和终局原因；
7. 做单变量消融和多种子评估；
8. 用回放确认行为。

12.13 本章练习

1. 计算 $\phi_t = 20, \phi_{t+1} = 22, \gamma = 0.99$ 的势能塑形。
2. 为什么只在 `end_turn` 扣分会鼓励 “永不结束回合” ？
3. 规则和局与外部截断在价值 bootstrap 上有何区别？
4. 若击杀奖励很小，为什么仍要估算一局可累计次数？

第 13 章 DQN：从动作价值到深度 Q 网络

13.1 DQN 解决什么问题

DQN (Deep Q-Network) 用神经网络近似最优动作价值：

$$Q_{\theta}(s, a) \approx Q^*(s, a)$$

在离散动作空间中，若能估计每个动作的长期价值，就可以选择：

$$a_t = \arg \max_a Q_{\theta}(s_t, a)$$

DQN 适合动作集合固定且可枚举的离散任务。它不是本项目主线算法，但 `main.py` 暴露了 `--algorithm dqn`，因此必须说明真实兼容边界。

13.2 从 Q-learning 开始

表格 Q-learning 更新：

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

方括号是 TD 误差。定义目标：

$$y_t = r_{t+1} + \gamma(1 - d_t) \max_a Q_{\theta^-}(s_{t+1}, a')$$

- $d_t = 1$ 表示规则终止；
- θ^- 是目标网络参数；
- $Q_{\theta}(s_t, a_t)$ 是在线网络当前预测。

DQN 最小化：

$$L(\theta) = \mathbb{E}[(y_t - Q_{\theta}(s_t, a_t))^2]$$

数字例：奖励 2， $\gamma = 0.9$ ，下一状态最大目标 Q 为 5，当前 Q 为 4：

$$y = 2 + 0.9 \times 5 = 6.5, \quad y - Q = 2.5$$

网络应提高已执行动作的 Q 预测。

13.3 经验回放

连续游戏转移高度相关。DQN 把转移存入 replay buffer:

$$(s_t, a_t, r_{t+1}, s_{t+1}, d_t)$$

训练时随机抽取小批。作用:

- 打破相邻样本相关性;
- 同一昂贵转移可重复使用;
- 使数据分布比单条当前轨迹更平滑。

代价是样本来自旧策略。环境和动作语义必须稳定; 若 Flat Discrete 索引在每个状态按列表位置重建, 索引 17 不具有固定语义, 网络学习难度显著增加。

13.4 目标网络

若用同一网络同时预测当前 Q 和目标最大 Q, 更新目标会随参数同时移动, 可能形成正反馈。DQN 使用延迟目标网络 θ^- , 每隔若干步从在线网络复制或缓慢更新:

$$\theta^- \leftarrow \theta$$

这不是让目标绝对正确, 而是让优化期间目标变化更慢。

13.5 ϵ -greedy

$$a_t = \begin{cases} \text{随机动作}, & \text{概率 } \epsilon \\ \arg \max_a Q_\theta(s_t, a), & \text{概率 } 1 - \epsilon \end{cases}$$

训练早期 ϵ 较高以探索, 之后衰减。若随机动作不使用合法掩码, 探索会浪费在非法动作。Flat Discrete 精确掩码可限制随机索引; 普通 SB3 DQN 本身不接受 sb3-contrib MaskablePPO 的掩码接口, 因此需要环境保证动作集合或定制策略。

13.6 与策略梯度的区别

DQN	PPO/A2C
输出每个离散动作 Q 值	输出动作概率和价值
Off-policy, 可重放旧数据	On-policy, 主要用新 rollout
ϵ -greedy 探索	随机策略与熵
只支持 Discrete (SB3 实现)	支持 Discrete/MultiDiscrete 等
动作很多时输出头变大	结构化分布可拆多头

13.7 本项目的兼容性事实

当前环境默认：

```
MultiDiscrete([10, 8, W, H, W, H])
```

SB3 2.9.0 的 DQN 只支持 `spaces.Discrete`。实测构造会抛出断言错误。因此：

```
python main.py --mode train --algorithm dqn
```

在当前默认环境下不是可用训练路径。基础 CLI 创建环境时没有切换 `action_space_type="flat_discrete"`。

Flat Discrete 环境可以构造 DQN：

```

from stable_baselines3 import DQN

from reinforcetactics.rl.gym_env import StrategyGameEnv

env = StrategyGameEnv(
    map_file="maps/1v1/starter.csv",
    opponent="noop",
    action_space_type="flat_discrete",
    max_flat_actions=128,
)

model = DQN(
    "MultiInputPolicy",
    env,
    buffer_size=10_000,
    learning_starts=1_000,
    batch_size=64,
    device="cpu",
)

```

“可构造”不等于“适合”。动态位置索引、动作上限和对手非平稳仍是实质限制。

13.8 为什么项目不以 DQN 为主线

1. 默认动作天然由多个条件字段组成；
2. 小地图 Flat 输出尚可，大地图动作上限迅速增长；
3. Flat 索引的语义随合法列表变化；
4. 自对弈使数据分布和对手同时变化；
5. 项目已建立 MaskablePPO 课程、评估和模型加载工具链。

DQN 仍有教学价值：它清楚展示 value-based、off-policy、经验回放和目标网络。

13.9 常见失败

现象	原因
构造即报动作空间不支持	使用默认 MultiDiscrete
Q 值持续增大	目标过估计、奖励尺度或终止处理错误
只选列表前部动作	Flat 排序偏差、探索不足
replay 中旧动作无法解释	动作索引语义不稳定
训练 loss 降但胜率不升	拟合 TD 目标不代表任务策略改善

13.10 CPU 兼容性验证

```
python agents/learning-guide-v2/tools/smoke_labs.py
```

查看 JSON 中 `action_space_compatibility` : DQN 在 `multi_discrete` 下记录断言失败, 在 `flat_discrete` 下记录 `construct_ok`。

13.11 本章练习

1. 计算 $r = 1, \gamma = 0.99, \max Q^- = 8, Q = 6$ 时的 TD 目标和误差。
2. 目标网络为什么能提高稳定性?
3. 为什么 Flat Discrete 可构造 DQN 仍不表示它是本项目最佳算法?
4. 当前 `main.py --algorithm dqn` 的具体不兼容点是什么?

第 14 章 A2C: 同步优势 Actor-Critic

14.1 从策略梯度到 Actor-Critic

策略梯度直接优化期望回报：

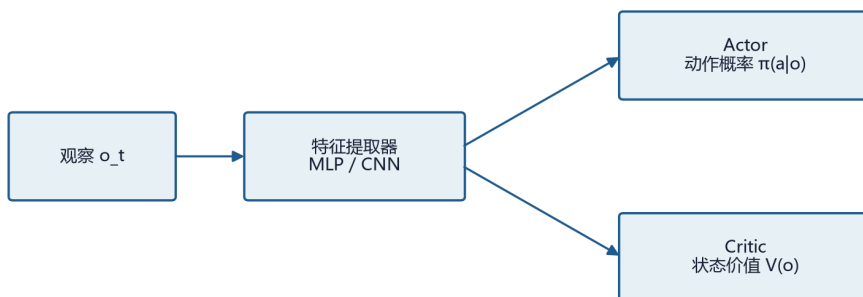
$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[G(\tau)]$$

策略梯度定理给出更新方向的核心形式：

$$\nabla_{\theta} J(\theta) = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A_t]$$

若 $A_t > 0$ ，提高该动作概率；若 $A_t < 0$ ，降低概率。Actor 表示策略，Critic 估计 $V(s)$ 以构造优势。

Actor-Critic: 共享观察、两类输出



Actor-Critic

14.2 A2C 的含义

A2C 通常解释为 Advantage Actor-Critic 的同步版本。它从一个或多个环境收集短 rollout，计算 n 步回报和优势，然后同步更新策略与价值网络。

相比异步 A3C，A2C 在一次批更新前等待并组合多个环境的数据，工程实现更适合现代批计算。

14.3 n 步目标

$$R_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k+1} + \gamma^n V(s_{t+n})$$

优势估计：

$$\widehat{A}_t = R_t^{(n)} - V(s_t)$$

若两步奖励为 1、2， $\gamma = 0.9$ ，两步后的价值估计为 5：

$$R_t^{(2)} = 1 + 0.9 \times 2 + 0.9^2 \times 5 = 6.85$$

若当前价值为 4，优势为 2.85。

14.4 联合损失

策略损失：

$$L_{policy} = -\mathbb{E}[\log \pi_{\theta}(a_t|s_t)\widehat{A}_t]$$

价值损失：

$$L_{value} = \mathbb{E}[(R_t^{(n)} - V_{\phi}(s_t))^2]$$

熵奖励鼓励探索：

$$\mathcal{H}(\pi) = -\sum_a \pi(a|s) \log \pi(a|s)$$

常见总损失：

$$L = L_{policy} + c_v L_{value} - c_e \mathcal{H}(\pi)$$

负号表示最小化损失时会提高熵。 c_v 和 c_e 分别控制价值项和熵项。

14.5 与 PPO 的差别

A2C 没有 PPO 的新旧概率比裁剪。一次更新过大时，策略可能发生显著变化。A2C 每批通常只做较直接的更新，结构简单、速度快，但在复杂任务上对学习率和奖励尺度更敏感。

A2C	PPO
直接策略梯度更新	裁剪概率比
rollout 通常较短	常收集更长 rollout
每批利用较少	同一批多 epoch
实现简单	更新更保守、超参更多

14.6 项目实施路径

基础 CLI 构造：

```
A2C(
    "MultiInputPolicy",
    env,
    learning_rate=3e-4,
    n_steps=5,
)
```

SB3 A2C 支持 MultiDiscrete，因此在默认动作空间下可构造。问题在于基础 CLI 使用普通环境和普通 A2C，没有把合法动作掩码加入策略分布。

在 103,680 个组合空间中，普通 A2C 会大量采样无效动作。环境的 -10 无效惩罚可能主导回报，重演旧 PPO 的接口学习问题。当前 `sb3-contrib` 没有与 MaskablePPO 对等的 MaskableA2C 主线，因此需要自定义分布或改用精确 Flat 空间。

14.7 为什么保留 A2C

- 展示最直接的 Actor-Critic；
- 训练循环比 PPO 更容易理解；
- 可作为 PPO 裁剪价值的对照；
- 在小型、合法动作简单的环境上是合理基线。

但本项目选择 MaskablePPO 作为主线，是因为动作合法性和策略更新稳定性比算法结构简洁更重要。

14.8 实验设计

若比较 A2C 与 PPO，应固定：

- 相同观察和特征提取器；
- 相同动作空间；
- 相同对手、地图和奖励；
- 相同总环境步；
- 多训练种子；
- 相同评估协议。

不能直接拿 A2C 默认 `n_steps=5` 与 PPO 课程配置比较，然后把差异全部归因于算法。

14.9 常见问题

现象	诊断
回报极低且无效率高	没有掩码
熵快速趋近 0	学习率大、熵系数小或惩罚捷径
value loss 爆炸	终局尺度过大、归一化不足
训练很快但完全不赢	短 rollout 与长信用链，或奖励投机
评估结果抖动	策略更新大、样本少、评估局数不足

14.10 本章练习

1. 用给定数字计算两步回报和优势。
2. Actor 与 Critic 分别输出什么？
3. 熵项为何在总损失中带负号？
4. A2C 在默认动作空间可构造，为什么仍不推荐作为主线？

第 15 章 PPO：受约束的策略更新

15.1 PPO 要解决的稳定性问题

策略梯度根据采样动作更新概率。如果一次更新把高优势动作概率从 0.01 推到 0.8，策略分布会发生巨大变化；同一批数据对新策略已不再具有代表性，价值估计也可能失效。

PPO (Proximal Policy Optimization) 通过限制新策略相对旧策略的变化，让每批数据的更新更保守。

15.2 新旧策略概率比

$$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$$

- $r = 1$ ：动作概率未变；
- $r = 1.2$ ：新概率是旧概率的 1.2 倍；
- $r = 0.7$ ：新概率降到旧概率的 70%。

实现中用 log probability:

$$r_t = \exp \left(\log \pi_{\theta}(a_t|s_t) - \log \pi_{old}(a_t|s_t) \right)$$

若旧概率 0.25、新概率 0.30，比例为 1.2。

15.3 未裁剪代理目标

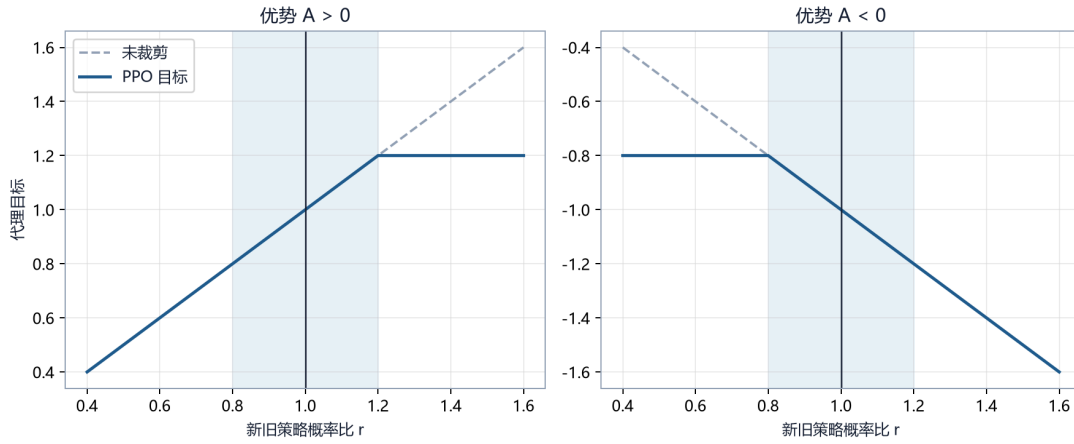
$$L^{PG}(\theta) = \mathbb{E}[r_t(\theta)\widehat{A}_t]$$

正优势希望 r 增大，负优势希望 r 减小。若反复在同一批数据上优化，比例可能离 1 很远。

15.4 裁剪目标

$$L^{CLIP}(\theta) = \mathbb{E} \left[\min \left(r_t \widehat{A}_t, \text{clip} \left(r_t, 1 - \epsilon, 1 + \epsilon \right) \widehat{A}_t \right) \right]$$

常见 $\epsilon = 0.2$ ，裁剪区间为 $[0.8, 1.2]$ 。



PPO 裁剪目标

正优势数字例

$\hat{A} = 2, r = 1.5, \epsilon = 0.2$:

$$r\hat{A} = 3, \quad \text{clip}(1.5, 0.8, 1.2)\hat{A} = 2.4$$

取较小值 2.4。超过 1.2 的继续增益被截断。

负优势数字例

$\hat{A} = -2, r = 0.5$:

$$r\hat{A} = -1, \quad 0.8 \times (-2) = -1.6$$

取较小值 -1.6。概率降低超过边界不会继续带来有利目标。

裁剪不是把所有梯度限制在固定范围，也不是保证 KL 永不超标；它只对代理目标形成悲观下界。

15.5 GAE

TD 误差：

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

GAE：

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots$$

λ 控制偏差与方差。项目默认常用 0.95。对长微动作 episode, $\gamma\lambda$ 决定优势向后传播速度：

$$0.99 \times 0.95 = 0.9405$$

相隔 50 步的 TD 误差权重约 $0.9405^{50} \approx 0.047$ 。

15.6 PPO 总损失

SB3 以最小化形式组合：

$$L(\theta) = -L^{CLIP} + c_v L_V - c_e \mathcal{H}(\pi)$$

价值损失：

$$L_V = \mathbb{E}[(V_\theta(s_t) - \widehat{R}_t)^2]$$

熵项鼓励探索。三项量级不同，`vf_coef` 和 `ent_coef` 用于权衡。若环境终局为 ± 5000 ，价值损失可能远大于策略项；仅调 `vf_coef` 不能替代奖励缩放。

15.7 一次 PPO 更新

```
用旧策略收集 n_steps × n_envs 条转移
→ 计算 value、return、GAE advantage
→ 固定 old_log_prob
→ 打乱为多个 minibatch
→ 重复 n_epochs:
    计算新 log_prob 与 ratio
    计算裁剪策略损失、价值损失、熵
    反向传播、梯度裁剪、优化
→ 丢弃 rollout, 重新采样
```

同一批数据重复 `n_epochs` 提高样本利用率，但 epoch 太多会使新策略偏离收集策略，表现为 `approx_kl` 和 `clip_fraction` 增大。

15.8 超参数解释

参数	主要作用	过大风险	过小风险
<code>learning_rate</code>	每次参数更新尺度	KL 大、崩溃	学习缓慢
<code>n_steps</code>	每环境 rollout 长度	内存和延迟	长期信息不足
<code>batch_size</code>	小批大小	更新次数少	梯度噪声
<code>n_epochs</code>	同批重复利用	过拟合旧批	利用不足
<code>gamma</code>	长期折扣	方差高	终局太远
<code>gae_lambda</code>	优势平滑	方差高	偏差高
<code>clip_range</code>	更新限制	约束弱	学不动
<code>ent_coef</code>	探索	难收敛	过早坍缩
<code>vf_coef</code>	价值损失权重	Critic 主导	价值学不准
<code>max_grad_norm</code>	梯度裁剪	约束弱	持续削弱更新

参数相互作用，不能用“单参数最佳值”概括。

15.9 项目为何选择 PPO

- 支持 Dict 观察和 MultiDiscrete/Discrete;
- 能结合自定义 CNN 特征提取器;
- sb3-contrib 提供 MaskablePPO;
- on-policy 数据与变化对手的语义较清晰;
- 课程、回调、评估和模型 Bot 已围绕 SB3 建立。

代价是样本利用率低，长局训练昂贵；掩码和奖励错误仍会使它稳定地学到错误策略。

15.10 历史困难

项目长期 PPO 训练的主要障碍不是 clip 公式本身，而是系统层：

- 逐维动作掩码产生大量无效组合；

- $\gamma = 0.99$ 对数百微动作终局过短;
- 展平特征缺乏空间归纳偏置;
- 奖励项鼓励刷伤害、拖延或落后状态;
- 课程晋级受评估噪声与 checkpoint 交接影响;
- 单随机种子无法区分改进与偶然。

PPO 章节必须与第十一、十二、十七和二十三章一起使用。

15.11 监控指标

- `approx_kl` 持续过高: 更新过大;
- `clip_fraction` 接近 1: 大量样本被裁剪;
- `entropy` 迅速归零: 策略坍缩;
- `explained variance` 长期接近 0 或为负: 价值预测无解释力;
- `value loss` 极大: 回报尺度或 Critic 失配;
- 平均回报涨而胜率不涨: 检查奖励分解。

15.12 本章练习

1. 旧概率 0.4、新概率 0.5, 计算比例。
2. $\epsilon = 0.2, A = 3, r = 1.4$ 时裁剪目标取多少?
3. 为什么 PPO 仍属于 on-policy?
4. 列出两个 clip 公式之外导致项目 PPO 失败的系统原因。

第 16 章 MaskablePPO 与第一次可验证训练

16.1 MaskablePPO 改变什么

MaskablePPO 保留 PPO 的 rollout、GAE 和裁剪更新，只把策略分布限制在合法动作上。它解决的是“动作合法性”，不是奖励、观察或信用分配。

对合法集合 $\mathcal{A}(s)$ ：

$$\pi'(a|s) = \begin{cases} \frac{\pi(a|s)}{\sum_{b \in \mathcal{A}(s)} \pi(b|s)}, & a \in \mathcal{A}(s) \\ 0, & a \notin \mathcal{A}(s) \end{cases}$$

合法动作重新归一化，策略梯度只在可执行选择之间比较。

16.2 环境构造

推荐从小地图、noop 对手、Flat Discrete 开始验证：

```
from reinforcetactics.rl.masking import make_maskable_env

env = make_maskable_env(
    map_file="maps/1v1/starter.csv",
    opponent="noop",
    action_space_type="flat_discrete",
    max_flat_actions=128,
    max_steps=32,
)
```

Flat 模式使掩码精确。noop 对手只用于接口与冷启动检查，不代表合理长期教师。

16.3 模型构造

```
from sb3_contrib import MaskablePPO

model = MaskablePPO(
    "MultiInputPolicy",
    env,
    learning_rate=3e-4,
    n_steps=32,
    batch_size=32,
    n_epochs=1,
    gamma=0.99,
    gae_lambda=0.95,
    clip_range=0.2,
    seed=42,
    device="cpu",
    verbose=1,
)
```

这里的数值专为 64 步冒烟。生产配置通常使用更大 rollout、更多 epoch、并行环境和空间特征提取器。

16.4 训练与预测

```
model.learn(total_timesteps=64)

obs, info = env.reset(seed=43)

mask = env.action_masks()

action, _ = model.predict(

    obs,

    action_masks=mask,

    deterministic=True,

)

obs, reward, terminated, truncated, info = env.step(action)
```

若忘记 `action_masks` , `predict` 不知道当前合法集合。训练 callback 也应使用 MaskablePPO 专用评估逻辑。

16.5 可直接运行的 CPU 冒烟

```
python agents/learning-guide-v2/tools/smoke_labs.py
```

脚本会：

1. 构造 MultiDiscrete 与 Flat 环境；
2. 验证 PPO、A2C、DQN、MaskablePPO 的空间兼容性；
3. 比较逐维掩码组合数与领域合法动作数；
4. 运行 64 步 Flat MaskablePPO；
5. 运行两次模拟的 MCTS 接口检查；
6. 写出 `assets/data/smoke-results.json` 。

成功标准是脚本退出码 0、数值有限、预测动作可执行。不是胜率标准。

16.6 第一次正式训练应怎样定义

第一次训练的合理验收分三层：

管线层

- 环境、模型和 callback 能创建；
- rollout 完成；
- 无 NaN、动作越界或掩码全空；
- checkpoint 与日志产生。

学习信号层

- episode 回报存在方差；
- 动作分布不是只剩 `end_turn` ；
- value loss 有限；
- 终局原因不全为相同截断。

能力层

- 固定对手和地图上胜率高于随机波动；
- 多种子方向一致；
- 回放显示目标行为；
- 对未用于训练的固定锚仍有效。

64 步只覆盖第一层。

16.7 为什么不是直接运行基础 CLI

`main.py --algorithm ppo` 构造普通 `stable_baselines3.PPO`，不使用 `MaskablePPO` 工厂。因此教学主线显式使用 `sb3_contrib.MaskablePPO` 或 `Bootstrap` 脚本。

基础 CLI 仍可用于说明 SB3 接口，但不应作为本项目复杂动作空间的推荐长训命令。

16.8 MultiDiscrete 与 Flat 选择

项目	MultiDiscrete	Flat Discrete
输出	六个分类头	一个索引头
掩码	逐维过度近似	完整动作精确
跨地图	维度随 W/H 变化	固定上限可统一
大地图	头较紧凑	上限可能很大
动作语义	字段稳定	索引位置动态

初学实操先用 Flat 验证合法性；研究大地图时应评估自回归或合法动作评分网络。

16.9 向量环境注意

SubprocVecEnv 在子进程中计算掩码，环境类必须原生实现 `action_masks`。不能把只存在主进程的闭包当作唯一掩码来源。

评估环境必须复用训练时的：

- 动作空间类型和上限；
- 地图 padding；
- 特征提取器；
- enabled units；
- gamma 与奖励配置；
- 对手参数与最大回合。

16.10 失败诊断

现象	优先检查
<code>ValueError</code> 无合法动作	<code>end_turn</code> 掩码与领域合法动作
训练合法、评估非法	评估漏传掩码
<code>Flat</code> 索引越界	<code>_current_actions</code> 重建时序与上限
只结束回合	奖励、对手、探索、动作列表排序
小图可训、大图加载失败	observation/action shape 与 padding
回报涨、胜率 0	奖励分解和终局原因

16.11 结果记录模板

```
提交：
环境版本：
地图 / 对手：
动作空间 / 上限：
观察提取器：
奖励配置哈希：
PPO 超参数：
训练种子：
总步数与运行时间：
评估地图 / 对手 / 局数 / 座次：
胜负和、置信区间、终局原因：
回放结论：
```

16.12 本章练习

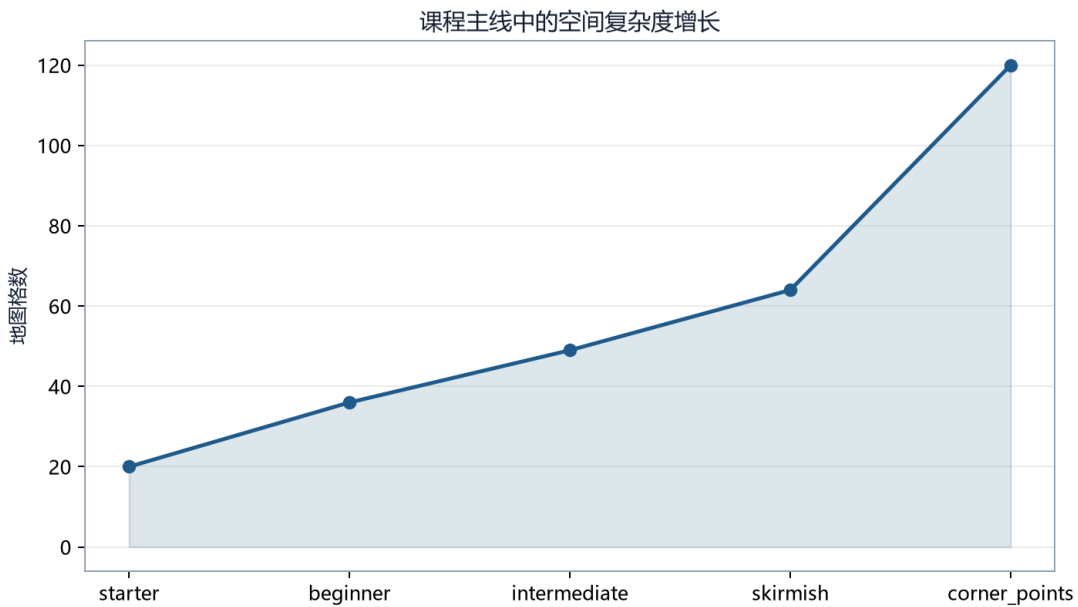
- MaskablePPO 相比 PPO 改变了哪个概率分布？
- 为什么 64 步冒烟不能说明模型会赢？
- 为什么训练和评估环境的 `max_flat_actions` 必须一致？
- 设计三个从“管线可运行”到“能力有效”的递进验收条件。

第 17 章 课程学习与 Bootstrap

17.1 冷启动问题

初始策略接近随机。在完整地图、全部单位和强对手下，它很难偶然完成“生产—行军—战斗—占领总部”的长链。若所有轨迹都失败且回报相似，优势估计缺乏可区分信号。

课程学习把任务按难度排列，使当前策略面对“略高于已有能力”的数据分布。Bootstrap 指通过弱对手、小地图、受限兵种、行为克隆或塑形让策略离开随机冷启动区。



课程地图规模增长

17.2 课程阶段

项目阶段可抽象为：

$$\mathcal{C}_i = (\text{map}, \text{opponent}, \tau_i, p_i, T_i, \text{reward}, \text{entropy}, \text{rules})$$

量	含义
τ_i	晋级胜率阈值
p_i	连续达标次数 patience
T_i	最大训练步预算
map	地图及观察/动作复杂度
opponent	对手类型与参数

阶段不是只换对手。地图尺寸、回合上限、奖励、熵系数和规则覆盖都可能变化。

17.3 训练状态机

```
加载配置与模型
→ 构建阶段 i 的训练/评估环境
→ 训练 eval_freq 步
→ 固定协议评估
→ 连续 p 次胜率 ≥ τ 且满足最低步数?
  └ 是: 恢复阶段最佳 checkpoint, 进入 i+1
  └ 否: 预算是否耗尽?
      └ 否: 继续训练
      └ 是: 抛出 CurriculumStalled
```

停滞应显式失败并保留历史，不能静默进入下一关。否则最终模型看似“完成课程”，实际可能跳过未掌握技能。

17.4 晋级噪声

评估胜率：

$$\widehat{w} = \frac{W}{n}$$

若真实胜率 0.7、评估 20 局，标准误约：

$$\sqrt{\frac{0.7 \times 0.3}{20}} \approx 0.102$$

一次 85% 可能只是波动。 `patience=2` 要求连续两次达标，降低偶然晋级；增加评估局数降低噪声。二者仍不能消除 winner's curse：恰好被选为“最佳”的 checkpoint 可能包含正向噪声。

17.5 MixedBot 难度桥

从 RandomBot 直接换 SimpleBot 可能改变的不只是强度，还包括行为分布。MixedBot 每局以概率 p_h 选择 hard，否则选择 easy：

$$O \sim \begin{cases} O_{hard}, & p_h \\ O_{easy}, & 1 - p_h \end{cases}$$

每局内部保持同一个 Bot，使对手策略连贯。课程可用 25%、50% 等难度桥逐步改变分布。

17.6 阶段交接：峰值而非末值

on-policy 策略在达到高胜率后仍会继续更新，可能向塑形捷径漂移。若下一阶段从“预算耗尽时的最后权重”开始，会丢掉阶段内最佳能力。

当前设计可在晋级时恢复 `best_model`。但“最佳”必须由与阶段一致的评估环境确定，且 checkpoint 保存时点要真实包含触发指标的参数。

历史审查发现 `best_model.zip` 有时仍是阶段带入模型，晋级后恢复会撤销本阶段学习。解决这类问题要核对：

- 首次评估前是否保存 carry-in 基准；
- 达到更好指标时是否覆盖；
- 回调调用顺序；
- 恢复后总步数与优化器状态；
- stage 目录中的时间戳和指标。

17.7 早期失败：确定性 noop 教师

完全不行动的对手适合检查造兵、移动和占领接口，但若作为长期唯一老师：

- 轨迹高度重复；
- 策略可用单一剧本获胜；
- 回报方差趋近零；
- 迁移到会争夺地图的对手时崩溃。

因此 noop 是单元测试和最初台阶，不是课程的能力终点。

17.8 地图迁移与价值错位

从 starter 到 beginner，空间尺寸、路径长度、收入节奏和终局距离改变。即使观察 padding 统一，价值网络对旧地图回报分布的估计也可能不适合新地图。

缓解方式：

- 中间地图；
- 地图切换时提高探索；
- 使用空间特征与坐标通道；
- 阶段首部做诊断评估；
- 允许 Critic 重新适应；
- 避免同时改变太多奖励和规则。

17.9 课程漂移与错误修复链

长期 Bootstrap 复盘可归纳为以下因果链。

现象

一些运行通过早期阶段，却在 `beginner_random_15/20` 或更大地图处形成大量和局；另一些在阶段内先达到高胜率，之后退化。

证据

- 胜率曲线先升后降；
- 平均回合靠近最大回合；
- 塑形回报继续增长；
- 伤害/击杀多而占领减少；
- 不同单种子运行分歧很大。

排除过程

实验依次检查熵、耐心、兵种限制、对手强度、checkpoint 交接、引擎常量、行为克隆和奖励项。多次结论后来被配置或评估环境不一致推翻，说明实验隔离比调参速度更重要。

已落地改进

- 恢复阶段最佳 checkpoint；
- 将规则常量显式放入 `engine_overrides`；

- 增加 MixedBot 与中间地图；
- 改进随机 Bot 与 RNG；
- 记录终局原因和行为组成；
- 调整战斗、占领、受伤与和局奖励；
- 支持最低训练步、patience 和停滞异常；
- 使用空间特征提取器。

当前开放问题

- 长微动作下 γ 的系统性消融；
- 多训练种子；
- Flat 动作索引的语义不稳定；
- 更适合大地图的结构化动作网络；
- 课程阈值的统计决策方法。

17.10 `max_timesteps` 是安全阀

阶段预算应表示“最多允许消耗多少数据后认定停滞”，不是保证这么多步一定足够。若指标长期不动，简单扩大预算可能只强化错误吸引子。

诊断顺序：

1. 是否有合法且多样的动作；
2. 回报是否有方差；
3. 终局原因；
4. 奖励分解；
5. 行为直方图；
6. 评估环境一致性；
7. 最后才考虑增加预算。

17.11 最小管线实验

本书提供一阶段配置：

```
python scripts/train/train_bootstrap.py `

--config agents/learning-guide-v2/labs/bootstrap-smoke.yaml `

--device cpu `

--output-dir tmp/guide-bootstrap-smoke `

--skip-plots `

--skip-videos `

--sanity-episodes 0 `

--no-gcs
```

该阶段阈值为 0、最低步数 32、预算 64，只验证模型创建、评估、晋级和输出目录。它不验证课程是否能学习游戏。

17.12 本章练习

1. 为什么 patience 能降低偶然晋级却不能消除 winner' s curse?
2. MixedBot 为何按 episode 选对手而不是每一步切换?
3. 阶段末模型与阶段最佳模型为何可能不同?
4. 看到 CurriculumStalled 后应先检查哪些证据，而不是立即加预算?

第 18 章 行为克隆

18.1 用专家动作初始化策略

行为克隆 (Behavior Cloning, BC) 把专家轨迹当作监督学习数据:

$$\mathcal{D} = \{(o_i, a_i)\}_{i=1}^N$$

训练策略提高专家动作概率:

$$L_{BC}(\theta) = -\frac{1}{N} \sum_{i=1}^N \log \pi_{\theta}(a_i|o_i)$$

BC 不直接使用奖励。它学习 “在专家访问过的观察上模仿动作”，适合让随机策略先掌握基本操作，再交给 PPO 优化长期目标。

18.2 数据如何采集

项目由脚本 Bot 产生对局，并在示范方行动时记录:

- Dict 观察;
- 六字段或 Flat 动作;
- 动作掩码;
- episode 结果与场景;
- 地图和示范者座次。

只记录示范者动作，不能把对手动作错误地配到示范者观察。地图尺寸和动作空间必须与待初始化模型一致。

18.3 交叉熵的数字例

若某观察下专家动作概率为 0.1:

$$-\log(0.1) \approx 2.303$$

训练后提高到 0.8:

$$-\log(0.8) \approx 0.223$$

损失显著下降。它只说明模型更像数据中的专家，不保证对分布外状态更会行动。

18.4 多字段动作的损失

MultiDiscrete 动作可把各字段负对数概率相加：

$$L_i = - \sum_{d=1}^6 \log p_{\theta}(a_i^{(d)} | o_i)$$

若各字段独立，仍存在组合语义问题。Flat 动作使用单一分类，但动态索引必须在数据记录和训练重建间保持一致。自回归策略则按条件概率分解损失。

18.5 类别不平衡

一回合只结束一次，却可能有很多移动和攻击；`end_turn` 在数据中的频率低，但执行错误会使回合无法推进。项目允许提高 `end_turn_weight`：

$$L = - \frac{1}{N} \sum_i w(a_i) \log \pi(a_i | o_i)$$

权重改变的是该样本的梯度，不是直接给 `end_turn` 增加推理 logit。权重太大可能让模型过早结束回合。

18.6 分布偏移

BC 只见过专家状态。部署时一次错误会进入专家很少访问的局面；模型更易犯第二次错误，误差逐步累积。这叫 covariate shift。

缓解方式：

- 收集多地图、多对手、多座次示范；
- 为脚本 Bot 引入受控随机性；
- 用 PPO 继续交互学习；
- 迭代加入模型失败状态的专家纠正；
- 评估不仅看 BC loss，还看真实对局。

18.7 历史失败 A：确定性 Bot 产生重复轨迹

表面采集 100 局，若 Bot、地图和决胜规则完全确定，可能只有一条独特轨迹。模型会迅速把训练 loss 降低，却没有状态覆盖。

诊断：

- 轨迹哈希；

- 独特观察/动作比例;
- 每场长度和动作分布;
- 不同座次的终局;
- 场景级统计。

修复包括随机决胜、独立 RNG、MixedBot 内部随机传播和多场景配置。

18.8 历史失败 B：测量环境漏参数

BC sanity evaluation 曾漏传生产环境的 `stochastic_tiebreak` 或其他 kwargs，导致评估环境与数据采集环境不同。结果看似“BC 使模型退化”，实际是测量工厂不一致。

可复用教训：训练、数据采集和评估都应从同一环境配置对象构造，而不是三处手写参数。

18.9 历史失败 C：好看的 loss 不等于策略强

确定性重复数据使训练准确率很高；模型可能只学会：

- 高频移动;
- 固定购买顺序;
- `end_turn` 时机;
- 单一地图路线。

价值头在纯 BC 中没有从回报学习，即使策略头已初始化，PPO 开始时 Critic 仍可能很差。BC warm-start 保存的 MaskablePPO 参数要与后续特征提取器、动作空间和地图形状完全一致。

18.10 BC → PPO

流程：

```
脚本 Bot 示范
→ 检查场景与独特轨迹
→ 建立同架构 MaskablePPO
→ 监督训练策略分布
→ 保存 warm-start
→ 在交互环境中 PPO
→ 固定 Bot 梯评估
```

BC 解决“最初怎样产生像样动作”，PPO 解决“怎样根据长期回报改进和纠正专家局限”。

18.11 项目入口

核心实现在 `reinforcetactics/rl/imitation.py` :

- `collect_demonstrations / collect_demonstrations_multi` ;
- `DemonstrationDataset` ;
- `behavior_clone` ;
- `make_warm_started_model` ;
- `evaluate_bc_against_bot_ladder` 。

最小正确性测试:

```
python -m pytest tests/test_imitation.py -q --no-cov
```

测试验证记录形状、只记录示范方、loss 可下降以及 BC 后 PPO 能运行。它不验证长期能力。

18.12 本章练习

1. 专家动作概率从 0.2 提高到 0.6, 负对数损失怎样变化?
2. 为什么 100 条重复轨迹不能等价于 100 条多样轨迹?
3. BC 为什么不能直接学好价值函数?
4. 为什么 BC 模型与 PPO 环境必须使用相同特征提取器和动作空间?

第 19 章 自对弈

19.1 为什么需要自对弈

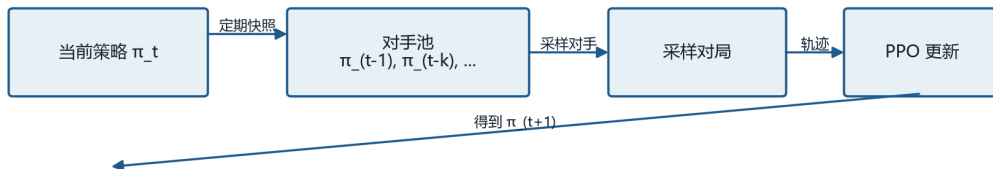
固定脚本 Bot 提供稳定课程，却有能力上限和可利用模式。自对弈让策略面对自己的历史版本，训练分布随能力增长。

最简单形式：

$$\pi_t \text{ 对战 } \pi_t$$

但只对战当前镜像容易产生同步退化和策略循环。项目使用快照与对手池稳定训练。

带历史快照池的自对弈



历史快照池自对弈

19.2 非平稳性

普通单智能体 MDP 假定环境转移固定。自对弈中对手随训练更新，策略看到的转移分布变为：

$$P_t(s'|s, a; \pi_t^{opp})$$

若每几步就把对手替换为当前网络，学习者追逐移动目标。更新太快会不稳定，太慢则容易过拟合旧对手。

19.3 快照池

对手池：

$$\mathcal{P}_t = \{\pi_{t_1}, \pi_{t_2}, \dots, \pi_{t_K}\}$$

每个 episode 从池中采样对手。选择策略：

- uniform：所有快照等概率；
- recent：偏向近期对手；
- prioritized：按胜率或难度加权。

均匀采样简单且提供历史覆盖；偏近期更聚焦当前前沿；优先采样需要可靠难度估计，否则会形成反馈偏差。

19.4 为什么历史对手重要

策略可能存在非传递关系：

A 克制 B, B 克制 C, C 克制 A

只对战最新对手可能忘记较早策略，出现战略循环。历史池相当于持续复习旧考试。

19.5 对手快照与权重安全

对手不应引用会在优化器更新时变化的同一参数对象。应复制或保存稳定快照。若为了推理临时交换权重，必须在异常时通过 `finally` 恢复，否则训练模型可能被对手权重污染。

项目的自对弈工厂在 episode reset 时创建绑定当前 `GameState` 的 `ModelBot`；快照更新频率和池大小由配置控制。

19.6 玩家视角与座次

随机交换座次可以减少先手偏差，但需要完整翻转：

- grid 和 unit 归属；
- own/opp 金币；
- 当前玩家；
- 可见性；
- 奖励符号；
- winner 判断；

- 合法动作查询玩家。

任何漏项都会让“对手视角观察”内部矛盾。战争迷雾下尤其要防止用原玩家可见性。

19.7 固定评估锚

如果训练和评估都面对不断变强的自对弈对手，胜率可能一直约 50%，无法判断绝对能力。必须保留固定锚：

- RandomBot;
- Simple/Medium/AdvancedBot;
- 固定历史 checkpoint;
- 固定地图与座次协议。

自对弈训练指标回答“相对当前对手如何”，固定锚回答“绝对能力是否进步”。

19.8 混合训练

混合策略以概率 ρ 对脚本 Bot，其余对自对弈：

$$O \sim \begin{cases} O_{bot}, & \rho \\ O_{self}, & 1 - \rho \end{cases}$$

脚本 Bot 提供稳定技能锚，自对弈提供不断变化的挑战。比例不是越偏自对弈越先进；冷启动策略的镜像对手同样不会提供高质量战术。

19.9 与课程和 BC 的组合

合理顺序：

- BC（可选）掌握基本操作
 - Bot 课程建立造兵、占领、战斗能力
 - 自对弈扩展策略
 - 固定 Bot 梯与锦标赛持续评估

直接让两个随机策略自对弈，双方可能共同稳定在拖延或无效行为。

19.10 项目实施

`reinforcetactics/r1/self_play.py` 提供:

- `OpponentPool` ;
- `SelfPlayEnv` ;
- `make_self_play_env` ;
- `make_self_play_vec_env` 。

训练脚本:

```
python scripts/train/train_self_play.py `

--mode self-play `

--use-opponent-pool `

--pool-size 4 `

--n-envs 1 `

--no-subprocess `

--total-timesteps 64 `

--n-steps 32 `

--batch-size 32 `

--n-epochs 1 `

--device cpu `

--max-steps 32 `

--eval-freq 32 `

--n-eval-episodes 1 `

--checkpoint-freq 64 `

--log-dir tmp/guide-self-play-smoke
```

这是接口冒烟; 64 步不足以形成有意义对手池能力。

19.11 RNG 与推理热路径

2026-07-24 审查指出自对弈 RNG 和对手推理路径存在可复现与性能风险。应记录池采样 RNG、环境种子和对手推理设备；对手网络应使用 `eval()`、`no_grad()`，避免不必要梯度和随机层。

19.12 常见失败

现象	原因
自对弈胜率恒 50%	指标相对且对手同步变化
对固定 Bot 退化	灾难遗忘、池太偏近期
多环境结果不复现	子进程种子或池 RNG
训练越来越慢	对手推理在热路径、设备切换
一方观察异常	视角翻转不完整
两边一起拖延	冷启动和奖励共同形成坏均衡

19.13 本章练习

1. 为什么只对战当前镜像不足以避免战略遗忘？
2. 自对弈胜率约 50% 时，怎样判断绝对能力？
3. 说明座次翻转至少要同步改变的四类信息。
4. 为什么自对弈通常应放在 Bot 课程或 BC 之后？

第 20 章 Feudal RL：用层级决策分解长期战略

20.1 为什么需要层级

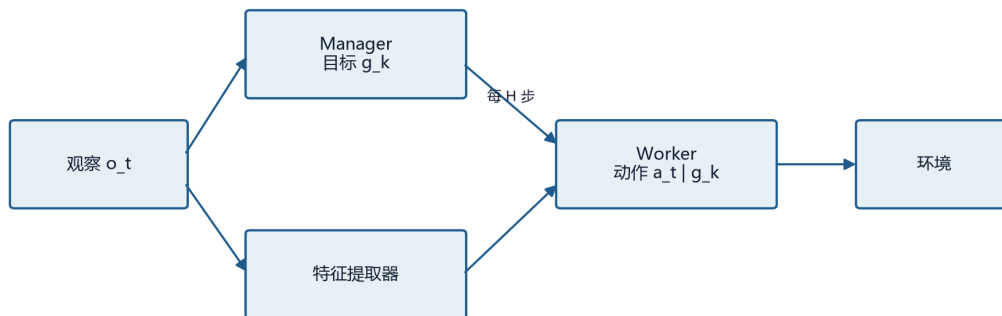
前面的 PPO、MaskablePPO 都在同一个时间尺度上作决定：策略每接收一次观察，就直接输出一环境动作。这个做法适合局部决策，却很难自然表达“先夺取中央资源点，再集结远程单位，最后进攻基地”这样的多步意图。

策略游戏同时包含两种时间尺度：

- **战略层**决定一段时间内应当追求什么；
- **执行层**决定下一步具体移动、生产、攻击还是结束回合。

Feudal Reinforcement Learning（封建式强化学习，简称 Feudal RL）把一个策略拆成 Manager 和 Worker。Manager 以较低频率提出目标，Worker 在接下来的若干微动作中，依据当前观察和目标选择具体动作。

Feudal RL 的经理—工人结构



Reinforce Tactics 的层级结构

这不是把两个完全独立的智能体放进游戏。二者共享同一环境轨迹，只是承担不同职责，并分别拥有策略网络、价值网络和训练数据。

20.2 从时间抽象理解 Manager 与 Worker

设环境微动作编号为 t ，Manager 每隔 c 个微动作输出一次目标：

$$g_k \sim \pi_M(g \mid s_{kc})$$

其中：

- k 是 Manager 的决策段编号；
- c 是 `manager_horizon`；
- s_{kc} 是该段开始时的状态；
- g_k 是这一段要追求的目标；
- π_M 是 Manager 策略。

在段内，Worker 每一步都接收同一个目标：

$$a_t \sim \pi_W(a \mid s_t, g_k), \quad kc \leq t < (k+1)c$$

与普通策略相比，Worker 多了一个条件变量 g_k 。相同状态下，目标不同，动作分布也应不同。例如，目标位于左上资源点时，向左移动的概率应提高；目标是防守基地时，召回单位或结束冒进的概率应提高。

这种设计的关键不是网络数量，而是**时间抽象**：Manager 的一个动作对应 Worker 的一段动作序列。

20.3 项目中的目标表示

本项目的 Manager 输出三维连续目标，概念上可写为

$$g = (x_g, y_g, z_g)$$

前两个分量描述地图目标位置，第三个分量描述目标类别或高层语义。网络输出会被限制到规定范围，再转换为 Worker 能消费的目标向量。这样做有两个好处：

1. 目标空间远小于完整微动作组合，Manager 不必直接选择单位、动作类型和目标坐标；
2. 目标位置可以与卷积提取出的空间特征对齐。

连续目标并不意味着环境存在“移动到坐标”这一原子动作。Worker 仍然需要选择真实合法动作，一步步改变状态。

目标条件不是命令执行器

Manager 的目标只改变 Worker 的概率分布，并不保证 Worker 一定到达目标。训练早期尤其可能出现：

- Manager 给出的目标没有战略意义；
- Worker 尚未学会响应目标；
- Worker 忽略目标，只依赖环境奖励；

- 目标在当前局面不可到达；
- 目标频繁变化，尚未执行就被替换。

因此，层级方法比单层 PPO 多出一个“协同学习”问题：Manager 要提出 Worker 能完成且有价值的目标，Worker 又要表现出足够稳定的目标响应，Manager 才能学到目标的后果。

20.4 内在奖励：怎样告诉 Worker 目标是否有进展

只把终局胜负奖励交给 Worker，目标变量很容易被忽略。项目使用内在奖励衡量单位与目标的空间关系。抽象地写，可以把一个简单版本表示为

$$r_t^{\text{int}} = d(s_t, g) - d(s_{t+1}, g) + b \cdot 1[\text{reach}(s_{t+1}, g)]$$

逐项解释：

- $d(s, g)$ ：状态中相关单位到目标的距离；
- $d(s_t, g) - d(s_{t+1}, g)$ ：距离缩短时为正，远离时为负；
- $1[\cdot]$ ：指示函数，条件成立取 1，否则取 0；
- b ：到达目标的额外奖励；
- 单位：与环境奖励一样，都是“每个微动作的奖励值”，但数值尺度可能不同。

假设单位到目标的曼哈顿距离从 5 降到 3，未到达目标，且距离改善系数为 1，那么内在奖励为 $5 - 3 = 2$ 。若下一步从距离 1 到达目标，且到达奖励为 5，则奖励为 $1 + 5 = 6$ 。

这里的关键是，内在奖励只回答“是否按照 Manager 的目标行动”，并不直接回答“这个目标能否赢得游戏”。后者由外在环境奖励和 Manager 学习承担。

20.5 Worker 奖励：必须以实际代码为准

项目当前缓冲区中的合成公式是：

```
self.w_rewards.append(
    intrinsic_reward
    + worker_reward_alpha * extrinsic_reward
)
```

因此，Worker 实际接收

$$r_t^W = r_t^{\text{int}} + \alpha r_t^{\text{ext}}$$

其中：

- r_t^{int} 是目标进展奖励；
- r_t^{ext} 是经过 `reward_scale` 缩放后的环境奖励；
- α 对应 `worker_reward_alpha`，默认配置为 0.5；
- r_t^W 是进入 Worker GAE 和 PPO 更新的奖励。

这不是

$$(1 - \alpha)r_t^{\text{int}} + \alpha r_t^{\text{ext}}$$

形式的凸组合。换句话说，`worker_reward_alpha=1.0` 在当前实现中表示“完整内在奖励加上一倍外在奖励”，并不表示“只保留外在奖励”。如果命令帮助或旧实验注释把它描述为内外奖励的插值比例，应当把那种描述视为不精确说明，以这段实际数据路径为准。

数值例子

假设：

- 原始环境奖励为 100；
- `reward_scale=0.001`；
- 内在奖励为 2；
- `worker_reward_alpha=0.5`。

先缩放外在奖励：

$$r^{\text{ext}} = 100 \times 0.001 = 0.1$$

再合成 Worker 奖励：

$$r^W = 2 + 0.5 \times 0.1 = 2.05$$

这个例子揭示了尺度关系：若内在奖励经常在 1 到 5，而缩放后外在奖励只有 0.1，Worker 的短期行为主要受目标塑形控制。Manager 则需要通过累计外在奖励判断这个目标是否真正有利于胜利。

20.6 Manager 奖励与段级回报

Manager 的一个动作持续 k_t 个环境微动作。项目在目标段结束时记录该段累计的外在奖励和实际段长：


```
def end_manager_segment(self, cumulative_reward, done, segment_length):

    self.m_rewards.append(cumulative_reward)

    self.m_dones.append(done)

    self.m_segment_lengths.append(segment_length)
```

段长不能被忽略。若仍对每个 Manager 段固定乘一次 γ ，就相当于把持续 2 步和持续 10 步的目标视为具有相同时间跨度。当前 `_compute_gae` 对 Manager 使用

$$\gamma_t = \gamma^{k_t}$$

并把 TD 残差写为

$$\delta_t = R_t^M + \gamma^{k_t}(1 - d_t)V(s_{t+1}) - V(s_t)$$

其中：

- R_t^M 是整个目标段的累计外在奖励；
- k_t 是该段真实持续的微动作数；
- d_t 表示段末是否终局；
- V 是 Manager 价值估计。

若 $\gamma = 0.99$ ，段长为 10，则跨段折扣为

$$0.99^{10} \approx 0.904$$

而不是 0.99。这样才能保持与逐微动作时间尺度一致。

Manager 的 GAE 递推也要使用段折扣：

$$A_t = \delta_t + \gamma^{k_t}\lambda(1 - d_t)A_{t+1}$$

这是一处很容易被“照抄普通 PPO”遗漏的实现细节。

20.7 两套 Worker 动作头

项目保留两类 Worker。

20.7.1 传统多头 Worker

传统 Worker 为 MultiDiscrete 的各维分别输出概率，例如动作类型、来源坐标、单位类型和目标坐标。训练时可保存每一维的布尔掩码，并在 PPO 重算对数概率时应用同一组掩码。

优点是结构简单、并行计算方便。局限与第 11 章讨论的一样：逐维掩码只能说明单个取值是否在某些合法动作中出现，不能保证各维组合后仍然合法。

20.7.2 自回归 Worker

自回归 Worker 按条件顺序生成结构化动作：

$$p(a \mid s, g) = p(a_{\text{type}} \mid s, g) \times p(a_{\text{src}} \mid a_{\text{type}}, s, g) \times p(a_{\text{unit}} \mid a_{\text{type}}, a_{\text{src}}, s, g) \times p(a_{\text{target}} \mid \text{previous}, s, g)$$

取对数以后才变为各阶段对数概率之和：

$$\log p(a \mid s, g) = \sum_j \log p(a_j \mid a_{<j}, s, g)$$

每个阶段使用依赖前面选择的条件掩码。项目缓冲区会保存 `atype`、`src`、`unit_type` 和 `target` 四组采样时掩码，在 PPO 更新时重放，避免“采样分布与训练分布不一致”。

这比传统多头更接近精确动作掩码，但代价是：

- 推理要顺序执行多个头；
- 缓冲区必须保存更多条件信息；
- 某个早期头选择不佳，会限制后续所有选择；
- 实现与测试明显更复杂。

20.8 Manager 和 Worker 都使用 PPO

项目不是用一种新损失完全替代 PPO，而是在两个时间尺度上分别应用 PPO。对任一层 $X \in \{M, W\}$ ，策略损失为

$$L_X^{\text{clip}} = -\mathbb{E} \left[\min \left(r_t^X A_t^X, \text{clip} \left(r_t^X, 1 - \epsilon, 1 + \epsilon \right) A_t^X \right) \right]$$

再加价值损失、减去熵奖励：

$$L_X = L_X^{\text{clip}} + c_v L_X^V - c_e H_X$$

项目为 Manager 和 Worker 分别统计 policy loss、value loss 和 entropy。诊断时不能只看总回报：

- Worker 内在奖励持续升高，但胜率不升：Worker 会执行目标，Manager 目标可能无用；
- Manager 回报有改善，但 Worker 到达目标率低：高层意图没有可靠落地；
- 两层价值损失都很大：奖励尺度或终局脉冲可能过强；
- 熵迅速接近零：某一层过早形成确定性策略。

20.9 为什么项目尝试 Feudal RL

本项目的长回合、空间地图和组合动作天然具有层级结构。Feudal RL 的潜在优势是：

1. **延长信用分配范围。** Manager 以段为单位学习，不必把每个微动作分别关联到终局；
2. **形成可解释中间变量。** 目标坐标可绘制、统计到达率；
3. **复用执行技能。** 一个会“向目标移动与作战”的 Worker 可以服务多个高层目标；
4. **匹配策略语言。** 占点、集结、进攻等计划本来就跨越多个动作。

代价同样明显：

1. 两层同时变化，训练分布非平稳；
2. 内在奖励设计会引入新的偏好；
3. 目标空间是否足够表达战略并无保证；
4. 样本和调参成本高于单层 MaskablePPO；
5. 传统 Worker 仍承受组合动作掩码的近似误差。

因此，Feudal RL 在本项目中属于进阶实验路径，不应因为结构更复杂就预设它一定优于主线 MaskablePPO。

20.10 历史困难的因果整理

当时现象

- 训练能够运行，但 Worker 内在奖励与胜率不同步；
- Manager 的价值损失容易受大额终局奖励影响；
- 逐维动作头仍会形成非法组合；
- 多环境采集、断点恢复和自对弈快照需要保存额外层级状态；
- 配置中的环境平衡、奖励和单位限制曾有未完整传递的风险。

调查证据

单看 episode reward 无法判断问题来自哪一层。后来增加了 Worker 内在/外在奖励、到达目标率、Manager/Worker 各自损失等诊断；缓冲区也显式保存动作掩码和 Manager 段长。

已排除的简单解释

“层级网络没有学会”过于笼统。至少要分别检查目标是否可完成、Worker 是否响应目标、Manager 是否获得正确段奖励，以及掩码在更新时是否与采样一致。

已落地修复

- 外在奖励进入层级缓冲区前支持 `reward_scale` ;
- Manager GAE 使用真实段长的 γ^{k_t} ;
- Worker 采样掩码可随轨迹保存并重放;
- 增加自回归 Worker 与阶段条件掩码;
- 训练配置把环境奖励、引擎覆盖、单位集合和观察尺度传给 Feudal 环境;
- checkpoint 保存训练计数、最佳指标和快照节奏。

当前仍开放的问题

- 三维目标是否是最合适的高层表示;
- 内在奖励是否诱导“抵达坐标”而非“取得战略收益”;
- Manager horizon 是否应随局面变化;
- 自回归 Worker 的额外复杂度能否稳定转化为胜率;
- 在相同计算预算下, 层级方法是否显著优于更充分训练的 MaskablePPO。

20.11 最小实操

先运行单元与集成测试:

```
pytest -q tests/test_feudal_rl.py tests/test_feudal_rl_integration.py
```

再用教材配置作 CPU 冒烟训练:

```
python scripts/train/train_feudal_rl.py `
    --mode feudal `
    --config agents/learning-guide-v2/labs/feudal-smoke.yaml
```

冒烟训练的验收目标不是胜率, 而是:

- 能完成一次 rollout 和两层 PPO 更新;
- Manager 与 Worker 的损失均为有限数;
- 轨迹长度、段长和奖励分量可记录;
- checkpoint 可写入临时目录;
- 使用自回归 Worker 时, 四组条件掩码数量与 Worker 步数一致。

正式实验应至少比较:

实验	Worker	reward_scale	目的
F0	传统多头	1.0	原始尺度基线
F1	传统多头	0.001	检查价值损失稳定性
F2	自回归	0.001	检查精确条件掩码收益
F3	自回归	0.001	改变 horizon, 检查时间抽象

所有比较都应固定地图、对手、训练步数、评估种子和评估局数。

20.12 常见误解

误解一：Manager 直接控制游戏。

Manager 只产生目标；环境动作全部由 Worker 输出。

误解二：内在奖励越高，策略越强。

它只说明 Worker 更符合目标。目标本身可能无助于胜利。

误解三：worker_reward_alpha=1 等于只用外在奖励。

当前实现是内在奖励加一倍外在奖励。

误解四：Manager 每段折扣一次普通 γ 即可。

段长不同，应使用 γ^{k_t} 。

误解五：层级网络天然比单层网络强。

它提供了归纳偏置，也引入了协同、奖励设计和非平稳问题，结论必须由等预算实验给出。

20.13 本章小结

Feudal RL 通过 Manager—Worker 把长期目标与微动作执行分开。项目当前实现包含独立的两层 PPO、目标内在奖励、段级 GAE、掩码重放和可选自回归 Worker。理解这一实现最重要的三点是：目标不是原子动作；Worker 奖励是实际代码规定的加法结构；Manager 的时间折扣必须反映真实段长。

20.14 练习

1. 当 $\gamma = 0.98$ 、Manager 段长为 8 时，跨段折扣是多少？为什么不能直接使用 0.98？

2. 若内在奖励为 3、原始外在奖励为 500、`reward_scale=0.001`、`worker_reward_alpha=0.2`，计算 Worker 奖励。
3. 解释“到达目标率上升但胜率下降”可能对应的两种原因。
4. 为什么 PPO 更新时必须重放采样时使用的条件掩码？
5. 设计一个消融实验，区分自回归动作头和奖励缩放各自的作用。

第 21 章 蒙特卡洛树搜索：在行动之前进行推演

21.1 从反应式策略到显式搜索

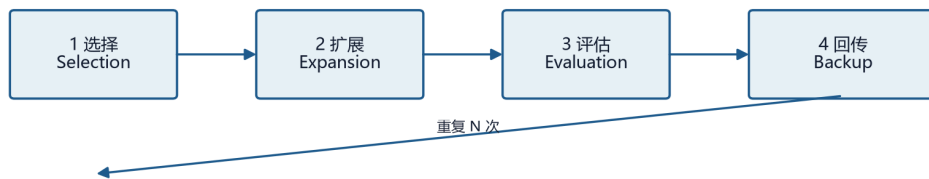
PPO 或 DQN 的网络通常只做一次前向传播，就给出当前状态下的动作。MCTS 则在真正执行动作之前，复制当前局面并反复模拟可能的后续分支，以访问次数形成改进后的决策。

这两种方法回答不同问题：

- 神经网络回答“凭已有经验，这个局面看起来怎样”；
- 树搜索回答“如果沿着若干候选动作继续推演，哪些分支更值得选择”。

Reinforce Tactics 的 AlphaZero 实验把二者结合：网络提供先验概率和叶节点价值，MCTS 在当前局面内重新分配搜索预算。

蒙特卡洛树搜索的一次模拟



MCTS 的四个阶段

21.2 搜索树中的统计量

树中每个节点代表一个游戏状态，边代表一个合法动作。对父节点 s 和动作 a ，需要维护：

- $P(s, a)$ ：神经网络给出的先验概率；
- $N(s, a)$ ：这条边被访问的次数；
- $W(s, a)$ ：经过这条边得到的累计价值；

- $Q(s, a) = W(s, a)/N(s, a)$: 平均值。

第一次访问时 $N = 0$ ，项目把对应 Q 视为 0，避免除零。

数值例子

假设三个合法动作的统计如下：

动作	P	N	W	Q
攻击	0.50	20	8	0.40
占点	0.30	5	3	0.60
结束回合	0.20	1	0	0.00

攻击的平均价值不如占点，但访问更多；结束回合证据最少。选择公式要同时利用现有价值与探索不足。

21.3 Selection: PUCT 选择

项目使用 AlphaZero 风格 PUCT：

$$\text{score}(s, a) = Q(s, a) + c_{\text{puct}} P(s, a) \frac{\sqrt{N(s) + 1}}{1 + N(s, a)}$$

其中：

- $N(s)$ 是父节点访问次数；
- c_{puct} 控制探索强度；
- $P(s, a)$ 使网络认为较可能的动作优先得到搜索机会；
- 分母 $1 + N(s, a)$ 会降低已经充分访问的分支的探索奖励。

假设父节点访问 25 次、 $c_{\text{puct}} = 1.5$ 。对上表“占点”：

$$U = 1.5 \times 0.30 \times \frac{\sqrt{26}}{1 + 5} \approx 0.382$$

所以总分约为 $0.60 + 0.382 = 0.982$ 。

对“攻击”：

$$U = 1.5 \times 0.50 \times \frac{\sqrt{26}}{21} \approx 0.182$$

总分约为 0.582。虽然攻击先验更高，但占点当前的价值和访问不确定性使它更值得继续探索。

玩家视角的符号

两人零和游戏中，“对当前玩家有利”通常意味着“对对手不利”。项目节点保存玩家身份，选择与回传时依据节点玩家和根玩家关系翻转 Q 或累积价值的符号。

这是实现 MCTS 最容易出错的部分之一。若所有层都不翻号，搜索会假设对手配合根玩家；若重复翻号，又会把本来正确的价值变反。阅读代码时必须同时确认：

1. 网络价值从谁的视角输出；
2. 终局价值从谁的视角定义；
3. 回传存储统一根视角还是节点视角；
4. 选择时是否还需翻转。

当前实现的约定是：网络估值最初来自叶节点当前玩家视角，必要时转为根玩家视角；回传依据节点玩家与根玩家关系加减；选择子节点时再使 Q 与当前父节点视角一致。

21.4 Expansion: 只扩展合法动作

到达未扩展叶节点后，搜索取得真实合法动作，把它们映射到 Flat Discrete 索引，然后只为这些索引创建子节点。

项目平坦动作空间大小为：

$$|\mathcal{A}| = 10 \times W \times H$$

其中 10 是动作类别数， W, H 为地图宽高。对 20×20 地图，策略头输出 4000 个 logit，但每个局面只扩展合法索引。

先验概率在网络中已经用动作掩码归一化。非法动作 logit 被设为约 -10^8 ，softmax 后概率接近零。树仍以游戏引擎给出的合法动作表为最终依据，而不是假设所有高概率索引均可执行。

惰性复制

项目创建子节点时不立即复制并推进全部状态。只有某个子节点首次在 Selection 中被选中，才：

1. 深复制父状态；
2. 把保存的动作引用解析到复制后的对象；
3. 执行动作；
4. 更新当前玩家和终局标志。

这样可避免为从未访问的分支支付完整状态复制成本。代价是动作对象不能直接从原状态复用，因为深复制后的单位与建筑是不同对象；必须按坐标或稳定标识重新解析。

21.5 Evaluation: 策略头与价值头

叶节点不再进行随机 rollout，而由网络一次输出：

$$(p, v) = f_{\theta}(s)$$

- p 是 Flat Discrete 动作空间上的概率；
- $v \in [-1, 1]$ 是当前玩家视角的终局结果估计。

若节点已经终局，则不调用网络：

$$v = \begin{cases} +1, & \text{根玩家获胜} \\ 0, & \text{和局} \\ -1, & \text{根玩家失败} \end{cases}$$

网络减少了从叶节点一直随机模拟到终局的成本，但估值质量会直接限制搜索质量。未经训练的随机网络仍能完成流程测试，却不能提供可靠棋力。

21.6 Backup: 把叶节点证据传回根

一条模拟路径得到叶节点价值 v 后，对路径上的节点更新：

$$\begin{aligned} N &\leftarrow N + 1 \\ W &\leftarrow W + \sigma v \end{aligned}$$

其中 σ 根据节点玩家与根玩家是否一致取 $+1$ 或 -1 。然后

$$Q = \frac{W}{N}$$

访问次数增加会使同一分支的平均价值逐渐稳定，也会降低它在 PUCT 中的探索项。

需要注意，MCTS 的一次 “simulation” 并不是一局完整游戏；它通常只从根沿当前树走到一个未展开叶节点，评估一次，再回传。

21.7 根节点狄利克雷噪声

自对弈若始终沿最大先验搜索，很快会反复产生相似轨迹。项目训练搜索在根节点混入狄利克雷噪声：

$$\begin{aligned} P'(s, a) &= (1 - \varepsilon)P(s, a) + \varepsilon \eta_a \\ \boldsymbol{\eta} &\sim \text{Dirichlet}(\alpha, \dots, \alpha) \end{aligned}$$

其中 `dirichlet_epsilon` 决定噪声权重，`dirichlet_alpha` 决定噪声形态。

- 较小 α ：概率质量集中到少数随机动作，探索更尖锐；
- 较大 α ：噪声更平均；

- $\varepsilon = 0$: 完全不注入根噪声。

项目默认 `dirichlet_alpha=0.3`、噪声权重 0.25。评估时应设置 `add_noise=False`，否则同一模型比较会混入不必要的搜索随机性。

21.8 从访问次数得到动作分布

搜索结束后，项目先用访问比例构造完整动作分布：

$$\pi(a | s) = \frac{N(s, a)}{\sum_b N(s, b)}$$

这既是选动作的依据，也是 AlphaZero 训练策略头的监督目标。

温度 τ 进一步控制采样：

$$\pi_\tau(a | s) = \frac{N(s, a)^{1/\tau}}{\sum_b N(s, b)^{1/\tau}}$$

- $\tau = 1$: 保持访问比例；
- $0 < \tau < 1$: 分布更尖锐；
- $\tau \rightarrow 0$: 趋向访问次数最大的动作；
- 项目对 `temperature=0` 直接使用 `argmax`。

假设访问次数为 [50, 30, 20]。 $\tau = 1$ 时概率为 [0.5, 0.3, 0.2]； $\tau = 0.5$ 时平方后归一化：

$$[2500, 900, 400]/3800 \approx [0.658, 0.237, 0.105]$$

训练前期使用较高温度可增加局面多样性，后期和评估使用贪心选择可提高稳定性。

21.9 当前代码的完整调用路径

核心搜索接口是：

```
action_probs, root_value = mcts.search(
    game_state,
    add_noise=True,
)
```

或者直接选动作：

```
flat_action, action_probs = mcts.select_action(
    game_state,
    temperature=1.0,
    add_noise=True,
)
```

`search` 返回二元组，而不是包含字段的字典：

1. `action_probs`：长度为 $10WH$ 的访问分布；
2. `root_value`：根节点累计价值的均值。

搜索内部链路为：

```
GameState
-> build_observation + legal flat mask
-> AlphaZeroNet.predict
-> expand legal children
-> repeat PUCT / lazy state transition / evaluate / backup
-> visit-count policy
```

21.10 计算成本

若每个真实动作运行 S 次 simulation，一局有 T 个微动作，那么至少约有 $S \times T$ 次叶节点网络评估，还伴随大量 Python 状态深复制、合法动作枚举和引擎执行。

例如 $S = 100, T = 300$ ，单局就可能需要约 3 万次模拟扩展。训练又需要许多自对弈局，因此 MCTS/AlphaZero 的 CPU 成本远高于一次前向传播选动作的 PPO。

可采用的工程优化包括：

- 批量评估多个叶节点；
- 缓存状态评估；
- 更紧凑的可复制状态；
- 减少无意义的微动作；
- 并行自对弈；
- 训练初期用较少 simulations，后期增加；
- 对等预算比较，而不是只比较训练迭代数。

项目当前实现更偏向清晰、可验证的实验原型，不能把它的吞吐率直接等同于高度优化的棋类 AlphaZero 系统。

21.11 失败模式与诊断

全零访问分布

可能原因包括：合法动作映射为空、simulation 数为 0、动作执行异常或根节点没有成功扩展。项目选择动作时有结束回合的 fallback，但 fallback 只能防止崩溃，不能说明搜索正确。

所有价值都接近零

随机初始化网络常出现这种情况。检查访问分布是否仍由先验和探索项形成；真正训练后再判断价值头是否有区分力。

搜索偏爱非法动作

检查三层一致性：

1. 网络掩码形状是否为 $10WH$ ；
2. flat 索引编码/解码是否一致；
3. 深复制后动作引用是否重新解析。

加大 simulation 数却没有提升

可能是价值网络误差形成系统性误导，也可能是动作粒度过细：大量搜索预算花在同一游戏回合内的排列选择，尚未看到长期局面变化。

评估结果波动

需要固定随机种子、关闭根噪声、使用 `temperature=0`，并扩大对局数。搜索本身的随机性与对局随机性必须分开控制。

21.12 为什么 MCTS 适合又不完全适合本项目

适合之处：

- 游戏规则可准确模拟；
- 状态可以深复制；

- 动作合法性可枚举；
- 双方对抗、终局胜负明确；
- 空间局面适合策略价值网络。

困难之处：

- 微动作分支多、对局长；
- 动作顺序可能产生大量近似等价分支；
- 状态复制和 Python 引擎执行较重；
- 不完全信息开启后，单一真实状态树会泄露隐藏信息；
- 训练网络之前，搜索缺乏可靠叶节点评估。

当前主线训练默认不启用战争迷雾，AlphaZero 路径也建立在可观测 `GameState` 上。若未来在 POMDP 条件下使用搜索，需要考虑信念状态、信息集合或 determinization，而不能直接让智能体搜索它本不应看到的信息。

21.13 最小实操

先运行 MCTS/AlphaZero 单元测试：

```
pytest -q tests/test_alphazero.py
```

教材冒烟工具使用小网络与极少 simulation：

```
python agents/learning-guide-v2/tools/smoke_labs.py --only mcts
```

预期检查项是：

- 搜索返回长度正确的概率数组；
- 所有概率有限且非负；
- 有访问时概率和约为 1；
- 非零概率只出现在合法 flat 索引；
- `root_value` 位于合理范围；
- 原始 `GameState` 没有被模拟修改。

这仍然是结构验证，不是棋力评估。

21.14 本章小结

MCTS 通过 Selection、Expansion、Evaluation 和 Backup，把网络的即时判断变成经过局面推演的访问分布。PUCT 平衡平均价值、网络先验和访问不确定性；根噪声与温度负责自对弈探索。对本项目而言，最关键的正确性条件是玩家视角、合法动作映射、状态复制和动作引用解析的一致性。

21.15 练习

1. 计算父访问次数为 99、子访问 9、先验 0.2、 $Q = 0.3$ 、 $c_{\text{puct}} = 1.5$ 时的 PUCT 分数。
2. 为什么评估时要关闭狄利克雷噪声并令温度为 0？
3. 若访问次数为 $[9, 4, 1]$ ，分别计算 $\tau = 1$ 与 $\tau = 0.5$ 的动作概率。
4. 说明深复制状态后不能直接复用原单位对象引用的原因。
5. 战争迷雾下直接搜索完整 `GameState` 会产生什么信息泄露？

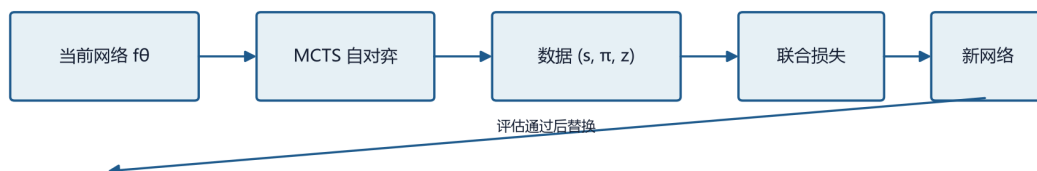
第 22 章 AlphaZero：让搜索生成更强的学习目标

22.1 核心闭环

AlphaZero 不是“神经网络加 MCTS”这一静态组合，而是一个循环：

1. 当前网络指导 MCTS；
2. MCTS 产生比网络原始策略更强的访问分布；
3. 双方按该分布自对弈，得到终局结果；
4. 用访问分布训练策略头，用结果训练价值头；
5. 候选网络与当前最佳网络比较；
6. 达到门槛后替换最佳网络，再开始下一轮。

AlphaZero 的训练外环



AlphaZero 训练闭环

网络为搜索提供方向，搜索又为网络提供改进目标。这种“策略改进—策略评估”循环与动态规划思想相通，只是使用神经网络近似大状态空间中的策略与价值。

22.2 训练样本 (s, π, z)

自对弈每一步保存：

$$(s_t, \pi_t, z_t)$$

其中：

- s_t : 玩家相对视角编码的观察;
- π_t : MCTS 访问次数归一化后的动作分布;
- $z_t \in \{-1, 0, +1\}$: 对局结束后, 从 s_t 当前玩家视角回填的结果。

这里的策略目标不是实际采样出的单个动作, 而是完整分布 π_t 。例如一次搜索访问三个动作的次数是 50、30、20, 那么目标为 $[0.5, 0.3, 0.2]$ 。它保留了搜索对多个候选分支的相对判断, 比 one-hot 动作标签包含更多信息。

终局结果要按样本视角回填。若最后玩家 0 获胜:

- 玩家 0 回合采集的状态标签为 +1;
- 玩家 1 回合采集的状态标签为 -1;
- 和局则双方状态均为 0。

若忘记视角翻转, 价值头会被同一局中的互相矛盾标签训练。

22.3 项目的双头网络

AlphaZeroNet 先把三部分观察组合起来:

1. `grid` 空间通道;
2. `units` 空间通道;
3. 广播到每个网格位置的 `global_features`。

张量从 (B, H, W, C) 转为 PyTorch 卷积使用的 (B, C, H, W) , 经过输入卷积和若干残差块, 再分成两个头。

策略头

策略头输出:

$$\mathbf{l}_\theta(s) \in \mathbb{R}^{10WH}$$

其中 $\mathbf{l}$ 是 logits。预测时将非法动作位置设为约 -10^8 , 再做 softmax:

$$p_\theta(a | s) = \frac{\exp l_a}{\sum_{b \in \mathcal{A}_{\text{legal}}} \exp l_b}, \quad a \in \mathcal{A}_{\text{legal}}$$

非法动作概率近似为 0。

价值头

价值头输出一个标量:

$$v_\theta(s) = \tanh(u_\theta(s)) \in [-1, 1]$$

这个范围与胜、和、负标签 $+1, 0, -1$ 相匹配。

为什么使用残差块

残差块计算

$$y = x + F(x)$$

网络只需学习相对输入的修正 $F(x)$ 。跳连有助于梯度跨越较深网络，并允许多层卷积逐渐扩大感受野。项目默认 6 个残差块、128 个通道；教材冒烟会缩到 1 个块和少量通道，以验证流程而非模型容量。

22.4 联合损失

对一个 batch，项目策略损失是目标访问分布与网络 log-softmax 的交叉熵：

$$L_p = -\frac{1}{B} \sum_{i=1}^B \sum_a \pi_i(a) \log p_\theta(a | s_i)$$

价值损失是均方误差：

$$L_v = \frac{1}{B} \sum_{i=1}^B (v_\theta(s_i) - z_i)^2$$

代码组合为：

$$L = L_p + L_v$$

并由优化器的 `weight_decay` 实现参数正则化。书写成理论形式可记为

$$L(\theta) = (z - v_\theta(s))^2 - \pi^\top \log p_\theta(\cdot | s) + c \|\theta\|^2$$

数值例子

设搜索目标为

$$\pi = [0.7, 0.2, 0.1]$$

网络输出

$$p = [0.5, 0.4, 0.1]$$

则策略损失为

$$L_p = -(0.7 \ln 0.5 + 0.2 \ln 0.4 + 0.1 \ln 0.1) \approx 0.899$$

若终局标签 $z = 1$ ，价值预测 $v = 0.4$ ，则

$$L_v = (1 - 0.4)^2 = 0.36$$

忽略正则项，总损失约为 1.259。

逐符号理解：

- π 不是网络旧策略，而是搜索后的教师分布；
- p_θ 是网络当前预测；
- z 是完整对局结束后的真实结果；
- v_θ 是当前状态下的结果预测。

22.5 搜索为何可看作策略改进

网络先验 p_θ 只经过一次前向传播。MCTS 利用规则模型向后推演，把计算集中到看起来更有价值或证据不足的分支，最后得到 π 。

在理想情况下：

$$\pi \approx \text{Improve}(p_\theta, v_\theta)$$

训练再使

$$p_{\theta'} \approx \pi$$

于是新网络把这次搜索获得的知识“摊销”进一次前向传播。下一轮搜索从更好的先验和价值出发，可以产生更好的目标。

这并不保证每轮都改进：

- 搜索预算太小， π 可能主要反映噪声；
- 价值头错误会误导深层选择；
- 自对弈数据过于相似会导致策略覆盖变窄；
- 训练过度会过拟合最近 buffer；
- 候选评估样本过少会误判晋级。

22.6 Replay Buffer 与数据时效

项目的 `ReplayBuffer` 保存多个自对弈迭代的样本，容量由 `buffer_size` 控制。训练从中采样 batch。

保留历史数据的优点：

- 降低相邻自对弈局之间的相关性；
- 防止网络只记住最近候选策略；
- 提高样本复用率。

代价是数据由旧网络和旧搜索产生。容量太大时，训练目标可能滞后；容量太小时，数据多样性不足。需要联合考虑：

- 每轮自对弈局数；
- 每局样本数；
- buffer 容量；
- 每轮 epoch 数；
- 网络更新幅度。

“更大的 buffer 一定更稳定” 不是普遍结论。

22.7 自对弈中的探索日程

项目用 `temperature_threshold` 控制一局中的探索阶段：

- 前若干动作按访问分布采样；
- 超过阈值后用低温或贪心选择。

原因是开局需要覆盖多种布局 and 战略，后期则希望减少随机失误，使终局标签更能反映策略质量。根节点狄利克雷噪声仅在自对弈时打开。

如果从第一步就完全贪心，buffer 很快被少数开局占据；如果全局一直高温，许多本可获胜的局会因后期随机动作而丢失，价值标签噪声上升。

22.8 候选模型晋级

每轮训练后，项目可让候选网络与上一最佳网络对局。设评估 n 局，候选胜 w 局，则经验胜率：

$$\hat{p} = \frac{w}{n}$$

达到 `eval_threshold` 才接受候选。默认阈值为 0.55。

若只评估 20 局，55% 对应 11 胜。11 比 10 只多一局，统计证据很弱。以二项分布的近似标准误差估计，在 $p \approx 0.5, n = 20$ 时：

$$SE(\hat{p}) \approx \sqrt{\frac{0.5(1-0.5)}{20}} \approx 0.112$$

一次随机波动就足以跨越门槛。因此：

- 冒烟测试可用少量对局检查流程；
- 正式晋级应扩大对局数；

- 双方交换先后手；
- 固定评估种子与搜索预算；
- 关闭根噪声；
- 报告胜/和/负，而不只给单一胜率。

如果候选未达门槛，当前实现恢复最佳网络参数，而不是继续以失败候选作为下一轮起点。这是“候选—最佳”门控机制。

22.9 当前实现状态

项目已经具备：

- 残差策略价值网络；
- 合法动作掩码；
- 神经网络引导的 MCTS；
- 根噪声和温度选动作；
- 自对弈样本生成；
- replay buffer；
- 联合策略/价值训练；
- 候选对最佳网络评估；
- checkpoint 与恢复入口；
- AlphaZeroBot 推理接入；
- 单元与冒烟测试。

同时必须保持准确定位：这是项目中的实验路径，并非已经由大规模、多种子、等计算预算实验确认优于 MaskablePPO 的生产级主线。MCTS 的 Python 状态复制和微动作搜索成本也使完整训练远比命令表面上的“100 次迭代”昂贵。

22.10 历史困难的因果整理

当时现象

- 搜索流程可以运行，但训练耗时很高；
- 小样本候选评估波动明显；
- flat 动作索引、合法掩码与真实引擎动作需要三方一致；
- 状态复制后，动作中的对象引用可能指向原状态；
- 价值的玩家视角容易在自对弈、叶估值和回传之间混淆。

调查证据

对应测试分别验证了网络输出形状与范围、掩码归一化、MCTS 返回分布、动作解析、自对弈样本以及训练损失。性能问题则来自每个真实微动作都要进行多次状态复制与叶评估，而不是单一网络前向本身。

已落地措施

- 使用 Flat Discrete 的统一编码；
- 搜索扩展只依据合法动作；
- 子状态惰性创建；
- 深复制后重新解析动作引用；
- 明确网络价值和终局标签的玩家相对视角；
- 自对弈与评估分别控制噪声、温度；
- 通过最佳模型门控避免每个候选自动成为新基线。

当前开放问题

- 如何降低状态复制和动作枚举成本；
- 微动作层搜索是否应改为游戏回合级宏动作；
- 候选评估应采用多大样本与何种置信门；
- 网络规模、搜索预算与自对弈数量的最优分配；
- 长期是否需要并行自对弈和批量叶评估。

22.11 最小训练

教材提供的 CPU 冒烟命令为：

```
python scripts/train/train_alphazero.py `

--map-file maps/1v1/starter.csv `

--res-blocks 1 `

--channels 16 `

--num-simulations 1 `

--iterations 1 `

--games-per-iter 1 `

--epochs-per-iter 1 `

--batch-size 2 `

--buffer-size 128 `

--max-game-steps 4 `

--temperature-threshold 2 `

--eval-games 0 `

--checkpoint-dir tmp/learning-guide-v2/alphazero `

--device cpu
```

该命令已在基准环境中验证：生成 4 个自对弈样本，完成一次联合损失更新，并写出 iteration 与 final checkpoint。eval-games=0 在当前脚本中表示跳过候选晋级对局。

冒烟成功只表示：

- 自对弈能生成至少一个样本；
- policy target 形状为 $10WH$ ；
- 价值标签在 $[-1, 1]$ ；
- 一次反向传播损失有限；
- checkpoint 可保存和读取。

不能由一次 16 步截断小局推断策略已经学会游戏。

22.12 结果应怎样阅读

训练日志至少同时看：

- `policy_loss`：网络与 MCTS 分布的差异；
- `value_loss`：结果预测误差；
- `buffer` 样本数；
- 每局平均长度；
- 搜索访问分布熵；
- 候选对最佳的胜/和/负；
- 每秒生成的自对弈步数；
- 每个真实动作的搜索时间。

策略损失下降不是充分成功条件。若 MCTS 教师本身质量差，网络只是更准确地模仿一个弱教师。价值损失下降也可能来自大量和局，使网络统一预测 0。必须结合对局和固定锚点评估。

22.13 与 PPO、自对弈 PPO 的区别

方面	MaskablePPO	自对弈 PPO	AlphaZero
动作目标	PPO 优势	PPO 优势	MCTS 访问分布
价值目标	GAE 回报	GAE 回报	终局结果
是否显式搜索	否	否	是
数据来源	环境对手	历史策略池	双方同源自对弈
每步推理成本	一次策略前向	一次策略前向	多次模拟与前向
主动作空间	MultiDiscrete/掩码	同左	Flat Discrete
当前项目定位	主训练路径	进阶训练策略	实验路径

AlphaZero 并不是 PPO 加一个自对弈开关。它改变了训练标签、动作表示、推理过程与计算预算结构。

22.14 常见误解

误解一：MCTS 分布是真实最优策略。

它只是给定网络、搜索预算和探索参数下的改进近似。

误解二：价值头直接回归每步即时奖励。

项目目标是终局结果 z ，不是单步 shaped reward。

误解三：候选胜率 55% 就已证明更强。

结论强度取决于评估局数、置信区间和先后手控制。

误解四：增加 simulations 总会线性提高实力。

弱价值网络、冗余动作树和计算瓶颈都可能产生收益递减。

误解五：一次冒烟训练可与完整 AlphaZero 结果比较。

冒烟验证软件路径，不验证算法极限。

22.15 本章小结

AlphaZero 用网络指导搜索，再用搜索生成的访问分布与终局结果训练网络。项目实现了双头残差网络、合法动作掩码、MCTS、自对弈 buffer、联合损失与候选晋级。正确理解它需要同时把握三个视角： π 是搜索教师， z 是玩家相对终局标签，候选评估是带统计误差的实验而不是确定性裁决。

22.16 练习

1. 已知 $\pi = [0.6, 0.4]$ 、 $p = [0.75, 0.25]$ ，计算策略交叉熵。
2. 为什么每个历史状态的 z 必须按当时行动玩家的视角回填？
3. 分析 buffer 太大和太小各自可能产生的问题。
4. 20 局评估中 11 胜是否足以确信候选更强？请用标准误差解释。
5. 设计一组等计算预算实验，比较 MaskablePPO 与 AlphaZero，而不是简单比较“训练迭代数”。

第 23 章 实验设计、统计误差与可复现性

23.1 训练成功不是单次曲线上升

强化学习结果同时受初始化、环境随机性、探索动作、对手行为和评估样本影响。一次训练中胜率上升，只能说明“这一条随机轨迹在这一套条件下得到这个结果”，不能自动证明算法或配置更好。

一个可解释实验至少要明确：

- 研究问题；
- 对照组与实验组；
- 唯一主动改变的因素；
- 训练预算；
- 随机种子；
- 评估协议；
- 主要指标与次要指标；
- 失败判据；
- 代码、配置和环境版本。

23.2 先写假设，再运行

以“精确掩码是否优于逐维掩码”为例，合格的预注册式描述是：

在相同地图、对手、网络、总环境步数和随机种子集合下，自回归精确掩码将降低无效动作率，并提高固定评估集上的平均胜率。

它包含：

- 自变量：动作头与掩码方法；
- 因变量：无效动作率、胜率；
- 控制变量：地图、对手、网络、步数、种子；
- 方向性预测：降低或提高。

“试试这个参数是否更好”没有说明“好”的指标，也没有排除同时变化的因素。

23.3 随机种子控制了什么

项目训练可能涉及 Python `random`、NumPy、PyTorch、Gymnasium 环境以及 Bot 内部 RNG。只调用一次 `torch.manual_seed` 不等于完整复现。

理想做法是为每次 run 记录一个主种子 s ，再确定性派生：

$$s_{\text{env}}, s_{\text{policy}}, s_{\text{opponent}}, s_{\text{eval}} = h(s, \text{component})$$

这样各组件既可复现，又不会意外共享完全相同的随机流。

训练和评估种子应分离。若训练过程不断在同一组评估种子上选 checkpoint，就会对评估集过拟合；应保留最终测试种子，只在模型选择完成后使用。

23.4 均值、方差与标准误

设 n 个种子的最终胜率为 $x_1, \dots, x_n$ 。样本均值：

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

样本标准差：

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

均值标准误：

$$\text{SE}(\bar{x}) = \frac{s}{\sqrt{n}}$$

假设五个种子的胜率为 0.40、0.55、0.45、0.60、0.50，均值是 0.50，样本标准差约 0.079，标准误约 0.035。

标准差描述单次 run 的离散程度；标准误描述对总体均值估计的不确定性。不能用“均值 $\pm$ 标准差”冒充均值置信区间。

23.5 胜率是二项比例

如果每局只有胜或负，胜局数

$$W \sim \text{Binomial}(n, p)$$

经验胜率为 $\hat{p} = W/n$ 。简单正态近似标准误为

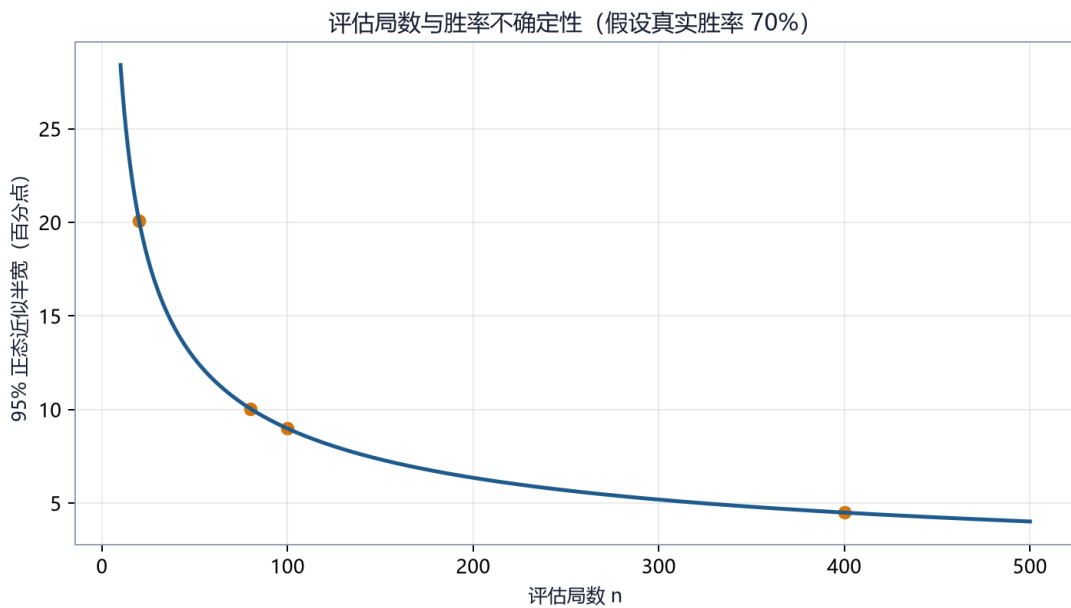
$$\sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

当样本很少或胜率接近 0/1 时，更适合 Wilson 区间，而不是对称正态区间。Wilson 95% 区间可写为

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z \sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}$$

其中 $z = 1.96$ 。

若 20 局赢 12 局，点估计是 60%，但区间仍很宽。项目中的课程晋级和候选模型门控因此需要耐心、最小阶段步数或更多评估局，不能把一次略过阈值视为稳定能力。



评估样本量与置信区间

23.6 和局与截断不能随意并入失败

策略游戏常有三种结果：胜、负、和。还要区分和局原因：

- `max_turns_draw`：游戏回合上限；
- `max_steps_truncate`：环境微动作上限；
- 规则定义的其他和局。

可报告：

$$\text{win rate} = \frac{W}{N}$$

$$\text{non-loss rate} = \frac{W + D}{N}$$

也可给和局半分的 score：

$$\text{score} = \frac{W + 0.5D}{N}$$

但三者含义不同。若奖励塑形产生拖延吸引子，模型可能通过增加和局提高 non-loss rate，却没有提高获胜能力。因此本项目评估工具保留 wins、losses、draws 和 end_reason 分解。

23.7 训练曲线不是独立样本

同一 run 中相邻评估点共享绝大多数训练历史，不能把 100 个时间点当作 100 个独立实验。平滑曲线只用于看趋势，不增加独立证据。

绘图时建议：

- 横轴使用环境步数，而非日志行号；
- 多种子画均值线与置信带；
- 同时保留每个种子的细线；
- 标记阶段切换和配置改变；
- 不用过强平滑掩盖崩溃；
- 指标定义改变时分图，不强行拼接。

23.8 消融实验

消融实验通过移除或替换一个机制，判断改进来自哪里。以 Bootstrap 某版本为例，若同时改了奖励、对手、动作空间和晋级门，即使结果变好，也不能知道贡献来源。

一个 2 × 2 设计：

组别	精确掩码	势能塑形
A	否	否
B	是	否
C	否	是
D	是	是

可以估计：

- 掩码主效应；

- 奖励主效应；
- 两者交互：精确掩码是否只有在某种奖励下才有效。

若计算预算有限，先做成对消融：从当前最佳配置只去掉一个机制。不要把不同训练预算的旧 run 当作严格对照。

23.9 配对评估

地图和先后手造成的方差可以通过配对降低。对模型 A 与 B：

1. 在地图 m 、种子 s 上让 A 先手；
2. 同一地图和种子让 B 先手；
3. 对所有地图、种子重复；
4. 对每一对的 score 差做统计。

配对差：

$$d_i = x_i^A - x_i^B$$

比较 $\bar{d}$ 比较两组互不对应的均值更容易消除局面难度差异。

23.10 多重尝试与选择偏差

如果试了 54 个版本，只报告最好一个，哪怕所有版本真实能力相同，也容易因随机波动找到一个“赢家”。这叫多重比较或研究者自由度问题。

应保留：

- 所有版本清单；
- 每次改动原因；
- 失败结果；
- 选择下一实验的依据；
- 独立最终测试集。

本书附录 D 将 v15–v54 等历史实验按版本索引，而正文只选有明确因果证据的案例深入解释。两者功能不同：正文教推理，索引防止选择性遗忘。

23.11 可复现实验记录

每个 run 建议至少保存：

```

experiment_id: mask_ablation_seed0

code_revision: <git commit>

created_at: <ISO-8601 time>

seed: 0

python: 3.12.13

gymnasium: 1.3.0

stable_baselines3: 2.9.0

sb3_contrib: 2.9.0

torch: 2.13.0+cpu

device: cpu

map: maps/1v1/starter.csv

opponent: balanced_random

total_timesteps: 100000

evaluation:

  seed: 10000

  episodes: 100

  deterministic: true

```

还需保存完整解析后的配置，而不只保存用户传入的 YAML。因为默认值可能随代码变化。

23.12 本项目的诊断指标

`evaluate_model` 除胜率与回报外，还能统计：

- 动作类型计数；
- `action`、`shaping_delta`、`invalid_penalty`、`terminal` 奖励分量；
- `hq_capture`、`elimination`、`max_turns_draw`、`max_steps_truncate`；
- 单位生产构成；
- `captures`、`kills`、`attacks`、`seize attempts`、伤害；

- 建筑治疗的 HP 与金币；
- 可夺取动作出现率；
- 最大合法动作数；
- 峰值军队规模与平均金币；
- 指定失败结局的逐步 JSONL trace。

这些指标用于区分“不会”与“不能”：

- `seize_available_rate` 很低：环境中很少出现可夺取机会；
- 机会很多但 `seize` 次数低：策略不愿选择；
- `max_legal_actions` 接近 flat 上限：动作截断风险；
- 胜率且伤害高：可能只会战斗，不会完成胜利条件；
- 回报高但终局多为截断：可能奖励吸引子。

23.13 一个完整实验模板

研究问题：`reward_scale=0.001` 是否稳定 Feudal 的价值学习？

1. 选择两个配置，只改变 `reward_scale`；
2. 使用种子 0、1、2、3、4；
3. 每个种子训练相同环境步数；
4. 每 10 万步在固定 50 局开发集评估；
5. 最终 checkpoint 在未使用的 200 局测试集评估；
6. 主要指标：测试胜率；
7. 次要指标：Worker/Manager value loss、截断率、到达目标率；
8. 报告五种子均值、标准差和原始点；
9. 若某 run 崩溃，不静默删除，记录失败原因。

23.14 本章小结

强化学习实验的基本单位不是一条漂亮曲线，而是定义明确、可复现、带不确定性的比较。随机种子、评估协议、和局原因、配对设计和失败记录共同决定结论是否可信。项目的细粒度评估指标应服务于因果诊断，而不能由单一 episode reward 代替。

23.15 练习

1. 五个种子结果为 0.3、0.4、0.4、0.5、0.9。计算均值，并说明只报告均值为何危险。
2. 100 局中 60 胜、20 和、20 负，计算 win rate、non-loss rate 和半分 score。

3. 为什么相邻训练评估点不能当作独立样本？
4. 为“课程学习是否优于固定难度训练”设计一个公平对照。
5. 找出一个同时改变三个因素的错误实验，并改写为可解释的消融。

第 24 章 规则 Bot、训练对手与游戏平衡

24.1 对手是训练分布的一部分

在单智能体 1v1 包装中，环境内部替玩家 2 调用 Bot。对学习器而言，对手行为决定了状态转移分布：

$$P(s_{t+1} | s_t, a_t; \pi_{\text{opp}})$$

更换对手策略 π_{opp} ，即使地图和奖励完全不变，智能体看到的状态、合法动作、战斗压力和回合长度都会改变。因此，“对手难度”不是界面设置，而是训练任务定义的一部分。

24.2 NoopBot：最低层的因果探针

NoopBot 不执行任何动作，只结束回合。它的用途不是提供有趣比赛，而是隔离故障：

- 若模型连静止对手都无法击败，问题大概率在探索、动作编码、奖励或胜利条件信用分配；
- 若能击败 Noop，却无法过渡到随机对手，问题更可能是对抗压力和策略鲁棒性；
- 若只会无限积累 shaped reward 而不终结 Noop，对奖励吸引子的判断更明确。

Noop 胜率高不是算法成熟的证据。它只通过了最低功能测试。

24.3 RandomBot 与动作吞吐量

RandomBot 每次从当前合法的非结束回合动作中均匀采样，最多尝试 `max_actions` 次，然后结束回合。

这带来一个常被忽略的难度维度：每回合动作吞吐量。`max_actions=1` 与 20 不只是同一随机策略的速度差异；前者很可能无法让所有单位行动，后者能持续生产、移动和攻击，造成完全不同的状态分布。

均匀随机合法动作也不等于均匀随机“战略”。如果移动动作数量远多于夺取动作，按动作条目均匀抽样会自然偏向移动。这就是动作集合基数诱导的行为偏差。

24.4 BalancedRandomBot：随军队规模增长的随机压力

BalancedRandomBot 每回合：

1. 若能生产，随机生产一个单位；

2. 对每个己方单位，从涉及该单位的合法动作中随机执行一个；
3. 结束回合。

它避免 `RandomBot(max_actions=1)` 在失去唯一单位后几乎不生产、形成自我停滞的问题，也避免固定 20 次动作在小军队阶段产生不自然高吞吐。压力大致随军队规模增长，因此适合放在 Noop 与强规则 Bot 之间。

24.5 SimpleBot: 局部启发式

SimpleBot 分成生产、单位行动、结束回合三个阶段。它具有：

- 单位购买优先级；
- Warrior 比例上限，避免军队永久退化为单一廉价单位；
- 占领当前位置结构；
- 受伤单位留在己方治疗地块；
- Cleric 治疗、Mage 麻痹等单位特性；
- 按距离和目标类别寻找敌军或可占结构；
- 可选随机 tie-break。

这类 Bot 的强项是稳定、便宜、可解释。弱点是启发式分阶段固定，缺少跨单位的全局组合优化。

24.6 Medium、Advanced 与 Master

难度提升并不是简单增加属性。后续 Bot 在 Simple 的基础上引入更丰富的战术评分、威胁判断、协同、撤退与技能组合。

教材使用的能力梯应理解为经验性序列：

```
Noop
-> BalancedRandom / 受限 Random
-> Simple
-> Medium
-> Advanced
-> Master
```

“类名更高级”不构成严格全序。某个 Bot 可能在地图 A 更强、地图 B 更弱；随机 Bot 也可能偶然克制确定性启发式。因此要用多地图、换边循环赛测量，而不是把名称直接当作 Elo。

24.7 MixedBot: 平滑难度边界

MixedBot 在每个 episode 创建时，从 easy 与 hard 两个内部 Bot 中抽取一个：

$$\pi_{\text{opp}} = \begin{cases} \pi_{\text{hard}}, & u < p_{\text{hard}} \\ \pi_{\text{easy}}, & u \geq p_{\text{hard}} \end{cases}$$

一整个 episode 使用同一个内部 Bot，不会在一局中途切换。这一点重要，因为中途切换会使对手行为突然改变，破坏局内策略一致性。

它可以构造桥接阶段：

- Simple/Medium 混合；
- Medium/Advanced 混合；
- 随课程进度逐步提高 `p_hard`。

混合训练的价值是降低突然换对手造成的分布跃迁，同时防止策略完全遗忘较弱对手上的基本终局能力。

24.8 确定性与随机 tie-break

规则 Bot 的排序常出现同分候选。若总按列表顺序取第一个，同一地图和先后手会产生字节级重复轨迹。

`games_per_side` 即使大于 1，也可能只是重复同一局。

项目允许向 Bot 注入 `random.Random`，仅在评分相同处随机打破平局。Tournament 的 `rng_seed` 会结合 game id、地图和双方名称派生每局种子：

- 同一总种子可复现整组比赛；
- 同一对阵的多局又不是完全重复；
- 战术启发式质量保持不变，只改变同分选择。

平衡分析应记录该种子；调试确定性 replay 时则可不注入 RNG。

24.9 训练对手选择的三个风险

过弱

模型学会对手特有漏洞，例如：

- 对手不生产；
- 对手只做一个动作；
- 对手从不防守 HQ；
- 对手固定目标顺序。

在训练对手上胜率很高，换对手立即崩溃。

过强

训练早期几乎没有正回报，优势估计主要是失败噪声。更强对手不必然提供更好的学习信号。

过窄

即使难度适中，单一确定性策略也会使训练分布狭窄。模型可能记住对手节奏，而非学会通用游戏机制。

解决方法是课程、混合对手、自对弈历史池和固定锚点评估共同使用。

24.10 游戏平衡与学习难度

单位数值、经济和地图不只是“游戏设计”，还决定 RL 优化地形。

经济滚雪球

占领建筑提高未来每回合收入，收入又增加单位数量：

capture → income → army → more capture

这产生长期正反馈。若 shaped reward 只奖励即时伤害，策略可能忽略经济链；若过度奖励占领，又可能反复寻找可刷的结构奖励。

单位耐久与和局

高防御、治疗和自动恢复可延长战斗，使 max-turn 和局增多。此时训练曲线可能显示生存奖励提高，却不代表终局能力提高。

地图距离

HQ 距离增大使终局回报延迟。对 $\gamma = 0.99$ ，200 步后的单位回报权重约为：

$$0.99^{200} \approx 0.134$$

同一奖励在长地图上的信用显著变弱，所以地图课程同时改变了探索与折扣难度。

动作分支

单位越多，合法移动与技能动作越多。最大合法动作数是平衡指标，也是学习复杂度指标。

24.11 Bot 行为指标

不能只按胜率给 Bot 排名。建议记录：

- 每回合平均动作数；
- 生产单位构成；
- 首次交战与首次占领回合；
- HQ 捕获、消灭与和局比例；
- 平均银行金币；
- 峰值军队规模；
- 伤害交换比；
- 技能使用率；
- 先手优势；
- 地图分层胜率。

一个 Bot 可能胜率相同，但以截然不同方式施压。对训练而言，这种行为差异本身就是多样性资源。

24.12 平衡实验范式

假设要判断 Warrior 成本调整是否合理：

1. 保持 Bot 代码、地图和随机种子不变；
2. 用 `engine_overrides` 建立成本基线与候选；
3. 运行换边循环赛；
4. 比较总胜率、生产构成、首战时间和回合长度；
5. 检查是否只在某张地图改变；
6. 再用 RL 训练检查学习难度，而不是只看规则 Bot；
7. 把游戏平衡结论与算法结论分开。

若同时改收入、成本和攻击力，不能把胜率变化归因于某个单位成本。

24.13 最小实操

规则 Bot 测试：

```

pytest -q `

tests/test_noop_bot.py `

tests/test_random_bot.py `

tests/test_balanced_random_bot.py `

tests/test_medium_bot.py `

tests/test_advanced_bot.py `

tests/test_master_bot.py `

tests/test_mixed_bot.py

```

构造训练环境时可直接传递对手及参数：

```

env = StrategyGameEnv(

    map_file="maps/1v1/starter.csv",

    opponent="mixed",

    opponent_kwargs={

        "easy": "simple",

        "hard": "medium",

        "p_hard": 0.5,

    },

)

```

验证时至少 reset 多个种子，并从实际创建的内部 Bot 或回合行为确认混合比例，而不能只检查配置字典。

24.14 本章小结

规则 Bot 同时是对手、课程阶段和诊断仪器。Noop 隔离环境与奖励问题，BalancedRandom 提供随军队规模增长的随机压力，规则启发式 Bot 提供稳定难度，MixedBot 平滑分布边界。对手选择与游戏平衡共同决定学习问题，必须用行为指标、多地图和换边实验测量。

24.15 练习

1. 为什么 `RandomBot(max_actions=1)` 与 `max_actions=20` 不能视为同一难度？
2. `MixedBot` 为什么按 episode 选内部 Bot，而不在每个回合重新抽取？
3. 设计一个能检测确定性重复对局的检查。
4. 给出“模型过拟合 `SimpleBot`”的三个可观察信号。
5. 设计 `Warrior` 成本调整的最小消融。

第 25 章 模型评估、Elo、循环赛与回放

25.1 评估回答的不是“训练奖励有多高”

训练 episode reward 受奖励塑形、探索动作和对手采样影响。评估应回答更直接的问题：

- 能否获胜；
- 以何种方式结束；
- 对哪些地图与对手有效；
- 是否依赖先手；
- 是否稳定复现；
- 是否出现拖延、刷分或动作退化。

因此需要把固定 episode 评估、成对模型比较、循环赛和回放结合起来。

25.2 `evaluate_model` 的调用契约

项目统一评估函数可用于普通 PPO 与 MaskablePPO：

```
results = evaluate_model(  
    model,  
    env,  
    n_episodes=50,  
    deterministic=True,  
    seed=10_000,  
    track_breakdown=True,  
    trace_dir="tmp/eval_traces",  
)
```

它通过检查 `model.predict` 的签名决定是否传 `action_masks`。这避免把掩码参数错误地传给普通 PPO，也避免 MaskablePPO 在评估时失去掩码。

评估循环的终止条件是：

```
done = terminated or truncated
```

但结果分类仍保留终局与截断原因。截断不是环境规则胜负，不能只因循环结束就当成失败。

25.3 平均回报与胜率的分工

平均回报可用于诊断奖励路径，胜率用于衡量任务结果。两者可能背离：

- 高伤害、高占领 shaped reward，但没有终结对局；
- 快速取胜使累计塑形少，回报反而低；
- 频繁截断获得生存或资源增量；
- 终局奖励尺度变化导致跨配置回报不可比。

所以同一表中应同时给：

指标	回答的问题
胜/和/负	是否完成对抗目标
<code>end_reason</code>	如何结束
episode reward	奖励函数怎样评价轨迹
episode length/turns	是否拖延
reward breakdown	哪一分量主导
action counts	策略实际做了什么

25.4 Elo 的基本公式

两个模型 A、B 的 Elo 差转为 A 的预期得分：

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}}$$

若 $R_A = 1600, R_B = 1400$:

$$E_A = \frac{1}{1 + 10^{-0.5}} \approx 0.760$$

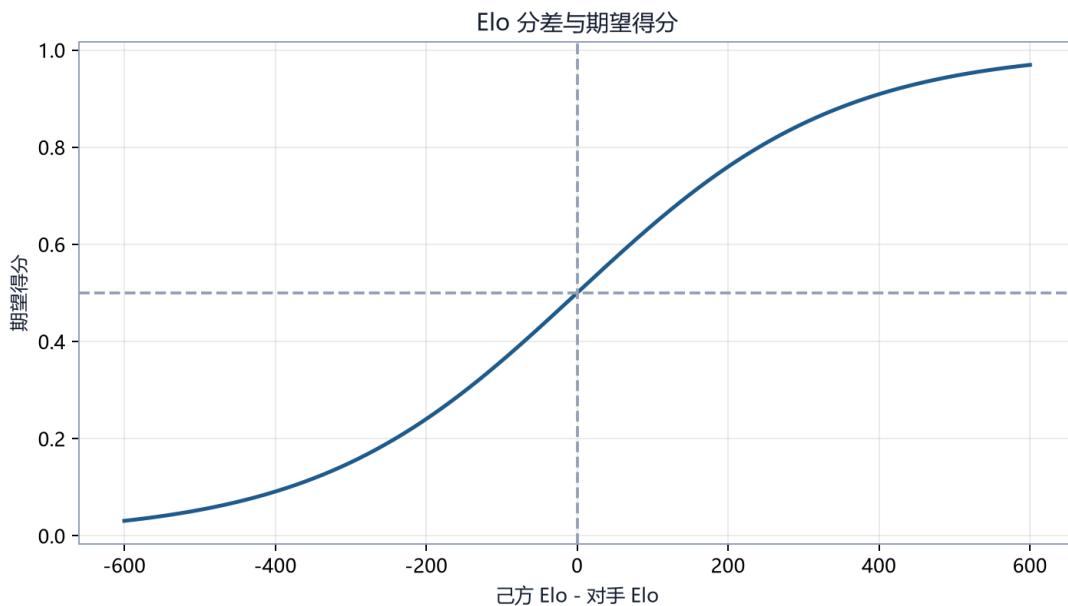
比赛实际得分 S_A 取胜 1、和 0.5、负 0。更新:

$$R_A' = R_A + K(S_A - E_A)$$

若 A 意外失败, $K = 32$:

$$R_A' = 1600 + 32(0 - 0.760) \approx 1575.7$$

B 对称增加约 24.3 分。



Elo 等级差与期望得分

25.5 Elo 的局限

Elo 假设一个近似一维、相对稳定的实力尺度。策略游戏常违反这些假设:

- A 克 B、B 克 C、C 克 A 的非传递循环;
- 地图改变相对强弱;
- 先手优势;
- 搜索预算不同;
- 模型仍在训练, 实力非平稳;
- 多局来自同一确定性轨迹, 不独立。

因此 Elo 是压缩摘要, 不能替代对阵矩阵。至少同时保存:

- 每一对的胜/和/负；
- 地图分层；
- 先后手分层；
- 原始比赛数；
- rating history；
- 计算使用的 K 与初始分。

25.6 循环赛调度

有 m 个参赛者，单循环无方向配对数：

$$\binom{m}{2} = \frac{m(m-1)}{2}$$

若每一对在每张地图上双方各先手 g 局，共 L 张地图，总局数：

$$G = \frac{m(m-1)}{2} \times L \times 2g$$

例如 6 个 Bot、3 张地图、每边 2 局：

$$G = 15 \times 3 \times 4 = 180$$

比赛预算很快增长。可先用小地图与少量局做冒烟，再用预先固定的完整协议正式评估。

25.7 TournamentConfig 的重要字段

项目 `TournamentConfig` 包含：

- `maps` 与 `map_pool_mode` ；
- `games_per_side` ；
- `max_turns` ；
- `output_dir`、`replay` 目录；
- `save_replays` ；
- `enabled_units` ；
- `rng_seed` ；
- `concurrent_games` ；
- 可选 LLM 日志与 API 间隔。

`validate()` 会检查地图存在、局数和回合数为正、模式取值、并发数、单位代码等。正式运行前先验证配置，避免比赛完成一半才发现地图或单位设置错误。

25.8 模型公平比较

比较两个 RL 模型时要锁定：

- 相同观察与动作空间；
- 相同 `pad_to_size` ；
- 相同单位集合和引擎覆盖；
- 相同战争迷雾；
- 相同动作掩码语义；
- 相同确定性设置；
- AlphaZero 相同 simulation 数；
- 相同硬件或至少报告推理时延；
- 换边和相同地图种子。

若一个模型使用搜索 200 次，另一个只前向一次，只报告胜率是不完整的。应同时报告每步墙钟时间或等计算预算结果。

25.9 回放是行为证据

聚合指标能发现异常，回放用来解释异常。应优先检查：

- 高回报但未获胜的局；
- `max_steps_truncate` ；
- 置信区间外的异常胜负；
- 先手和后手表现差异；
- 模型排名变化最大的地图；
- 高频无效动作或重复移动；
- 可夺取却不夺取的状态。

回放必须保存足够信息重建动作序列。项目测试包含 `replay determinism` 和 `save replay`，目的是确认同一动作记录能够重放为同一终局。

Trace 与完整 replay

`evaluate_model` 可把特定 `end_reason` 的逐步信息写成 JSONL trace。Trace 适合快速统计和文本诊断；完整 replay 适合在游戏界面中重现。二者都不应只保存最终得分。

25.10 模型比较的分层报告

推荐报告结构：

1. **总体结果**：胜/和/负、score、Elo；
2. **地图层**：每张地图结果；
3. **先后手层**：玩家 1/2；
4. **终局层**：HQ 捕获、消灭、回合和局、微动作截断；
5. **行为层**：动作、单位、经济、伤害、占领；
6. **效率层**：每步推理时间、峰值内存；
7. **不确定性**：局数、种子、置信区间；
8. **定性证据**：典型与失败回放。

25.11 最小实操

评估与锦标赛测试：

```
pytest -q `

tests/test_rl_evaluation.py `

tests/test_tournament_library.py `

tests/test_tournament_config.py `

tests/test_tournament.py `

tests/test_save_replay.py `

tests/test_replay_determinism.py
```

查看评估入口：

```
python scripts/eval_agent.py --help

python scripts/tournament.py --help
```

不要在正式报告里只引用 `--help`。应把实际调用命令、解析后的配置和输出 JSON 一并保存。

25.12 典型误判

只选 best checkpoint。

若 best 由很小评估集挑出，会有赢家诅咒。保留 final、best 和若干中间 checkpoint 的独立测试。

Elo 高就全面更强。

检查非传递对阵和地图分层。

多跑几次确定性 Bot 就等于增加样本。

没有随机 tie-break 时可能是重复 replay。

回放看起来合理就证明算法有效。

回放是解释工具，不代替统计样本。

截断都算和局就没有问题。

大量微动作截断往往是停滞或动作粒度问题，应单独报告。

25.13 本章小结

项目评估体系从单模型 episode 统计扩展到换边循环赛、Elo 和回放诊断。Elo 提供简洁相对尺度，但对阵矩阵、地图、先手和计算预算不可省略。可靠结论来自统计结果与行为证据相互验证。

25.14 练习

1. 计算 Elo 1500 对 1700 时前者的预期得分。
2. 8 个模型、4 张地图、每边 3 局的完整循环赛有多少局？
3. 为什么 AlphaZero 与 PPO 比较时必须报告搜索预算和推理时间？
4. 给出三类最值得人工查看的失败回放。
5. 设计一份能揭示非传递性的对阵报告。

第 26 章 LLM Bot：提示、状态序列化与安全执行

26.1 LLM Bot 不是强化学习训练

项目中的 LLM Bot 在每个游戏回合把状态转换成文本/JSON，请外部语言模型返回结构化动作，再校验并执行。它没有：

- rollout buffer;
- 策略梯度;
- Bellman 更新;
- 经验回放训练;
- 通过本局奖励更新模型参数。

因此它是推理型 Bot 与实验对手，不是本项目 RL 训练算法。LLM 可以参与行为比较、数据采集或生成示范，但这不改变它自身未在游戏中在线强化学习的事实。

LLM Bot 的推理流水线（不是 RL 训练）



LLM Bot 的数据管线

本章不调用任何付费 API，实操全部使用 mock 响应。

26.2 状态为什么要序列化

语言模型不能直接读取 Python `GameState` 对象。`_serialize_game_state()` 要把当前决策所需信息转为 JSON 可表示结构，例如：

- 当前玩家与回合；
- 地图尺寸、地形和建筑；
- 双方金币；
- 单位 ID、类型、坐标、生命、状态；
- 可用技能与冷却；
- 当前合法动作；
- 胜负状态。

序列化有三个目标：

1. **完备性**：做合法决策所需信息不能缺失；
2. **最小性**：删除与决策无关的内部对象，控制 token 和歧义；
3. **稳定性**：字段名、坐标系、单位 ID 不随一次调用意外变化。

序列化不是简单 `str(game_state)`。内部对象引用、枚举和循环关系既不适合 JSON，也会让模型无法精确引用某个单位。

26.3 玩家视角与信息边界

若启用战争迷雾，LLM 状态也必须遵守可见性。不能因为 LLM 接口在游戏进程内部，就把完整敌方单位列表直接暴露给它。否则与使用局部观察训练的 RL 模型比较不公平。

相对视角还需要统一：

- 坐标仍是绝对地图坐标，还是旋转到己方视角；
- `player` 字段使用真实编号，还是 `self/opponent`；
- 建筑所有者的编码；
- 单位 ID 是否只在本局稳定。

提示中的规则与序列化字段必须一致。

26.4 提示由规则、任务与格式组成

项目提供 basic、strategic 和 two-phase 等提示策略。一个有效的系统提示应包含：

- 胜利条件；
- 单位、建筑和关键战斗规则；
- 可用动作；
- 行动顺序约束；
- JSON 输出模式；

- 禁止额外自然语言的要求。

提示过短会导致规则幻觉，过长会增加延迟、费用和关键信息被淹没的风险。更重要的是，提示中的数值会随游戏平衡变化；若硬编码规则与当前 `UNIT_DATA` 不一致，LLM 即使完全遵循提示也会作出错误判断。

因此应增加“提示规则—当前常量”的一致性测试，而不是把 prompt 当成永远正确的静态文案。

26.5 两阶段规划

`two_phase_planning=True` 时，流程分两次调用：

1. 规划阶段输出局势判断、主要目标、动作序列和风险；
2. 执行阶段读取该计划与最新状态，输出可执行动作 JSON。

优点是把“决定做什么”与“填写精确坐标和 ID”分开，便于日志诊断。代价是：

- API 调用、token、费用和延迟约增加；
- 计划可能在动作逐条执行后失效；
- 第二阶段可能不遵守第一阶段；
- 第一阶段计划并不通过游戏引擎验证。

两阶段不是自动获得更强推理的保证，必须通过固定题集与对局实验比较。

26.6 结构化动作

典型响应概念上为：

```
{
  "reasoning": "先清除阻挡单位，再占领建筑。",
  "actions": [
    {"type": "attack", "unit_id": 2, "target_id": 7},
    {"type": "move", "unit_id": 3, "x": 5, "y": 4},
    {"type": "seize", "unit_id": 3},
    {"type": "end_turn"}
  ]
}
```

动作顺序有语义。第一步可能杀死目标、改变合法路径或结束游戏，后续动作必须依据更新后的 `GameState` 执行，不能只在响应开始时统一校验一次。

26.7 JSON 提取不是合法性验证

项目 `_extract_json` 依次尝试：

1. 整个文本直接 `json.loads` ；
2. Markdown JSON 代码块；
3. 文本中首尾花括号片段。

解析成功只说明语法是 JSON，不说明：

- 存在 `actions` ；
- `actions` 是列表；
- 动作类型已知；
- 单位属于当前玩家；
- 目标存在；
- 坐标在地图内；
- 技能未冷却；
- 动作在当前更新后的局面合法。

因此 `_execute_actions` 和各 `_execute_*` 方法仍需把单位 ID 映射回当前对象，并调用游戏规则校验。

26.8 逐动作重新校验

假设 LLM 返回：

1. 单位 2 攻击单位 7；
2. 单位 3 移动到单位 7 原位置；
3. 单位 3 占领。

若第一击没有击杀，第二步可能因位置占用而非法；若第一击直接赢得游戏，后续动作都应停止。正确执行器要在每一步：

- 检查 `game_over` ；
- 重新取得或验证对象；
- 依据当前状态执行；
- 捕获单步异常；
- 防止异常动作破坏整个回合；
- 最终在需要时安全结束回合。

这与 RL 动作掩码的思想相通：模型输出只是提议，环境规则才是合法性的最终裁决者。

26.9 失败恢复

`LLMBot.take_turn()` 的主流程包含重试。若调用失败或响应无效，Bot 应安全回退到结束回合，而不是让游戏主循环崩溃。

需要区分：

- 网络/API 错误；
- 超时或限流；
- 响应被截断；
- JSON 语法错误；
- schema 错误；
- 某个动作因状态变化无效；
- 模型主动 resign。

把所有失败都记为“模型选择 `end_turn`”会丢失诊断信息。日志应分别记录 stop reason、重试次数、原始响应、执行成功数和失败原因。

26.10 状态历史与 token

`stateful=True` 时，会把先前 user/assistant 消息加入后续调用。历史可能帮助模型保持计划，但也会：

- 重复发送大量旧状态；
- 让模型混淆旧单位位置；
- token 和延迟随回合增长；
- 把早期错误计划持续带入。

更稳健的做法是使用结构化短期记忆：保留上回合计划摘要，而把当前 `GameState` 始终作为权威事实。若使用完整历史，应明确最大长度与裁剪规则。

项目跟踪累计输入、输出 token，并可把每回合 prompt、响应、使用量和 stop reason 写入 JSON 日志。这些是比较 LLM Bot 成本不可缺少的数据。

26.11 延迟与成本模型

一局近似成本可写为

$$C = \sum_{t=1}^T (n_t^{\text{in}} c_{\text{in}} + n_t^{\text{out}} c_{\text{out}})$$

其中：

- T ：LLM 决策回合数；
- $n_t^{\text{in/out}}$ ：第 t 回合输入/输出 token；
- $c_{\text{in/out}}$ ：每 token 价格。

总延迟近似为

$$L = \sum_t (L_t^{\text{network}} + L_t^{\text{generation}}) + L^{\text{retry}}$$

价格与模型可随时间变化，本书不固化供应商报价。实验应记录调用时模型名、日期和实际 usage；本章也不发起真实请求。

26.12 安全边界

游戏中的 LLM 输出仍是不可信输入。执行器应：

- 只接受白名单动作；
- 限制每回合动作数量；
- 限制字符串与数组大小；
- 不把响应拼接成 Python 或 shell 执行；
- 不允许模型选择文件路径；
- 对日志中的 API key 做隔离；

- 对异常响应安全失败；
- 在提示中避免暴露不必要的环境变量与路径。

结构化输出降低解析歧义，不等于建立安全边界。

26.13 Mock 实操

运行已有测试，不需要 API key：

```
pytest -q tests/test_llm_bot.py tests/test_llm_prompts.py
```

测试应覆盖：

- 固定 GameState 的序列化快照；
- 纯 JSON、代码块 JSON、带前后文本 JSON；
- 缺少 actions；
- 未知动作类型；
- 错误单位 ID；
- 顺序执行后第二动作失效；
- API 异常后的重试和 end-turn fallback；
- token 统计与日志格式；
- 两阶段规划 mock。

一个最小 mock 的思想是继承 LLMBot，让 _call_llm 返回固定字符串。这样测试的是本地数据管线，不产生网络费用。

26.14 LLM 与 RL 的可比性

比较 LLM Bot 和 RL Bot 时应说明：

- LLM 是否看到完整规则文本；
- 两者是否看到同样的战争迷雾；
- LLM 一次返回整回合动作，RL 每次返回微动作；
- 是否允许两阶段规划；
- 每步/每局延迟；
- token 成本与硬件成本；
- 是否使用外部模型更新后的版本；
- 随机温度；
- 失败 fallback 如何计分。

否则“胜率更高”可能主要来自信息量和计算预算差异。

26.15 本章小结

LLM Bot 是一个状态序列化—提示—结构化响应—逐动作校验—安全回退的工程管线，而不是 RL 算法。决定其可靠性的往往不是提示辞藻，而是信息边界、schema、动态合法性校验、日志与失败处理。项目测试可以在完全不调用付费 API 的情况下验证这些关键环节。

26.16 练习

1. 解释 JSON 语法正确与游戏动作合法之间的区别。
2. 为什么多个动作必须按执行后的新状态逐个校验？
3. 为 LLM 响应设计一个最小 JSON Schema。
4. 列举 `stateful=True` 的两个收益和三个风险。
5. 设计一组不调用外部 API 的 LLM Bot 回归测试。

第 27 章 DEV 工具链与综合研究实践

27.1 从“能运行”到“可研究”

强化学习项目同时包含游戏规则、环境适配、算法、配置、评估与可视化。某一层悄悄变化，最终表现可能完全不同。DEV 工具链的职责是把这些层的契约固定下来，使问题能够复现、定位和回归验证。

本项目在 `pyproject.toml` 中把开发依赖列为：

- `pytest`、`pytest-cov`;
- `pre-commit`;
- `Ruff`;
- `mypy`。

这些工具不提高策略网络的表达能力，却决定实验结论是否建立在稳定实现之上。

27.2 测试金字塔

单元测试

验证纯函数或局部契约：

- `flat` 动作编码/解码；
- 掩码形状与至少一个合法动作；
- `GAE` 数值；
- `Elo` 更新；
- `JSON` 提取；
- 配置验证。

集成测试

跨模块验证：

- `StrategyGameEnv.reset/step` ；
- `MaskablePPO` 与 `wrapper`;
- `BC checkpoint` 加载到 `PPO`;

- Feudal rollout 与两层 update;
- MCTS 深复制后执行动作;
- replay 保存与重放。

冒烟测试

用极小预算跑完整路径:

- 64 个环境步;
- 1 次训练迭代;
- 1 局短自对弈;
- 1 个 checkpoint;
- 1 次导出。

冒烟成功表示接口贯通, 不表示算法效果。

统计回归

固定小任务和种子, 允许合理波动地检查:

- Noop 最低胜率;
- 无效动作率不超过上限;
- episode 不全部截断;
- loss 为有限数;
- 性能没有数量级退化。

统计回归不能写成要求每次 reward 完全相同, 除非环境和算法路径确实确定性。

27.3 质量检查命令

项目级检查可依次运行:

```
ruff check .  
  
mypy reinforcetactics  
  
pytest
```

`pyproject.toml` 当前配置 pytest 覆盖率门槛为 65%。只运行少量测试时, 全局覆盖率门槛可能导致 “测试本身通过, 但命令因覆盖率不足退出”。局部调试可明确使用:

```
pytest -q tests/test_alphazero.py --no-cov
```

最终提交前仍应运行项目规定的完整测试与覆盖率检查，不能把 `--no-cov` 当作验收结果。

27.4 配置优先级

一个可靠的配置系统应明确：

代码默认值

- < 配置文件
- < 命令行显式参数

项目训练脚本先从 YAML/JSON 读取配置并设置 `argparse` 默认值，再由 CLI 覆盖。嵌套的 `reward_config`、`engine_overrides`、`opponent_kwargs` 等不能简单靠标量参数表达，因此需要显式传入环境工厂。

诊断配置时保存三份证据：

1. 原始配置文件；
2. 命令行；
3. 程序实际解析后的完整配置。

只看 YAML 不足以证明训练环境真正收到了字段。

27.5 诊断顺序

当训练失败时，按由低到高的顺序排查：

规则层

- > 环境契约
- > 观察与动作
- > 奖励与终止
- > 算法数值
- > 对手与课程
- > 统计评估

规则层

用固定动作或规则 Bot 确认胜负、伤害、占领和经济。

环境契约

检查空间包含 observation/action, `terminated` 与 `truncated` 正确。

观察与动作

检查玩家相对视角、padding、掩码与真实合法动作一致。

奖励与终止

打印 breakdown, 确认终局脉冲、势能差分和截断 bootstrap。

算法数值

检查 NaN、梯度范数、value loss、entropy、KL 与 clip fraction。

对手与课程

确认当前实际对手、地图、阶段、checkpoint 交接和晋级门。

统计评估

最后才判断算法是否真的更强。

这个顺序避免在环境动作编码错误时先调整学习率。

27.6 最小可复现问题

一个训练问题通常可以缩小为：

- 单地图；
- 单环境；
- CPU；
- 固定种子；
- 64–2048 步；
- 无渲染；
- 最少 callback；

- 输出观察形状、掩码数、奖励分解和终止原因。

若问题在最小版本消失，再逐个恢复 VecEnv、多地图、自对弈、课程和大网络。一次恢复一个因素，才能定位触发条件。

27.7 日志与 checkpoint

训练产物应形成一个不可歧义的 run 目录：

```
run/  
  resolved-config.yaml  
  metadata.json  
  tensorboard/  
  checkpoints/  
  eval/  
  traces/  
  final_model.zip
```

checkpoint 不只是网络参数。要可继续训练，还可能需要：

- 优化器状态；
- 当前环境步数；
- 学习率日程；
- replay/rollout 相关状态；
- 当前课程阶段；
- 最佳评估指标；
- 自对弈快照池；
- 随机数状态。

缺少这些状态时，“resume”可能只是从权重重新开始一个不同实验。

27.8 综合实践：从短训练到研究报告

下面给出一条不依赖旧知识文件的完整路线。

阶段 A：环境验收

运行：

```
python agents/learning-guide-v2/tools/smoke_labs.py --only env
python agents/learning-guide-v2/tools/smoke_labs.py --only masks
```

保存 observation 形状、合法动作数、随机合法动作的 step 结果，以及 MultiDiscrete 各维掩码。

阶段 B：接口边界

验证：

- DQN + 默认 MultiDiscrete 必须得到清晰的不兼容结果；
- DQN + Flat Discrete 可以构造；
- A2C 可以构造，但普通 A2C 不消费掩码；
- MaskablePPO 构造并学习 64 步。

这一步的结果是软件能力矩阵，不是算法排名。

阶段 C：第一条训练曲线

使用 `labs/maskable-ppo-smoke.yaml` 在 CPU 上训练短预算。确认：

- TensorBoard 有 rollout 和 train 指标；
- 模型能保存；
- 固定 5 局评估能运行；
- 结果中有终局原因和奖励分解。

阶段 D：课程训练

运行 `labs/bootstrap-smoke.yaml`，门槛设为 0 只验证阶段迁移。正式实验再恢复合理门槛、最小阶段步数、耐心和多局评估。

阶段 E：进阶路径

依次执行：

- BC 数据管线与一次交叉熵更新；
- BC 权重加载到 PPO；
- 自对弈快照池采样；
- Feudal 两层 rollout；
- MCTS 2 次 simulation；
- AlphaZero 一个自对弈样本与一次更新。

不要同时调试全部路径。每项先在单独报告中通过，再组合。

阶段 F：正式研究问题

选择一个可证伪问题，例如：

在 5 个种子、每个 50 万环境步的预算下，Bootstrap 是否比固定 BalancedRandom 训练提高对 MediumBot 的独立测试胜率？

建立两组配置，固定其余变量，预先确定主要指标与测试种子。

阶段 G：报告

报告至少包括：

1. 问题与假设；
2. 代码 revision 和环境版本；
3. 完整配置；
4. 训练预算；
5. 多种子曲线；
6. 最终原始结果和置信区间；
7. 终局、动作、奖励分解；
8. 失败 run；
9. 典型回放；
10. 限制与下一实验。

27.9 教材构建与验证

新版教材提供四类命令：

```
python agents/learning-guide-v2/tools/generate_figures.py
```

```
python agents/learning-guide-v2/tools/smoke_labs.py
```

```
python agents/learning-guide-v2/tools/build_book.py
```

```
python agents/learning-guide-v2/tools/verify_book.py
```

它们分别：

- 重绘教材静态图；
- 运行低成本接口冒烟；
- 从 `book.json` 的同一章节清单生成连续 HTML 与 PDF；
- 检查章节、链接、图片、算法覆盖、PDF 文本和书签。

PDF 还需通过 Poppler 渲染为逐页 PNG，在 `tmp/pdfs/` 中检查：

- 中文字体缺字；
- 表格或代码越界；
- 公式被截断；
- 标题孤行；
- 图片模糊；
- 空白页；
- 页眉页码冲突。

自动检查不能代替视觉检查。

27.10 DEV 中的常见错误

修复训练问题却没有测试环境契约。

算法层可能只是掩盖底层错误。

复制配置后只改文件名。

必须验证解析值真的不同。

用冒烟结果宣称性能。

冒烟只验证路径。

忽略工作树已有改动。

实验和文档修改应保持范围清晰，不覆盖无关用户文件。

只保存 final model。

无法恢复训练上下文，也无法解释 best 与 final 差异。

测试用 mock，最终却从未做集成测试。

mock 隔离外部依赖；真实组件边界仍需本地、免费、可控的集成验证。

27.11 研究伦理与结果表述

报告应明确：

- 未运行的实验不能写成已验证；
- 历史结果不能冒充当前代码复现；
- 单种子结果不能表述为一般结论；
- 失败版本不能从实验索引中消失；
- LLM mock 不能冒充真实供应商性能；
- 教材示意数字要标注“算例”，历史数据要标注“历史数据”。

严格表述不会削弱研究价值，反而使下一位读者能准确接续工作。

27.12 全书实践终点

完成本章后，读者应能够：

- 从游戏机制定义 MDP/POMDP；
- 推导并解释回报、价值、GAE 和 PPO；
- 理解 PyTorch、Gymnasium、SB3/sb3-contrib 的职责；
- 追踪观察、动作、掩码、奖励和终止的数据链；
- 运行 MaskablePPO 主线；
- 分析 DQN、A2C 的适用边界；
- 使用课程、BC 与自对弈；
- 理解 Feudal、MCTS 和 AlphaZero 的实验实现；
- 设计带置信区间和消融的研究；
- 使用 Bot、Tournament、Elo、回放和 trace 评估；
- 在不调用外部 API 的条件下测试 LLM Bot 管线；
- 构建并验证整本教材。

27.13 本章小结

DEV 工具链把游戏、环境、算法和研究结论连接成可验证系统。最有效的排障方式是从规则与契约逐层上升；最可靠的研究流程是先用冒烟确认软件，再以多种子、独立评估和完整记录回答性能问题。

27.14 练习

1. 为什么局部 pytest 可能在测试都通过后仍因覆盖率门槛退出？

2. 写出一个训练配置的三层优先级，并说明如何保存解析结果。
3. 把“模型不学习”缩小成一个最小可复现问题。
4. 设计一个 checkpoint 恢复一致性测试。
5. 按本章模板提出一个关于奖励塑形的可证伪研究问题。

附录 A 数学符号、公式与术语速查

本附录供连续阅读时回查。它不是对正文的外部依赖：每个核心概念在正文首次出现时仍有完整解释。

A.1 记号约定

符号	读法	含义	常见范围
t	t	环境微动作时间下标	0, 1, 2, ...
s_t	s 下标 t	时刻 t 的状态	状态空间 $\mathcal{S}$
o_t	o 下标 t	智能体实际观察	观察空间 $\mathcal{O}$
a_t	a 下标 t	智能体动作	动作空间 $\mathcal{A}$
r_t	r 下标 t	一步奖励	实数
$\pi(a s)$	pi	状态下的动作概率	[0, 1], 对动作求和为 1
θ	theta	神经网络参数	实向量
γ	gamma	折扣因子	通常 [0, 1)
λ	lambda	GAE 偏差—方差参数	[0, 1]
α	alpha	学习率或其他权重	由上下文决定
ϵ	epsilon	PPO 裁剪宽度或探索率	由上下文决定
$V^\pi(s)$	V	状态价值	期望折扣回报
$Q^\pi(s, a)$	Q	动作价值	执行动作后的期望折扣回报
$A^\pi(s, a)$	A	优势	$Q^\pi - V^\pi$
G_t	G	从 t 开始的回报	奖励的折扣和
δ_t	delta	TD 残差	实数
$\mathbb{E}[X]$	E	随机变量的期望	实数
$\text{Var}(X)$	Var	方差	非负
∇_θ	nabla	对参数的梯度	与 θ 同形
$H(\pi)$	H	策略熵	非负
D_{KL}	KL	分布差异	非负

符号	读法	含义	常见范围
$N(s, a)$	N	MCTS 访问次数	非负整数
$P(s, a)$	P	MCTS 网络先验	$[0, 1]$
$W(s, a)$	W	MCTS 累计价值	实数

同一希腊字母在不同算法中可能有不同含义。比如 α 在 DQN 中常指学习率，在 Feudal 章节中又表示 Worker 外在奖励权重。判断含义必须看公式定义，不能只看字母。

A.2 集合、下标与条件

$\mathcal{S}$ 表示所有可能状态的集合，单个状态写作 $s \in \mathcal{S}$ 。花体大写通常表示空间或集合。

$$\pi(a \mid s)$$

竖线读作“在.....条件下”。它表示已知状态 s 时选择动作 a 的概率，不是除法。

$$x_{1:T} = (x_1, x_2, \dots, x_T)$$

表示从 1 到 T 的一段序列。

$$1[C]$$

是指示函数：条件 C 成立取 1，否则取 0。它常用于终局、合法性或到达目标奖励。

$$\arg \max_a f(a)$$

返回使 $f(a)$ 最大的动作，而 $\max_a f(a)$ 返回最大数值。两者不要混淆。

A.3 概率与统计

概率分布

离散动作策略满足：

$$\pi(a \mid s) \geq 0, \quad \sum_{a \in \mathcal{A}} \pi(a \mid s) = 1$$

动作掩码后，求和只在合法集合上进行。

期望

离散随机变量：

$$\mathbb{E}[X] = \sum_x p(x)x$$

若一次攻击 70% 造成 10 点、30% 造成 0 点，期望伤害为 $0.7 \times 10 + 0.3 \times 0 = 7$ 。期望 7 不表示某次一定造成 7 点。

方差与标准差

$$\text{Var} (X) = \mathbb{E}[(X - \mathbb{E}[X])^2]$$

$$\text{SD} (X) = \sqrt{\text{Var} (X)}$$

平方保证正负偏差不会抵消；开根号使标准差恢复到原变量单位。

协方差

$$\text{Cov} (X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])]$$

正值表示二者倾向同向变化，负值表示反向变化。相关不等于因果。

样本均值

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

训练曲线若只有一个种子，曲线上的多个时间点不是独立样本，不能用它们代替多种子均值。

样本方差

$$s^2 = \frac{1}{n-1} \sum_i (x_i - \bar{x})^2$$

分母使用 $n-1$ 是对有限样本估计总体方差的校正。

标准误

$$\text{SE} (\bar{x}) = \frac{s}{\sqrt{n}}$$

增加独立种子能降低均值估计误差，但收益按平方根增长：把标准误减半，约需四倍样本。

置信区间

大样本正态近似：

$$\bar{x} \pm 1.96 \text{ SE} (\bar{x})$$

95% 置信区间的频率学解释是：若重复相同抽样过程，约 95% 的此类区间覆盖真实参数；不是 “参数有 95% 概率落在本次区间”。

A.4 向量与矩阵

向量：

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$$

可表示全局特征或网络隐藏状态。

点积：

$$\mathbf{w}^T \mathbf{x} = \sum_{i=1}^d w_i x_i$$

每个输入乘权重再求和。线性层：

$$\mathbf{y} = W\mathbf{x} + \mathbf{b}$$

若 $\mathbf{x} \in \mathbb{R}^d$ 、输出 $\mathbf{y} \in \mathbb{R}^k$ ，则 $W \in \mathbb{R}^{k \times d}$ 、 $\mathbf{b} \in \mathbb{R}^k$ 。

形状检查是调试神经网络的第一工具。矩阵乘法 AB 只有在 A 的列数等于 B 的行数时成立。

A.5 导数、梯度与链式法则

一元导数：

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

它描述局部斜率。若损失随参数增加而上升，导数为正，梯度下降会减小参数。

多参数梯度：

$$\nabla_{\theta} L = \begin{bmatrix} \partial L / \partial \theta_1 \\ \vdots \\ \partial L / \partial \theta_d \end{bmatrix}$$

梯度下降：

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L$$

链式法则：

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial x}$$

深度网络的反向传播就是把这个规则从输出层反复应用到输入层。PyTorch `loss.backward()` 计算梯度，`optimizer.step()` 才更新参数，`optimizer.zero_grad()` 清除默认累积的旧梯度。

A.6 激活、softmax 与对数概率

ReLU:

$$\text{ReLU}(x) = \max(0, x)$$

Tanh:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \in (-1, 1)$$

Softmax:

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

为了数值稳定，实际实现会减去最大 logit:

$$p_i = \frac{e^{z_i - z_{\max}}}{\sum_j e^{z_j - z_{\max}}}$$

概率乘积在长序列中容易下溢，因此策略梯度使用对数概率:

$$\log \prod_t \pi(a_t | s_t) = \sum_t \log \pi(a_t | s_t)$$

对数不会改变概率大小次序，并把乘积变为求和。

A.7 监督学习损失

均方误差:

$$L_{MSE} = \frac{1}{B} \sum_i (\hat{y}_i - y_i)^2$$

项目价值网络、Q 网络和 AlphaZero 价值头都会使用平方误差。

交叉熵:

$$L_{CE} = - \sum_k y_k \log p_k$$

one-hot 标签时只保留正确类别的 $-\log p$ 。BC 用它学习专家动作；AlphaZero 则以软目标 π 对完整动作分布求和。

类别加权交叉熵：

$$L = -w_y \log p_y$$

可提高少数动作类别的贡献，但不能创造数据中从未出现的状态—动作关系。

A.8 回报与折扣

有限 episode 回报：

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

递推形式：

$$G_t = r_t + \gamma G_{t+1}$$

有效时间尺度粗略为：

$$H_{eff} \approx \frac{1}{1-\gamma}$$

$\gamma = 0.99$ 时约 100 步。这不是硬截止线，而是理解折扣衰减的尺度。

A.9 价值、Q 与优势

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$$

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$$

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

优势为正表示该动作优于状态下的平均策略，负值表示较差。它不是“会赢的概率”，单位与回报相同。

A.10 Bellman 方程

状态价值：

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma V^\pi(s')]$$

最优动作价值：

$$Q^*(s, a) = \mathbb{E}[r + \gamma \max_{a'} Q^*(s', a') \mid s, a]$$

Bellman 方程把长期量分为“一步奖励 + 下一状态的长期量”，是 TD、DQN 和 Actor-Critic 的共同基础。

A.11 MC、TD 与 GAE

Monte Carlo 目标使用完整回报：

$$y_t^{MC} = G_t$$

无 bootstrap，偏差低但方差高，必须等待 episode 或片段结束。

一步 TD 目标：

$$y_t^{TD} = r_t + \gamma(1 - d_t)V(s_{t+1})$$

TD 残差：

$$\delta_t = r_t + \gamma(1 - d_t)V(s_{t+1}) - V(s_t)$$

GAE：

$$\hat{A}_t^{GAE(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$$

$\lambda = 0$ 接近一步 TD， $\lambda = 1$ 接近长回报。对时间限制截断，只要底层 MDP 未终局，就通常应保留下一状态 bootstrap。

A.12 策略梯度

基本策略梯度：

$$\nabla_{\theta} J(\theta) = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t]$$

用优势替代回报：

$$\nabla_{\theta} J(\theta) \approx \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t]$$

若优势为正，提高该动作概率；若优势为负，降低概率。baseline $V(s)$ 在不依赖当前动作时不会改变期望梯度方向，却能降低方差。

A.13 熵与 KL

离散策略熵：

$$H(\pi) = - \sum_a \pi(a) \log \pi(a)$$

均匀分布熵高，确定性分布熵低。熵奖励鼓励探索，但熵过高可能使策略长期随机。

KL 散度：

$$D_{KL}(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

它不对称，不是严格距离。PPO 日志的 approximate KL 用于监控新旧策略变化幅度。

A.14 DQN 公式

目标：

$$y_t = r_t + \gamma(1 - d_t) \max_{a'} Q_{\theta^-}(s_{t+1}, a')$$

损失：

$$L(\theta) = \mathbb{E}[(Q_{\theta}(s_t, a_t) - y_t)^2]$$

θ^- 是目标网络参数，在一段更新期间保持较慢变化；replay buffer 打乱相邻样本相关性。DQN 要枚举单一离散动作的 Q 值，因此不直接兼容项目默认 MultiDiscrete CLI。

A.15 A2C 公式

A2C 使用同步 rollout，常用 n -step 回报：

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V(s_{t+n})$$

Actor 损失：

$$L_{\text{actor}} = - \log \pi(a_t | s_t) \widehat{A}_t$$

Critic 损失：

$$L_{\text{critic}} = (V(s_t) - \widehat{G}_t)^2$$

总损失加入价值系数和熵项。项目普通 A2C 构造路径不消费动作掩码，这是接口限制而非公式限制。

A.16 PPO 公式

概率比：

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} = \exp [\log \pi_{\theta} - \log \pi_{\theta_{old}}]$$

裁剪目标：

$$L^{CLIP} = \mathbb{E}[\min(r_t \widehat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \widehat{A}_t)]$$

最小值会在有利于目标、但变化过大的方向上限制收益。它不是把所有概率比强行裁到区间内，也不能单独保证参数距离很小。

总损失：

$$L = -L^{CLIP} + c_v L^V - c_e H$$

实现通常最小化 L ，所以策略目标前有负号，熵奖励前也有负号。

A.17 动作掩码

给合法性 $m_i \in \{0, 1\}$ ，masked logits：

$$\tilde{z}_i = \begin{cases} z_i, & m_i = 1 \\ -M, & m_i = 0 \end{cases}$$

M 为很大正数。softmax 后非法概率接近 0。

MaskablePPO 必须在采样和更新时对同一状态使用一致掩码；评估时也必须调用支持掩码的预测接口。

逐维 MultiDiscrete 掩码保证各维取值分别出现过，但不保证笛卡尔积组合合法。自回归条件掩码解决的是组合依赖。

A.18 奖励塑形

势能塑形：

$$F(s, a, s') = \gamma \Phi(s') - \Phi(s)$$

塑形后奖励：

$$r' = r + F$$

在标准条件下，这种形式保持最优策略不变。任意事件奖励不一定具有该性质。项目中伤害、击杀、占领等分量必须通过 reward breakdown 和终局结果共同诊断。

A.19 BC

行为克隆最大化专家动作似然：

$$\max_{\theta} \sum_i \log \pi_{\theta}(a_i | s_i)$$

等价最小化交叉熵。BC 的核心问题是分布偏移：部署时一次错误进入专家数据很少覆盖的状态，后续错误会累积。BC→PPO 用示范建立初始技能，再以环境交互修正。

A.20 自对弈

对手从历史池采样：

$$\pi_{\text{opp}} \sim P_{\text{pool}}$$

如果只对当前最新模型训练，对手随自己同步变化，容易循环与遗忘。历史快照、规则锚点和多样采样能减轻非平稳。

A.21 Feudal

Manager：

$$g_k \sim \pi_M(g \mid s_{kc})$$

Worker：

$$a_t \sim \pi_W(a \mid s_t, g_k)$$

项目 Worker 奖励：

$$r_t^W = r_t^{\text{int}} + \alpha r_t^{\text{ext}}$$

Manager 段折扣：

$$\gamma^{k_t}$$

不要把 `worker_reward_alpha` 误读成内外奖励凸组合。

A.22 MCTS

PUCT：

$$Q(s, a) + c_{\text{puct}} P(s, a) \frac{\sqrt{N(s) + 1}}{1 + N(s, a)}$$

访问策略：

$$\pi(a \mid s) = \frac{N(s, a)^{1/\tau}}{\sum_b N(s, b)^{1/\tau}}$$

玩家视角翻号、合法 flat 索引与深复制后的动作引用是项目实现的关键正确性点。

A.23 AlphaZero

样本：

$$(s, \pi, z)$$

联合损失：

$$L = (z - v)^2 - \pi^\top \log p + c \|\theta\|^2$$

π 来自 MCTS，不是实际单一动作； z 是玩家相对终局标签。

A.24 项目术语

环境微动作 (env step)：一次 `env.step(action)`。

游戏回合 (game turn)：一名玩家的一组单位行动与结束回合。

episode：从 reset 到 terminated 或 truncated 的一局。

stage：Bootstrap 中固定地图、对手、奖励与门槛的一段训练。

rollout：on-policy 算法采集的一段连续交互。

buffer：保存训练样本的数据结构；不同算法语义不同。

wrapper：包裹环境并修改接口或行为的对象。

VecEnv：让 SB3 同时管理一个或多个环境的接口。

checkpoint：训练中间状态；是否能完整恢复取决于保存内容。

snapshot：自对弈历史策略的冻结副本。

anchor：长期固定、用于跨阶段比较的评估对手。

shaping attractor：高 shaped reward 但不完成真实目标的策略区域。

invalid combination：MultiDiscrete 每维看似可选、组合后违反规则的动作。

exact mask：相对当前条件精确描述合法选择的掩码。

historical result：旧代码、旧配置或旧环境得到的记录，未自动代表当前复现。

附录 B 命令、配置与参数参考

B.1 统一执行环境

本书验收基线：

日期	2026-07-29
Conda 环境	reinforce-tactics
Python	3.12.13
Gymnasium	1.3.0
Stable-Baselines3	2.9.0
sb3-contrib	2.9.0
PyTorch	2.13.0+cpu

在未激活环境时，所有命令可加前缀：

```
conda run -n reinforce-tactics python <script> <arguments>
```

先确认版本：

```
conda run -n reinforce-tactics python -c `

"import sys,gymnasium,stable_baselines3,sb3_contrib,torch; print(sys.version);
print(gymnasium.__version__,stable_baselines3.__version__,sb3_contrib.__version__,
torch.__version__)"
```

本书不包含 Cloud/Vertex 执行命令，也不要求任何 LLM API key。

B.2 教材构建

生成静态图：

```
conda run -n reinforce-tactics python `

agents/learning-guide-v2/tools/generate_figures.py
```

运行所有教材冒烟：

```
conda run -n reinforce-tactics python `
agents/learning-guide-v2/tools/smoke_labs.py
```

只运行一个项目：

```
conda run -n reinforce-tactics python `
agents/learning-guide-v2/tools/smoke_labs.py --only masks
```

允许的 `--only` 值由脚本固定，发布版包括环境、掩码、算法兼容、MaskablePPO、MCTS/AlphaZero 等低成本检查。

构建同源 HTML 与 PDF：

```
conda run -n reinforce-tactics python `
agents/learning-guide-v2/tools/build_book.py
```

PDF 构建阶段使用 Chromium/Edge 打印同源 HTML，使 MathML 公式在 PDF 中保留真正的上下标、分式、根号和希腊字母。构建器自动查找系统 Chrome/Edge；非标准安装位置可通过 `GUIDE_BROWSER` 指定。还需保证 Node.js 能解析 Playwright，Codex 工作区运行时已内置该依赖。

验证章节、链接、图片、算法覆盖、HTML、PDF 文本与书签：

```
conda run -n reinforce-tactics python `
agents/learning-guide-v2/tools/verify_book.py
```

输出位置：

```
agents/learning-guide-v2/export/Reinforce-Tactics-RL-Learning-Guide-combined.md
agents/learning-guide-v2/export/Reinforce-Tactics-RL-Learning-Guide.html
output/pdf/Reinforce-Tactics-RL-Learning-Guide-v2.pdf
```

B.3 测试命令

完整测试和覆盖率：

```
conda run -n reinforce-tactics pytest
```

环境、观察和动作：

```
conda run -n reinforce-tactics pytest -q `
    tests/test_rl_env.py `
    tests/test_rl_observation.py `
    tests/test_rl_action_mask.py `
    --no-cov
```

具体测试文件名以仓库当前文件为准。本教材验证器会检查正文引用路径是否存在；若测试重命名，应同步更新教材命令。

Feudal：

```
conda run -n reinforce-tactics pytest -q `
    tests/test_feudal_rl.py `
    tests/test_feudal_rl_integration.py `
    --no-cov
```

AlphaZero：

```
conda run -n reinforce-tactics pytest -q `
    tests/test_alphazero.py `
    --no-cov
```

LLM mock：


```
conda run -n reinforce-tactics pytest -q `

tests/test_llm_bot.py `

tests/test_llm_prompts.py `

--no-cov
```

Tournament 与 replay:

```
conda run -n reinforce-tactics pytest -q `

tests/test_tournament_library.py `

tests/test_tournament_config.py `

tests/test_tournament.py `

tests/test_save_replay.py `

tests/test_replay_determinism.py `

--no-cov
```

`--no-cov` 只用于局部快速反馈。最终验收运行完整 `pytest`，接受 `pyproject.toml` 中 65% 覆盖率门槛。

B.4 环境构造参数

`StrategyGameEnv` 的关键概念字段:

参数	类型	用途
<code>map_file</code>	路径或 <code>None</code>	固定 CSV 地图或随机地图
<code>opponent</code>	字符串	规则 Bot / 模型对手类型
<code>opponent_kwargs</code>	字典	对手构造参数
<code>max_steps</code>	整数	环境微动作截断上限
<code>max_turns</code>	整数或 <code>None</code>	游戏回合上限
<code>max_actions_per_turn</code>	整数或 <code>None</code>	单游戏回合的智能体微动作上限
<code>action_space_type</code>	字符串	<code>multi_discrete</code> 或 <code>flat_discrete</code>
<code>max_flat_actions</code>	整数	Flat 路径合法动作容量/防护参数
<code>fog_of_war</code>	布尔	是否启用局部可见性
<code>enabled_units</code>	列表或 <code>None</code>	允许单位代码
<code>pad_to_size</code>	二元组或 <code>None</code>	跨地图统一观察尺寸
<code>reward_config</code>	字典或 <code>None</code>	奖励分量
<code>engine_overrides</code>	字典或 <code>None</code>	经济、单位数据等稀疏覆盖
<code>gold_scale</code>	浮点	全局金币特征归一化尺度
<code>turn_scale</code>	浮点	回合特征尺度
<code>unit_count_scale</code>	浮点	单位数量尺度
<code>render_mode</code>	字符串或 <code>None</code>	训练通常为 <code>None</code>

最小交互：

```
from reinforcetactics.rl.gym_env import StrategyGameEnv

env = StrategyGameEnv(

    map_file="maps/1v1/starter.csv",

    opponent="noop",

    max_steps=64,

    action_space_type="multi_discrete",

    render_mode=None,

)

obs, info = env.reset(seed=0)

action = env.action_space.sample()

obs, reward, terminated, truncated, info = env.step(action)

env.close()
```

若使用掩码，随机采样必须从合法分布产生；直接 `action_space.sample()` 主要用于接口压力测试，会产生大量无效组合。

B.5 PPO 与 MaskablePPO 参数

参数	含义	影响
learning_rate	优化器步长	过大不稳，过小学习慢
n_steps	每环境每次 rollout 步数	越大回报统计更长、更新更慢
batch_size	minibatch 大小	必须适配 rollout 样本数
n_epochs	同一 rollout 重用轮数	过大可能使策略偏离旧策略
gamma	折扣	越高重视更远回报
gae_lambda	GAE 参数	调节偏差一方差
clip_range	PPO 裁剪宽度	限制有利方向更新收益
ent_coef	熵系数	鼓励探索
vf_coef	价值损失系数	平衡 critic 与 actor
max_grad_norm	梯度范数上限	防止单次极端更新
policy_kwargs	网络与 extractor	决定表示能力
device	cpu/cuda/auto	运行设备

rollout 大小:

$$N_{\text{rollout}} = n_{\text{steps}} \times n_{\text{envs}}$$

例如 4 个环境、n_steps=256，一次更新收集 1024 个样本。batch_size=64 时每 epoch 约 16 个 minibatch。

MaskablePPO 要求:

- 环境暴露 action_masks();
- 训练采样使用 MaskablePPO;
- 评估使用 MaskableEvalCallback 或显式向 predict 传 action_masks;
- 自定义 SubprocVecEnv 场景中，掩码方法要存在于子环境内部。

B.6 MaskablePPO 最小配置

教材 `labs/maskable-ppo-smoke.yaml` 的目标是短训练，不是推荐正式超参数：

```
algorithm: maskable_ppo

total_timesteps: 64

seed: 0

env:

  map_file: maps/1v1/starter.csv

  opponent: noop

  max_steps: 64

  max_turns: 12

  action_space_type: multi_discrete

  n_envs: 1

ppo:

  learning_rate: 0.0003

  n_steps: 32

  batch_size: 16

  n_epochs: 1

  gamma: 0.99

  gae_lambda: 0.95

  clip_range: 0.2

  ent_coef: 0.01

  vf_coef: 0.5

  max_grad_norm: 0.5

  device: cpu
```

短预算把 `n_epochs` 降为 1 以减少 CPU 时间; 不能据此比较最终性能。

B.7 Bootstrap

运行教材最小课程:

```
conda run -n reinforce-tactics python `
    scripts/train/train_bootstrap.py `
    --config agents/learning-guide-v2/labs/bootstrap-smoke.yaml `
    --output-dir tmp/learning-guide-v2/bootstrap `
    --device cpu `
    --skip-plots `
    --skip-videos `
    --sanity-episodes 0 `
    --no-gcs
```

通用入口:

```
conda run -n reinforce-tactics python `
    scripts/train/train_bootstrap.py `
    --config configs/ppo/bootstrap.yaml `
    --output-dir benchmarks/bootstrap/local-run `
    --device cpu `
    --set ppo.learning_rate=0.0003 `
    --skip-videos `
    --no-gcs
```

重要 CLI:

参数	作用
<code>--config</code>	Bootstrap YAML
<code>--output-dir</code>	run 目录
<code>--device</code>	<code>cpu/cuda/auto</code>
<code>--set KEY=VALUE</code>	可重复的点路径覆盖
<code>--build-bc</code>	课程前创建 BC warm start
<code>--bc-scenarios</code>	示范场景 YAML
<code>--bc-epochs</code>	BC epoch
<code>--skip-plots</code>	跳过图
<code>--skip-videos</code>	跳过 replay 视频
<code>--sanity-episodes</code>	最终 sanity eval 局数, 0 跳过
<code>--no-gcs</code>	强制不上传; 本书本地实验使用

阶段配置常见字段:

```
curriculum:

  stages:
    - name: starter_noop

      map_file: maps/1v1/starter.csv

      opponent: noop

      promotion_win_rate: 0.0

      n_eval_episodes: 2

      eval_freq: 32

      max_timesteps: 64

      patience: 1
```

正式训练需要合理的 `promotion_win_rate`、`min_timesteps_before_promotion`、`patience`、`eval_freq`、评估局数和阶段预算。门槛为 0 只验证控制流。

B.8 行为克隆

构建 checkpoint:

```
conda run -n reinforce-tactics python `
    scripts/build_bc_warmstart.py `
    --scenarios configs/imitation/bc_beginner_warmstart.yaml `
    --curriculum-config configs/ppo/bootstrap_sweep/v33_production_bc_warmstart.yaml `
    --output tmp/learning-guide-v2/bc/bc_warmstart.zip `
    --epochs 1 `
    --batch-size 32 `
    --learning-rate 0.0003 `
    --seed 0 `
    --map-file maps/1v1/beginner.csv `
    --enabled-units W M C A K `
    --max-turns 75 `
    --end-turn-weight 30
```

参数解释:

- `--scenarios` : 示范对手、地图和采样组合;
- `--curriculum-config` : 读取下游 PPO 的 extractor 与 net architecture, 保证精确加载;
- `--output` : 生成的 MaskablePPO checkpoint;
- `--epochs`、`--batch-size`、`--learning-rate` : 监督训练;
- `--seed` : 示范与训练随机种子;
- `--end-turn-weight` : 结束回合样本权重, 修正示范类别不均衡;
- `--enabled-units`、`--map-file`、`--max-turns` : 必须与下游空间匹配。

`--end-turn-weight=1` 关闭额外加权。自动值或较大权重用于对抗“永不结束回合”的类别不平衡吸引子，但必须用 held-out expert action accuracy 和实际 rollout 共同验证。

B.9 自对弈

下列命令是完整 mixed 训练入口。`n_envs` 必须至少为 2，因为脚本会把环境分别分配给规则 Bot 与自对弈对手。当前基准环境还需要安装 SB3 进度条的可选依赖 `rich` 与 `tqdm`；若没有这两个包，脚本会在创建进度条时退出，而不是进入训练：

```
conda run -n reinforce-tactics python `

scripts/train/train_self_play.py `

--config agents/learning-guide-v2/labs/self-play-smoke.yaml `

--mode mixed `

--use-opponent-pool `

--pool-size 2 `

--pool-strategy uniform `

--n-envs 2 `

--no-subprocess `

--total-timesteps 64 `

--n-steps 32 `

--batch-size 16 `

--n-epochs 1 `

--eval-freq 64 `

--n-eval-episodes 1 `

--checkpoint-freq 64 `

--device cpu `

--log-dir tmp/learning-guide-v2/self-play
```

教材的零依赖冒烟验证不安装额外软件，而是直接检查历史池、采样权重、换边环境、动作掩码以及一步交互：

```
conda run -n reinforce-tactics python `

agents/learning-guide-v2/tools/smoke_labs.py `

--only self-play
```

这一组件级命令不等价于完成一次策略更新；它的目的，是把“自对弈数据管线可用”和“完整 CLI 在当前依赖集下可以运行”区分开来。

池参数：

参数	含义
--opponent-update-freq	当前对手更新间隔
--use-opponent-pool	启用历史池
--pool-size	最大快照数
--pool-strategy	uniform/recent/prioritized
--add-to-pool-freq	添加候选快照间隔
--min-win-rate-for-pool	加入池的门槛
--bot-ratio	mixed 模式规则 Bot 比例
--swap-players	随 episode 换边；当前帮助显示默认开启

只有 64 步时，池可能还没到添加周期。教材冒烟会直接测试池对象，不能把“训练命令退出成功”当成已验证历史采样。

B.10 Feudal

运行：

```
conda run -n reinforce-tactics python `

scripts/train/train_feudal_rl.py `

--mode feudal `

--config agents/learning-guide-v2/labs/feudal-smoke.yaml
```

关键参数：

参数	含义
manager_horizon	一个目标持续的最大微动作数
worker_reward_alpha	外在奖励在 Worker 合成奖励中的乘数
manager_lr_scale	Manager 相对基础学习率
worker_lr_scale	Worker 相对基础学习率
autoregressive_worker	使用阶段条件 Worker
reward_scale	外在奖励进入层级 buffer 前的缩放

当前 Worker 公式：

$$r^W = r^{int} + \text{worker_reward_alpha} \times \text{reward_scale} \times r^{env}$$

不要把 alpha 当作 $(1 - \alpha, \alpha)$ 插值。

B.11 AlphaZero

完整脚本入口：

```
conda run -n reinforce-tactics python `

scripts/train/train_alphazero.py `

--map-file maps/1v1/starter.csv `

--res-blocks 1 `

--channels 16 `

--num-simulations 1 `

--c-puct 1.5 `

--dirichlet-alpha 0.3 `

--iterations 1 `

--games-per-iter 1 `

--epochs-per-iter 1 `

--batch-size 2 `

--buffer-size 128 `

--max-game-steps 4 `

--temperature-threshold 2 `

--eval-games 0 `

--eval-threshold 0.55 `

--lr 0.001 `

--weight-decay 0.0001 `

--checkpoint-dir tmp/learning-guide-v2/alphazero `

--device cpu
```

如果只验证搜索而不希望候选比赛耗时，使用：

```
conda run -n reinforce-tactics python `

agents/learning-guide-v2/tools/smoke_labs.py --only mcts
```

关键参数：

参数	作用
<code>--res-blocks</code> 、 <code>--channels</code>	网络容量
<code>--num-simulations</code>	每真实动作搜索次数
<code>--c-puct</code>	PUCT 探索
<code>--dirichlet-alpha</code>	根噪声形态
<code>--games-per-iter</code>	每轮自对弈局数
<code>--epochs-per-iter</code>	buffer 训练轮数
<code>--buffer-size</code>	历史样本容量
<code>--temperature-threshold</code>	一局内从采样转向贪心的步数
<code>--eval-games</code>	候选对最佳对局数
<code>--eval-threshold</code>	接受候选胜率

B.12 评估

查看精确接口：

```
conda run -n reinforce-tactics python scripts/eval_agent.py --help
```

库接口：

```
from reinforcetactics.rl.evaluation import evaluate_model

result = evaluate_model(

    model,

    env,

    n_episodes=100,

    deterministic=True,

    seed=10_000,

    track_breakdown=True,

    trace_dir="tmp/eval/traces",

    trace_end_reasons=("max_steps_truncate",),

)
```

正式报告保存 `rewards`、`lengths` 原始数组，而不只保留均值。

B.13 Tournament

查看入口：

```
conda run -n reinforce-tactics python scripts/tournament.py --help
```

配置核心：

```
{
  "name": "local-bot-ladder",
  "maps": [
    {"path": "maps/1v1/starter.csv", "max_turns": 50}
  ],
  "games_per_side": 2,
  "map_pool_mode": "all",
  "output_dir": "tmp/learning-guide-v2/tournament",
  "save_replays": true,
  "rng_seed": 20260729,
  "concurrent_games": 1
}
```

`rng_seed` 使整体可复现，同时让同一对阵的多局在 tie-break 上产生差异。没有 RNG 的确定性 Bot 可能重复完全相同轨迹。

B.14 TensorBoard

从 run 根目录启动：

```
conda run -n reinforce-tactics tensorboard `
  --logdir tmp/learning-guide-v2 `
  --port 6006
```

重点指标：

- `rollout/ep_rew_mean`、`ep_len_mean`；
- `train/entropy_loss`；
- `train/value_loss`；
- `train/approx_kl`；
- `train/clip_fraction`；

- `train/explained_variance` ;
- 课程阶段胜率、终局原因;
- Feudal 两层 loss 与奖励分量;
- AlphaZero policy/value loss。

不同奖励配置的 `ep_rew_mean` 不能直接横向比较。

B.15 配置覆盖与审计

Bootstrap `--set` 示例:

```
--set ppo.learning_rate=0.0001  
  
--set env.enabled_units='[W,M,C,A,K]'  
  
--set env.reward_config.draw=-50
```

PowerShell 中列表和字符串的引号要保留给 Python 解析。运行后检查 resolved config，而不是仅相信命令行输出。

配置审计清单:

1. 地图存在;
2. 所有阶段空间形状一致，或正确 padding;
3. warm start 的网络与空间完全匹配;
4. reward 字段进入环境;
5. engine override 已应用;
6. 评估对手与训练对手按设计分离;
7. `max_steps` 与 `max_turns` 的单位没有混淆;
8. 掩码训练与评估均开启;
9. 输出目录不覆盖旧 run;
10. 版本、种子和完整解析配置落盘。

附录 C 历史实验索引与因果证据表

C.1 怎样阅读本附录

本附录整理仓库保留的 Bootstrap 实验配置和两次 2026 年 7 月审查。所有数值与结论均标记为**历史记录**，其含义是：

- 当时的代码、配置、硬件与随机轨迹产生过该现象；
- 当前仓库可能已经修复、重构或改变默认值；
- 除非教材冒烟或独立复现实验明确说明，不能把历史结果当作 2026-07-29 当前代码的重复验证。

证据分为五栏：

1. **当时现象**：日志或结果中看到了什么；
2. **调查证据**：支持某个解释的指标、回放或消融；
3. **已排除原因**：至少有对照证据反对的解释；
4. **已落地变化**：代码或配置中已经存在的修正；
5. **开放问题**：仍需新的独立实验。

配置文件名是溯源坐标，不代替本附录中的解释。

C.2 v15–v23：课程门槛与熵调度阶段

版本	主要变更	历史现象	后续证据如何修正理解
v15	降低后段 entropy floor	在 random_10 附近停滞	不能只归因于对手；使用五单位 roster 和当时奖励/经济
v16	加入 turn penalty 等多项变化	形成新的课程墙	同时改 roster、奖励与经济，混杂严重
v17	random 阶段 patience 调整	仍在 random_15 附近停滞	延长/耐心不是充分修复
v18	全局推广 random patience	与 v17 相近	说明局部门槛波动并非唯一根因
v19	random_10 consolidation	当时记录为较深高水位	后续发现“最佳配置”判断受混杂影响
v20	降低 balanced-random entropy	更早退化	熵太低可使探索提前收缩
v21	拆分 consolidation 的 entropy 变化	未突破	同种子重复阶段也显示 CUDA/评估噪声
v22	缩减单位 roster	当时表现差，曾被解释为 roster 失败	后续 v26 等证明“缩减 roster 必败”是错误结论；v22 还缺 Cleric 并叠加其他变化
v23	降低 random_15/20 晋级门	仍未解决根因	放松门槛只能改变晋级判定，不能创造稳定策略

这一阶段最重要的方法论结论

当时曾形成三类过早结论：

- 缩减单位集合会灾难性失败；
- random_15 是单纯容量墙；
- 调整 patience/threshold 可能解决。

后续 v24–v27 的配置复现与单变量消融表明，奖励项和版本不一致才是关键混杂。这个案例说明：在没有逐字段配置 diff 之前，不能把“版本号相邻”当作“只有一个变量不同”。

C.3 v24–v28：复现与奖励项隔离

v24：看似忠实、实际不忠实的复现

`v24_reproduce_deep_config` 试图复现历史深进度配置，却仍在较早阶段停滞。最初由此怀疑现代引擎或经济变化。

后续调查发现，v24 仍携带三个与真正深配置不同的奖励项：

- `win_speed_bonus` ;
- `enemy_neutral_capture` ;
- `enemy_owned_capture` .

因此 v24 不是严格的忠实复现。

v25：random_10 二分诊断

v25 用更窄阶段缩小问题位置。它的价值主要是定位，而不是提供最终训练方案：单阶段探针可以回答“checkpoint 在这个对手上是否仍有能力”，却不能替代完整课程的分布迁移。

v26：真正对齐奖励后的复现

`v26_faithful_deep_reward_on_head` 在现代代码上把上述三个项归零，并重现更深进度。由此排除“现代引擎必然导致课程失败”这一宽泛解释，确认关键差异可由配置表达。

历史对照的核心关系：

配置	代码时代	三个奖励项	历史结果
v24	现代	开启/含惩罚	较早停滞
v26	现代	全部归零	清过对应课程块

v27a/b/c: 单项回加

版本	仅回加的项	历史结果	因果判断
v27a	win_speed_bonus: 50	停滞，峰值约 0.80	独立有害证据
v27b	enemy_neutral_capture: -8	5/5 清除，约 0.9875	在该条件下相对无害
v27c	enemy_owned_capture: -15	停滞，峰值约 0.90	独立有害证据

这组消融比“v24 对 v26”更强，因为每次只回加一个候选项。结果还说明，有害项不一定使胜率归零；它们可能让策略偶尔越过门槛，却不能维持 patience 所需的连续评估。

v28: 生产奖励修正版

v28 把 v26/v27 的结论合入较完整课程，并改进阶段 handoff。它成为后续多项实验的基线之一。其深度不应被简化为“奖励已经全部解决”，因为 random_15 的策略漂移、评估噪声和后续阶段仍存在。

C.4 v29–v32: 时间上限、warm start 与晋级噪声

版本	变更	历史观察	结论
v29	random_15 的 max_turns 探针	从更长前缀进入仍可停滞	仅增加游戏回合上限不是充分条件
v30	从 random_10 峰值 checkpoint 单阶段 warm start	能清 random_15	checkpoint 质量与交接点重要；不是“给同一 stage 更多任意训练”
v31	加 min_timesteps_before_promotion=500k	比 v28 更早停在 random_10	强制等待可错过暂时但真实的可晋级窗口
v32	去掉强制门，random_15 评估局数 80→160	目标是降低门判噪声	更多评估降低方差，但不修复策略本身

v31 的具体历史轨迹中，350k 与 400k 的评估曾达到约 0.925 和 0.8375，却因最小步数门不能晋级，之后能力下降。这说明“训练更久”不是单调改进；on-policy 策略会继续漂移。

v30 与 v31 不矛盾：前者从特定峰值 checkpoint 开始单阶段训练，后者改变完整课程中的晋级时机。实验问题不同。

C.5 v33: BC warm start

v33 使用规则示范训练 BC 初始策略，再进入 PPO。历史上它帮助跨过 random_15 墙，并暴露新的问题：

- 示范动作类别严重不均衡；
- `end_turn` 在每回合只出现一次，非结束动作很多；
- 未加权交叉熵会形成“永不结束回合”倾向；
- BC checkpoint 与下游 PPO 的 observation/action space、extractor、net architecture 必须精确匹配。

已落地机制包括结束回合样本权重、按下游配置构造同结构模型和 BC→PPO 加载检查。BC 解决的是探索与初始技能，不自动解决后续奖励吸引子。

C.6 v34–v40: 单一单位构成吸引子

v34: 深进度不等于策略多样

v34 开启更积极的战斗塑形，历史上清过多个阶段，但单位构成几乎完全是 Warrior。这证明：

- 课程深度和胜率不能充分描述策略；
- shaped reward 可以加速已有行为，却不一定创造多样战略；
- 应记录 `units_built` 和构成比例。

v35–v37: 四类杠杆

版本	主要杠杆	历史结果摘要
v35	Warrior 攻防削弱，加入 intermediate r10/r15，放宽回合	在 beginner r10 附近漂移
v36	v35 + r10 预算 1.5M→3M	仍不能证明构成问题解决
v37a	撤销削弱 + MixedBot 桥接	能通过桥接，构成仍可能单一
v37b	撤销削弱 + 更高门槛	清 15 个阶段且每段曾达 100%，仍为单一 Warrior

v37b 是关键反例：更高晋级门和更深课程没有自动产生单位多样性。

v38 与 v39：结构削弱和成本削弱

- v38 把 Warrior 攻击、防御、HP 一起降低；
- v39 保留战斗数值，只把成本 200 提到 300。

两者都在紧时钟 starter 地图遇到新墙。它说明平衡改动会改变学习可达性，尤其在地图时间预算很短时。不能只根据目标构成判断 nerf 是否合理。

v40：跳过 starter

v40 从 beginner 开始课程，使成本 nerf 有足够经济与时间空间发挥。它把“学习通用能力”和“在极短 starter 时钟下适应成本变化”分开。

开放问题是：跳过 starter 后的模型是否仍在 starter 上可靠。课程不训练某分布，就不能假定泛化。

C.7 v41–v46：击杀刷分、和局与对手多样性

v41：kill-farm draw plateau

v41 在 r10 使用多项稳定杠杆，较快清除部分阶段，却在 r15 形成高战斗、低终结的和局平台。历史行为证据表现为：

- 能击杀、能获得战斗塑形；
- 夺取比例低；
- 大量对局到达回合或微动作上限；
- episode reward 与胜率脱钩。

这比“容量不足”更符合奖励吸引子。

v42：去掉战斗刷分

v42 仅把 `damage_scale` 与 `kill` 归零。方向上减少了击杀刷分，却仍停在相近墙，并出现训练更稀疏、廉价支持单位偏好等新表现。

因而“加战斗奖励”和“完全去掉战斗奖励”都不是完整解。需要考虑对称代价、终局、占领可达性和对手分布。

v43a/b: 对手多样性与容量配对

版本	变更	历史峰值/阶段	解释
v43a	Mixed 对手多样性, 网络 [256,256]	r15 峰值约 0.8375, 较深	
v43b	v43a + [512,512]	反而早一阶段停	容量不是当时主要瓶颈

v43a 能通过更难的 mixed r10, 支持 “对手多样性减轻策略漂移” 。但它仍没有单独消除可获利和局。

v44–v46

- v44 调整 HQ 收入, 考察经济肉墙与治疗循环;
- v45 调整 r15 entropy floor, 考察探索收缩;
- v46 去掉 unit-difference 奖励, 考察屯兵与单位数塑形。

这些版本继续从经济、探索和奖励三条轴隔离问题。它们必须结合各自配置 diff 与 run 结果阅读, 不能把后续版本号视为单调改进。

C.8 v48–v54: 动作空间、对称战斗、截断与可占结构

仓库没有 v47 配置。版本号缺口不是文档错误; 实验索引应忠实保留。

版本	主要实验轴	需要回答的问题
v48	MultiDiscrete + opponent diversity	flat 路径容量/截断是否是主因
v49	对称战斗代价、负和局、强化 HQ 胜利	让受伤也付代价、和局不再是安全港后是否终结
v50	HP-scaled damage model	机制层伤害与奖励尺度是否更一致
v51	预算、entropy floor、patience	在新奖励下是否需要更稳阶段节奏
v52a	按 max_turns 缩放 draw	不同地图时间上限下和局惩罚是否可比
v52b	v52a + 更高 max_flat_actions	flat 合法动作容量是否截断
v53	可占结构相关变化	夺取瓶颈是“不能夺”还是“不愿夺”
v53b	v53 + HQ income	HQ 经济是否造成防守/恢复吸引子
v53c	entropy stability	全局熵设置是否造成 100% 和局平台
v54	放宽微动作上限与前沿配置	max_steps 是否提前截断深局，同时检查关闭 guard 的风险

v49 的奖励几何

历史配置的核心意图：

- win_by_hq_capture 提高；
- draw 改为明显负值；
- unit_diff 归零；
- 增加承受伤害的对称代价；
- 提高 HQ capture 的即时价值。

这不是简单“奖励更大”，而是试图让战斗奖励接近净交换，并让结束游戏优于持续安全刷分。

v52a 与 v52b

v52a 针对不同阶段 max_turns 缩放和局惩罚。v52b 在此基础上提高 flat 动作容量，用于排除大军队局面的合法动作截断。若 max_legal_actions 接近上限，只有提高容量后仍观察相同瓶颈，才能较有力排除截断。

v53 系列

该系列把注意力从“策略为什么不选 seize”推进到：

- 当前决策点是否真的出现 seize；
- 哪类结构可占；
- HQ 收入和恢复是否让对手形成肉墙；
- entropy 是否使策略永远保持过度随机。

这也是当前 `evaluate_model` 保留 `seize_available_rate`、`captures_by_type`、治疗经济与军队/金币指标的背景。

v54

v54 提高 `max_steps`，给长对局更多微动作空间。2026-07-24 审查提醒：提高上限只减少“被提前截断”，也可能关闭防止“永不结束回合”策略无限拖延的护栏。必须同时看：

- `max_steps_truncate` 比例；
- `max_turns_draw`；
- 每回合微动作数；
- end-turn 频率；
- 终局类型。

C.9 2026-07-12 训练审查

该次审查集中于“为什么 Bootstrap run 不能稳定清课”。主要历史发现：

1. 默认 `bootstrap.yaml` 与深 run 的奖励修正不完全一致；
2. 多个深 run 在仍晋级时因外部停止结束，不能简单写成算法停滞；
3. 高正回报可与 100% 和局并存；
4. Mixed 对手、多样性、负和局和 entropy floor 是相关杠杆；
5. 许多结论基于单种子，证据强度不足；
6. 需要把奖励、终止、动作空间和观察尺度的最终解析值落盘。

这次审查的重要贡献不是提出一个万能新配置，而是把“训练停滞”拆成外部终止、评估噪声、奖励吸引子和配置漂移。

C.10 2026-07-24 全管线审查

第二次审查扩展到 RL pipeline：

- self-play RNG 与对手推理热路径;
- flatten 与空间 extractor;
- lr_schedule 等配置字段是否真正传到 SB3;
- 最大合法动作数与 flat 容量;
- reward 默认值与实验最佳值漂移;
- auto-heal、收入、军队规模等诊断;
- 50/56 run 使用同一种子, 无法支持强泛化结论;
- 历史 run 与当前代码的比较边界。

审查还强调: 修复观察、动作或奖励语义后, 修复前后的数字不再严格可比。此时应重跑一个明确基线, 而不是把旧曲线继续拼在新曲线上。

C.11 跨版本因果链

奖励吸引子链

终局回报稀疏

- > 加入战斗/单位/占领塑形
- > 早期学习变快
- > 策略发现可重复获得塑形但不终局
- > 高 reward + 高和局
- > 对称代价、负和局、HQ 目标与行为诊断

课程交接链

短评估偶然越门

- > checkpoint 交到更难阶段
- > 能力漂移或崩溃
- > patience / 最小步数 / 更多评估
- > 发现强制训练也会错过能力峰值
- > 保留 best、固定锚与交接审计

单位构成链

便宜单位在多个轴占优

- > PPO 收敛到单一单位
- > 课程胜率仍很高
- > 战斗塑形进一步强化
- > 统计 `units_built` 暴露问题
- > 成本/属性/地图时钟消融

对手分布链

固定对手轨迹窄

- > 策略学会特定节奏
- > 切换阶段时分布跃迁
- > MixedBot / 历史池
- > 提高多样性但引入非平稳
- > 固定锚点评估

C.12 当前应如何复现历史结论

推荐顺序：

1. 固定当前代码 revision;
2. 选择一个结论，不一次复现全部 v15-v54;
3. 找到最小成对配置;
4. 用配置 diff 确认只有目标变量不同;
5. 至少 3-5 个种子;
6. 使用独立固定评估集;
7. 保存 resolved config;
8. 报告胜/和/负、终局、奖励与行为;
9. 将结果写为“当前复现”并与“历史记录”分栏;
10. 若不能复现，先检查版本、默认值与数据管线，不直接判定旧记录错误。

C.13 版本总索引

v15 lower_entropy_floors
v16 turn_penalty
v17 random_patience
v18 random_patience_global
v19 consolidate_random10
v20 lower_balanced_random_entropy
v21 split_consolidate
v22 reduced_units
v23 lower_random_threshold
v24 reproduce_deep_config
v25 bisect_random10_repro
v26 faithful_deep_reward_on_head
v27a ablate_win_speed_bonus
v27b ablate_enemy_neutral_capture
v27c ablate_enemy_owned_capture
v28 production_reward_fixed
v29 random15_maxturns_probe
v30 random15_warmstart_probe
v31 production_minsteps_gate
v32 drop_gate_higher_eval
v33 production_bc_warmstart
v34 aggressive_combat
v35 warrior_nerf
v36 warrior_nerf_more_budget
v37a revert_nerf_mixed_bridge
v37b revert_nerf_higher_thresholds
v38 structural_warrior_nerf
v39 cost_only_nerf
v40 skip_starter
v41 r10_stability
v42 remove_combat_farm
v43a opponent_diversity
v43b opponent_diversity_capacity
v44 opp_diversity_hq_income_cut
v45 opp_diversity_r15_entropy_floor
v46 opp_diversity_no_unit_diff
v47 仓库无对应配置
v48 multidiscrete_opp_diversity
v49 symmetric_combat_negative_draw
v50 hp_scaled_damage
v51 budget_floor_patience2
v52a maxturn_scaled_draw
v52b maxturn_scaled_draw_plus_maxflat

```
v53  capturable_structures  
v53b capturable_structures_hq_income  
v53c entropy_stability  
v54  uncapped_frontier
```

附录 D 章节练习参考答案

参考答案给出一种可接受解法。设计题通常没有唯一答案，判断重点是变量、数据路径和证据是否清楚。

D.00 第 0 章

1. 策略游戏的占领、经济与终局常隔着几十到数百个微动作，早期动作的价值必须由远期结果回传，因而直接展示信用分配；单步分类的标签在当前样本就已给出。
2. 训练短跑只覆盖很少状态，主要验证环境、反向传播和保存接口。偶然一胜或 loss 下降不能证明在地图、多对手上形成稳定策略。
3. 若运行了错误解释器，依赖版本、包路径甚至项目代码都可能不对。此时调学习率不可能修复 import 或 ABI 问题，应先确认 `sys.executable` 与版本。
4. 领域层执行规则与状态转移；环境层把规则包装成 observation/action/reward/termination；算法层根据轨迹更新策略或价值。

D.01 第 1 章

1. 生产单位会立即减少金币，却增加未来占领、作战和防守能力；从当前余额看是负，从长期回报看可能为正。
2. 何时结束回合影响行动顺序、被反击风险和未来收入。若环境自动结束，智能体无法学习“继续行动还是交权”的权衡，也无法表达整回合组合策略。
3. 不足以。评估任务变难本身会降低胜率。要在相同地图、对手、种子和协议上比较新旧 checkpoint，才能判断模型是否退化。
4. 前一个动作会改变单位位置、生命、金币、冷却和占领状态，使原合法动作失效，也会创造新动作。

D.02 第 2 章

1. GUI 动画帧数、按钮高亮或音效属于显示信息，通常不应直接作为奖励；它们与战略目标没有稳定因果关系。
2. 完整真实状态不能由当前观察唯一确定：被迷雾遮挡的敌军位置不同，可能产生相同观察。最优决策需依赖观察历史或信念状态。
3. $0.99^{200} \approx 0.134$ 。也就是 200 步后的终局奖励在 G_0 中只保留约 13.4% 权重。

4. 依赖表示包已安装或某些模块可使用；当前主训练 `StrategyGameEnv` 仍是 `Gymnasium` 单智能体接口，对手在环境内部推进，并没有暴露 `PettingZoo` 的逐 agent API。

D.03 第 3 章

1. 胜率 $\hat{p} = 14/20 = 0.7$ 。正态近似标准误：

$$\sqrt{0.7 \times 0.3/20} \approx 0.102$$

小样本更宜报告 Wilson 区间。

2. 它们共享相同初态与确定性决策，不含新的随机信息。样本数形式上增加，信息量没有增加，是伪重复。
3. 相同均值可能来自稳定中等表现，也可能来自一半极高、一半极低；还可能一个多胜负、另一个多和局。部署风险和能力含义不同。
4. 标准误与 $1/\sqrt{n}$ 成正比：

$$\sqrt{25/100} = 1/2$$

D.04 第 4 章

1. $1 \times 2 + 2 \times 0 + 3 \times (-1) = -1$ 。
2. 更新 $\theta' = \theta - 0.05(-4) = \theta + 0.2$ ，所以增加 0.2。
3. 多个线性层复合仍是一个线性变换，不能表示弯曲决策边界。非线性激活使网络能逼近复杂函数。
4. 卷积共享局部模式参数，保留邻接与平移结构；纯展平把相邻格与远格仅视为不同列，需要更多数据重新学空间关系。

D.05 第 5 章

1. $G_0 = 2 + 0.9 \times 0 + 0.9^2 \times 3 = 4.43$
2. $A = Q - V = 4 - 7 = -3$ 。在其他条件相同时，策略梯度应降低该动作概率。
3. MC 使用完整真实回报，bootstrap 偏差低但受整条随机轨迹影响、方差高；一步 TD 使用当前价值估计，方差低但引入函数近似偏差。
4. 更高 γ 只减慢折扣，仍有探索不足、奖励尺度、价值误差、长方差和动作组合问题；过高还会增大方差并使 critic 更难拟合。

D.06 第 6 章

1. `x.permute(0, 3, 1, 2)` 得到 `(8,32,10,12)`。

2. `eval()` 切换 dropout/batch norm 等模块行为；`no_grad()` 停止记录自动求导图、降低内存。推理通常两者都要。
3. 梯度裁剪只限制更新幅度。奖励正负号错会让优化目标方向错，裁剪只能使错误方向走得慢。
4. 普通平均把补零区也算进分母，不同地图 padding 比例会改变特征尺度。掩码平均只除以真实有效格数量。

D.07 第 7 章

1. `observation`, `reward`, `terminated`, `truncated`, `info`。
2. `max_turns` 是游戏规则定义的和局，可作为终局；`max_steps` 是环境采样时间限制，底层游戏未必结束，属于截断。
3. $512 \times 4 = 2048$ 条转移。
4. 空间、返回值和 `dtype` 正确只证明 API 合规；状态是否满足 Markov、奖励是否表达目标、截断是否正确 bootstrap 仍是建模问题。

D.08 第 8 章

1. `MultiInputPolicy` 是处理 Dict observation 的策略网络/特征入口；PPO 是用 rollout、GAE 和裁剪目标更新该策略的算法。
2. PPO 的概率比依赖生成样本的旧策略。策略多次更新后，久远数据与当前分布偏离，普通 PPO 目标不再可靠；DQN 是 off-policy，并用 Bellman 目标处理历史转移。
3. 否则评估分布允许训练时概率为零的非法动作，测到的是另一种策略，并可能制造大量 invalid penalty。
4. `observation` 字段/形状/`dtype`、`action space`、padding、单位集合、`feature extractor`、`net architecture`、掩码形式和环境语义。

D.09 第 9 章

1. `reset` 会创建或替换局面对象。旧 Bot 若仍引用旧 `GameState`，会在不可见的旧局面行动，当前环境状态不变或错乱。
2. 势能差分第一步会多出 $\gamma\phi(s_1) - 0$ ，把初始局面的绝对势能误当作智能体动作创造的进展。
3. CLI 默认构造 `MultiDiscrete`，而 SB3 DQN 只支持单一 `Discrete action space`；必须显式走 Flat Discrete 路径。
4. `GameState` 经观察编码生成 `grid(H,W,Cg)`、`units(H,W,Cu)`、`global_features(F)`；`VecEnv` 增加 batch；CNN 将空间张量转为 (B,C,H,W) ，与全局特征融合，再由策略头输出各动作维 logits 或 flat logits。

D.10 第 10 章

1. 相同位置和生命下，已行动单位不能再次移动/攻击；缺少该信息会让同一观察对应不同合法转移，破坏 Markov 近似。
2. 单整数让网络误以为类别有大小和等距关系，例如地形代码 4 比 2 “大两倍”。one-hot 把类别作为无序标识。
3. MultiDiscrete 的坐标维大小由地图尺寸决定；观察补到同一大小不自动改变 action space 的 `nvec`。必须统一动作维或使用另一编码。
4. 构造两个真实状态，使可见区域完全相同、迷雾内敌军不同；编码结果在所有可观察通道必须相同，合法动作也不能依赖隐藏敌军。

D.11 第 11 章

1. 逐维掩码描述各取值的边际可用性。合法动作 A 的来源与合法动作 B 的目标交叉组合后，可能从未出现在真实合法集合。
2. 它给每个当前合法结构化动作一个精确索引，消除组合误差；但索引语义可能随状态合法动作列表排序改变，网络难以形成稳定“动作编号含义”，还受容量上限影响。
3. 更新时要重算采样动作概率。若掩码不同，概率比的分子分母来自不同支持集；评估不传掩码又会评测非法动作分布。
4.
$$p(a | s) = p(a_{type} | s)p(a_{src} | a_{type}, s)p(a_{unit} | a_{type}, a_{src}, s)p(a_{target} | a_{type}, a_{src}, a_{unit}, s)$$

D.12 第 12 章

1.
$$F = 0.99 \times 22 - 20 = 1.78$$
2. 智能体可用重复移动、无效组合或其他微动作避免触发扣分，同时继续取得某些正塑形，形成拖延。
3. 规则和局使底层 MDP 终局，不应从下一状态 bootstrap；时间限制截断只是采样结束，若有最终观察，价值目标通常保留 bootstrap。
4. 小奖励乘以数百次可超过一次终局奖励。必须比较“单次幅度 × 可重复次数”，而非只看配置中的单个数字。

D.13 第 13 章

1.
$$y = 1 + 0.99 \times 8 = 8.92$$

预测误差 $Q - y = 6 - 8.92 = -2.92$ ，平方误差约 8.5264。
2. 若目标也随每个梯度步快速变化，网络在追逐自己移动的预测。冻结或软更新目标网络使监督目标短期更平稳。

3. Flat 动作数大、每状态合法动作稀疏、索引语义和长 horizon 仍使 DQN 困难；可构造只证明接口接受。
4. `main.py --algorithm dqn` 默认环境为 MultiDiscrete，而 SB3 DQN 的 supported action spaces 只有 Discrete，构造阶段即拒绝。

D.14 第 14 章

1. 若题中采用本章算例 $r_t, r_{t+1}, V(s_{t+2})$ ，代入
$$G_t^{(2)} = r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2})$$
再以 $A = G_t^{(2)} - V(s_t)$ 计算。答案必须写出是否在终局处去掉 bootstrap。
2. Actor 输出动作分布参数；Critic 输出状态价值 $V(s)$ 。
3. 实现最小化 loss，而我们希望最大化熵，所以写为 $-c_e H$ 。
4. 普通 A2C 不消费项目动作掩码。它虽接受 MultiDiscrete，却会在巨大非法组合空间上浪费样本。

D.15 第 15 章

1. $r = 0.5/0.4 = 1.25$ 。
2. 裁剪比为 1.2。未裁项 $1.4 \times 3 = 4.2$ ，裁剪项 $1.2 \times 3 = 3.6$ ，取最小为 3.6。
3. 数据仍由当前/旧一版策略采集，只在有限 epochs 内重复；它没有像 DQN 那样长期复用任意历史 buffer。
4. 示例：逐维掩码的组合无效；奖励塑形诱导和局；课程交接噪声；观察展平导致空间信息利用差；对手过窄。任列两个并说明证据即可。

D.16 第 16 章

1. 把策略分布限制到当前合法动作支持集，并在该 masked distribution 上采样与计算 log probability。
2. 64 步通常不足一局或只产生极少更新，只能发现接口、NaN 和保存问题。
3. 它影响 action space 形状与策略输出层。训练/评估不同会导致模型空间不匹配，或 flat 索引映射语义不同。
4. 递进例：① 64 步无异常并保存；② 固定小任务多局能明显优于随机；③ 多种子、独立地图/对手上胜率提升且无效动作与截断率正常。

D.17 第 17 章

1. patience 要求连续通过，可降低单个高点触发；但多次反复评估后，最终被选中的连续高点仍有选择偏差，且都使用同一开发集。
2. 每步切换会让同一局对手策略非平稳、行为不连贯；按 episode 抽样既提供局间多样性，又保持局内一致。
3. 达到峰值后继续 on-policy 更新可能漂移。阶段末是最后参数，阶段最佳是开发指标峰值参数。
4. 检查评估原始胜/和/负、置信区间、历史峰值、终局原因、动作与奖励分解、checkpoint 是否正确交接、实际配置/对手、是否外部中断。

D.18 第 18 章

1. 负对数损失从 $-\ln 0.2 \approx 1.609$ 降到 $-\ln 0.6 \approx 0.511$ ，降低约 1.099。
2. 重复只增加相同状态—动作的权重，不增加状态覆盖，模型在偏离专家轨迹后的状态仍无数据。
3. BC 数据只有专家动作标签，没有由未来奖励计算的回报标签；项目也明确让价值头留给 PPO 拟合。
4. 精确参数加载要求 state dict 形状一致；空间或 extractor 不同会改变输入层、输出头和参数键。

D.19 第 19 章

1. 当前镜像与自己同步变化，可能只保留当前策略循环，忘记过去能克制的战略。历史池提供滞后对手和能力记忆。
2. 对固定规则 Bot、冻结历史模型和固定地图锚点测胜率/Elo；自我对局 50% 只说明相对对称。
3. observation 所有者/己敌通道、金币与全局特征、合法动作与掩码、奖励与 winner 视角；还包括当前玩家和模型输入。
4. 初始随机策略互相产生低质量、近乎随机数据。Bot 课程或 BC 先建立合法动作、终局与基本战术，再自对弈扩展。

D.20 第 20 章

1. $0.98^8 \approx 0.8508$
段跨越 8 个微动作，使用 0.98 会把远期价值高估为只经过一步。
2. 缩放外在奖励 $500 \times 0.001 = 0.5$ 。Worker 奖励：
 $3 + 0.2 \times 0.5 = 3.1$
3. Manager 可能提出易到达但无战略价值的坐标；或内在奖励过强，Worker 为到达目标牺牲防守/终局目标。
4. PPO 概率比必须比较同一支持集上的新旧概率。更新时换掩码会使采样动作概率和重算概率不可比。
5. 做 2×2 ：传统/自回归 Worker $\times$ reward_scale=1/0.001，其余固定，使用相同种子集合；主看胜率，次看 invalid、value loss、到达率，估计两个主效应与交互。

D.21 第 21 章

1.
$$U = 1.5 \times 0.2 \times \frac{\sqrt{100}}{10} = 0.3$$

总分 $Q + U = 0.6$ 。
2. 去除根随机扰动并用最大访问动作，使评估尽量测当前模型/搜索能力，降低非必要方差并可复现。
3. $\tau = 1$: $[9, 4, 1]/14 \approx [0.643, 0.286, 0.071]$ 。
 $\tau = 0.5$: 平方为 $[81, 16, 1]$ ，除以 98，约 $[0.827, 0.163, 0.010]$ 。
4. 深复制后的单位是新对象；原引用属于父状态。用原对象执行会查找失败或修改错误状态，必须按坐标/ID解析到副本。
5. 搜索会利用智能体观察不到的敌方位置与状态，相当于给决策器全知信息，评估不公平并破坏 POMDP 定义。

D.22 第 22 章

1.
$$L_p = - [0.6 \ln 0.75 + 0.4 \ln 0.25] \approx 0.727$$
2. 同一终局对赢家状态应是正标签，对输家状态应是负标签；不翻转会让相似输入被矛盾训练，价值失去玩家相对意义。
3. 太大：旧策略目标滞后、适应慢；太小：样本相关、多样性不足、遗忘旧局面、过拟合最近自对弈。
4. $11/20=0.55$ 。在 $p \approx 0.5$ 时标准误约 $\sqrt{0.25/20} = 0.112$ ，一局差远小于不确定性，不能有力确信更强。
5. 规定相同墙钟或总网络前向/环境转移预算；多种子训练；独立换边对局；AlphaZero 报 simulations，PPO 报推理成本；比较胜率、时间和资源，而非迭代名称。

D.23 第 23 章

1. 均值 $(0.3 + 0.4 + 0.4 + 0.5 + 0.9)/5 = 0.5$ 。0.9 是明显高点，其余多在 0.3–0.5；只报均值掩盖方差、偏态和不稳定。
2. win rate = 0.60; non-loss = $(60 + 20)/100 = 0.80$; 半分 score = $(60 + 10)/100 = 0.70$ 。
3. 它们共享同一 run 历史与参数，存在强自相关，不是独立重训。
4. 两组使用相同算法、总步数、网络、地图集合、种子和最终评估；实验组按课程变对手/地图，对照组固定目标难度。最终都在同一独立对手矩阵测试，并报告学习曲线面积与最终表现。
5. 错误例：新组同时改奖励、网络 and 对手。改为三次成对实验，每次从同一基线只改一项；若关心交互，再用完整因子设计。

D.24 第 24 章

1. 每回合动作吞吐量决定生产、移动和攻击覆盖，改变转移分布与压力；不是同一随机策略的无关实现参数。
2. 保持一局内对手意图一致，避免每回合策略突变；同时在局间提供混合分布。
3. 固定地图/双方/配置，多局保存 action history 哈希。若所有哈希相同，则是重复；设置 `rng_seed` 后应整体可复现但局间哈希有多样性。
4. 对 Simple 高胜、对 Medium/随机 tie-break 崩溃；动作序列固定针对 Simple 目标顺序；换地图或先手即大降；对历史 Simple 变体遗忘。任三项。
5. 只把 Warrior cost 从 200 改为 300，保持单位属性、收入、Bot、地图和种子；换边多局比较胜率、Warrior 构成、总生产和回合长度。

D.25 第 25 章

1.
$$E = \frac{1}{1 + 10^{(1700 - 1500)/400}} = \frac{1}{1 + 10^{0.5}} \approx 0.240$$
2.
$$\binom{8}{2} \times 4 \times 2 \times 3 = 28 \times 24 = 672$$
3. AlphaZero 每步做多次状态复制和网络评估，PPO 通常一次前向。只看胜率会把额外计算当成免费，无法判断效率或等预算能力。
4. 大量截断；高回报却失败/和局；模型间结论相反或置信区间外异常；可夺取却不夺取。任三类。
5. 给出完整行列对阵矩阵，每格含换边 W/D/L 与局数，另按地图分层；明确标出 A>B、B>C、C>A 的环，而不只按 Elo 排序。

D.26 第 26 章

1. JSON 正确只满足文本语法；游戏合法还要求动作 schema、当前玩家、对象存在、坐标、资源、冷却和执行时状态都满足规则。
2. 前一动作会杀死/移动单位、花金币、改变冷却或终局，使后续合法集合变化。
3. 最小 schema 应要求顶层 object、必需 actions 数组；每项必需 type 且枚举白名单；按类型用 `oneOf` 要求 unit_id/target_id/x/y；限制 `maxItems`；禁止额外关键字段。
4. 收益：保持跨回合计划、减少重复解释。风险：旧状态污染、token/成本增长、上下文超限、早期错误持续、隐私/日志面扩大，任三项。
5. 固定状态序列化快照；多种 JSON 提取；未知动作与错误 ID；顺序失效动作；调用异常重试/fallback；token 与日志；两阶段 mock。所有 `_call_llm` 返回本地固定字符串。

D.27 第 27 章

1. `pyproject.toml` 给 `pytest` 配了全包覆盖率与 65% 门槛。只运行一个文件即使断言全过，也只覆盖很少包，最终 `coverage` 检查失败。
2. 代码默认 < 配置文件 < 显式 CLI。保存原 YAML、原命令和程序合并后的 `resolved config`；同时记录代码 `revision`。
3. 固定一张小地图、Noop、单环境、CPU、种子 0、64–2048 步、无渲染；输出 `observation/action space`、合法掩码、`reward breakdown`、终止、`loss` 与梯度。先确认能否学最小任务，再逐项恢复复杂性。
4. 在固定种子训练 `N` 步保存；记录参数、优化器、计数和一次固定观察预测；加载后验证预测/计数一致，再继续 `M` 步，与不中断 `N+M` 步 `run` 比较允许的确定性误差。
5. 示例假设：在 5 种子、50 万步、相同对手/网络下，势能形式的结构控制塑形相比事件式占领奖励，提高独立 `MediumBot` 测试胜率且降低重复占领奖励占比。预先规定指标、种子和消融配置。

附录 E 源码符号索引与原始文献

E.1 索引使用原则

正文已经包含理解概念所需的解释。本附录中的路径用于：

- 核验当前实现；
- 定位调试入口；
- 继续阅读完整上下文；
- 追踪论文和框架定义。

路径不承担“请读者自行查看源码才能理解正文”的责任。行号会随代码变化，所以索引以稳定的模块与符号名为主。

E.2 游戏领域层

`reinforcetactics/core/game_state.py`

`GameState` 是规则状态的核心聚合对象，负责：

- 地图、玩家、金币、单位与当前回合；
- 创建单位；
- 移动、攻击、占领；
- heal、cure、paralyze、haste、buff；
- 合法动作枚举；
- 回合推进、胜负与和局；
- 动作历史和 replay 相关状态。

阅读规则时从 `get_legal_actions` 与各动作执行方法成对核对：前者定义“可以提议什么”，后者定义“执行后怎样变化”。

`reinforcetactics/constants.py`

集中保存单位数据、经济和战斗常量。实验的 `engine_overrides` 会在运行时稀疏覆盖其中一部分语义。分析历史平衡时必须记录覆盖后的有效值，而不只看源文件默认。

`reinforcetactics/game/mechanics.py`

放置战斗或移动的共享规则辅助函数。若 Bot、GUI 与 RL 环境出现规则不一致，应检查它们是否都通过同一领域入口执行。

E.3 规则 Bot

`reinforcetactics/game/bot_base.py`

- `BaseBot` : Bot 共同接口;
- `BotUnitMixin` : 距离、tie-break、动作辅助和诊断记录。

`reinforcetactics/game/bot.py`

- `NoopBot` : 只结束回合;
- `RandomBot` : 均匀采样合法非结束动作, 受 `max_actions` 限制;
- `BalancedRandomBot` : 先尝试生产, 再让每个单位随机行动一次;
- `SimpleBot` : 局部启发式、单位能力和目标优先;
- `MediumBot`、`AdvancedBot`、`MasterBot` : 逐级丰富威胁、组合和战术;
- `MixedBot` : 每 episode 抽取一个内部对手。

Bot 名称是实现层级, 不是数学保证的严格实力顺序。

`reinforcetactics/game/model_bot.py`

把训练模型包装为游戏 Bot。核验点包括 observation 玩家视角、动作解码、动作掩码、设备与 eval/no_grad 推理。

`reinforcetactics/game/alphazero_bot.py`

把策略价值网络与 MCTS 接到游戏 Bot 接口; 评估时应明确 simulation 数、温度和根噪声。

E.4 Gymnasium 环境

`reinforcetactics/rl/gym_env.py`

关键符号:

- `StrategyGameEnv` : 主 Gymnasium 环境;

- `StructuredActionMasks` : 自回归动作条件掩码容器;
- `build_per_dim_masks` : `MultiDiscrete` 逐维掩码;
- `build_structured_masks` : 阶段条件掩码;
- `build_flat_actions` : `Flat Discrete` 合法动作映射。

`StrategyGameEnv.reset` 负责新建局面与对手, `step` 负责解码当前玩家微动作、执行、奖励、必要时推进对手回合, 并返回 `Gymnasium` 五元组。

`reinforcetactics/rl/observation.py`

- `build_observation` : 把 `GameState` 转换为 `Dict observation`;
- `GRID_CHANNELS`、`UNIT_CHANNELS`、`GLOBAL_FEATURES_DIM` : 网络输入契约;
- 玩家相对所有者编码;
- `padding`、战争迷雾与尺度处理。

观察通道改动会影响所有保存模型和 `AlphaZero` 网络输入, 属于公共训练契约变化。

`reinforcetactics/rl/masking.py`

- `ActionMaskedEnv` : 向 `sb3-contrib` 暴露掩码;
- `make_maskable_env` : 单环境工厂;
- `make_maskable_vec_env` : 向量环境工厂;
- `validate_action_mask` : 掩码诊断。

`SubprocVecEnv` 使用时, 掩码方法必须在子进程环境内可调用。

E.5 特征提取与策略网络

`SpatialFeatureExtractor` 的实际定义可用:

```
rg -n "class SpatialFeatureExtractor" reinforcetactics
```

它接收 `grid`、`units` 与 `global_features`, 用卷积保存空间局部关系, 再融合全局向量。核验:

- `NHWC` 到 `NCHW` 的转换;
- `padding` 是否参与 `pooling`;
- 输出 `features_dim`;
- `policy_kwargs` 是否在训练与加载时一致。

普通 `MultiInputPolicy` 的默认 `CombinedExtractor` 会分别展平/处理 Dict 字段，不等于项目自定义空间 CNN。

E.6 配置

`reinforcetactics/rl/config.py`

包含环境、PPO、课程阶段与总训练配置的数据结构，负责 YAML/JSON 解析、默认值和验证。

关键审计问题：

- CLI 是否覆盖配置；
- 嵌套字典是否传给环境工厂；
- 未识别字段是否报错或静默丢弃；
- resolved config 是否落盘。

`reinforcetactics/utils/run_config.py`

提供运行配置的辅助处理。配置文件是输入，真正生效的是脚本解析后的对象。

E.7 Bootstrap

`reinforcetactics/rl/bootstrap.py`

- `CurriculumStalled`：阶段预算耗尽异常；
- `_resolve_curriculum_pad_size`：跨地图统一观察；
- `_default_model_factory`：MaskablePPO 等模型构造；
- `_write_stage_config`、`_write_run_status`：审计产物；
- `run_curriculum`：阶段训练、评估、晋级、checkpoint 交接；
- `make_stage_env`：按阶段创建环境；
- `record_curriculum_replays`：回放。

`scripts/train/train_bootstrap.py`

headless CLI，支持配置、点路径覆盖、BC 预训练、本地输出、绘图/视频开关和本地禁用 GCS。教材不涉及 Cloud。

E.8 行为克隆

`reinforcetactics/rl/imitation.py`

- `Demonstration` : 单个观察—动作—掩码样本;
- `DemonstrationDataset` : 数据集;
- `_ActionRecorder` : 规则 Bot 动作拦截;
- `record_episode`、`collect_demonstrations` : 采集;
- `DemonstrationScenario` : 多场景定义;
- `collect_demonstrations_multi` : 场景混合;
- `behavior_clone` : masked cross-entropy;
- `make_warm_started_model` : 构造与加载;
- `evaluate_bc_against_bot_ladder` : 部署评估。

`scripts/build_bc_warmstart.py`

独立 BC 构建入口。其帮助文本明确说明：价值头不由 BC 拟合，空间与下游 curriculum 必须精确匹配。

E.9 自对弈

`reinforcetactics/rl/self_play.py`

- `OpponentPool` : 历史策略池;
- `SelfPlayEnv` : 环境 wrapper;
- `make_self_play_env`、`make_self_play_vec_env` : 工厂;
- `callback` 类: 更新对手、添加快照、记录评估。

核验 RNG 来源、池采样策略、player swap、对手模型设备、`eval()` 与 `no_grad()`。

`scripts/train/train_self_play.py`

CLI 支持 self-play/mixed、历史池、规则 Bot 比例、换边、VecEnv、resume 与 PPO 超参数。

E.10 Feudal RL

`reinforcetactics/rl/feudal_rl.py`

- `ManagerNetwork` : 高层目标;
- `WorkerNetwork` : 传统多头 Worker;
- `AutoregressiveActionHead`、`AutoregressiveWorkerNetwork` : 阶段条件动作;
- `_compute_gae` : 支持 `segment_lengths` ;
- `FeudalRolloutBuffer` : 两层轨迹、掩码与奖励分量;
- `FeudalRLAgent` : 采样与两层 PPO update;
- `compute_intrinsic_reward` : 目标进展奖励。

当前 Worker 奖励的权威语义在 `FeudalRolloutBuffer.add_worker_step` :

```
intrinsic_reward + worker_reward_alpha * extrinsic_reward
```

Manager GAE 的段折扣在 `_compute_gae` :

```
gamma ** segment_lengths[t]
```

`scripts/train/train_feudal_rl.py`

包括 flat PPO 对照与真正 Manager—Worker 训练路径。文件顶部旧注释若提到特定部署环境，不改变本书仅使用本地 CPU 的范围。

E.11 MCTS 与 AlphaZero

`reinforcetactics/rl/alphazero_net.py`

- `ResidualBlock` ;
- `AlphaZeroNet` ;
- 策略头大小 $10WH$;
- $\tanh$ 价值头;
- `predict` 中合法动作 masked softmax。

reinforcetactics/rl/mcts.py

- `MCTSNode` : `prior`、`visit_count`、`value_sum`、`children`;
- `_resolve_action_refs` : 深复制后的对象解析;
- `_execute_action_on_state` : 模拟转移;
- `_obs_from_game_state` : 网络观察与 flat mask;
- `MCTS.search` : 返回 (`action_probs`, `root_value`);
- `MCTS.select_action` : 温度采样;
- PUCT、根噪声与回传。

reinforcetactics/rl/alphazero_trainer.py

- `ReplayBuffer` ;
- `self_play_game` ;
- `AlphaZeroTrainer` ;
- 自对弈、联合损失、候选评估、checkpoint。

当前联合损失代码为策略交叉熵加价值 MSE, L2 由 optimizer `weight_decay` 实现。

scripts/train/train_alphazero.py

提供网络、搜索、自对弈、评估、设备与恢复参数。

E.12 评估、Tournament 与 Replay

reinforcetactics/rl/evaluation.py

- `_model_accepts_action_masks` : 识别 predict 接口;
- `evaluate_model` : 胜/和/负、回报、长度、终局、动作、奖励、单位、战斗、经济、可夺取和 trace。

reinforcetactics/tournament/

- `config.py` : `TournamentConfig` ;
- `schedule.py` : 地图与换边赛程;
- `runner.py` : 比赛执行;
- `bots.py` : 参赛者描述与 Bot 构造;
- `elo.py` : 期望得分与 rating 更新;
- `results.py` : 聚合与输出。

`reinforcetactics/utils/replay_actions.py`

动作记录与序列化。

`reinforcetactics/utils/replay_player.py`

回放读取与逐动作重建。Replay determinism 测试用于防止记录能写却不能重现。

E.13 LLM Bot

`reinforcetactics/game/llm_bot.py`

- `LLMBot`：通用状态、提示、重试、解析、执行与日志；
- `_serialize_game_state`；
- `_run_planning_phase`；
- `_format_prompt`；
- `_execute_actions`；
- `_extract_json`；
- 按动作类型的 `_execute_*`；
- `OpenAIBot`、`ClaudeBot`、`GeminiBot`：供应商适配。

本教材只测试 `LLMBot` 的本地 mock 数据管线，不调用这些供应商。

`reinforcetactics/game/llm_prompts.py`

basic、strategic、two-phase planning/execution 的提示模板。提示中规则数值应与当前常量做一致性测试。

E.14 DEV 与测试索引

领域	代表测试
环境与观察	<code>tests/test_rl_*.py</code>
Bot	<code>tests/test_*_bot.py</code>
Masking	掩码与环境相关测试
Bootstrap	课程/配置/评估相关测试
BC	imitation 相关测试
Self-play	self-play 相关测试
Feudal	<code>test_feudal_rl.py</code> 、 <code>test_feudal_rl_integration.py</code>
AlphaZero	<code>test_alphazero.py</code>
LLM mock	<code>test_llm_bot.py</code> 、 <code>test_llm_prompts.py</code>
Tournament	<code>test_tournament*.py</code>
Replay	<code>test_save_replay.py</code> 、 <code>test_replay_determinism.py</code>

精确文件清单可运行：

```
rg --files tests
```

E.15 原始论文与教材

以下只列原始论文、作者教材或框架官方文档。

强化学习基础

1. Richard S. Sutton, Andrew G. Barto. *Reinforcement Learning: An Introduction, Second Edition*. 2018.
<https://incompleteideas.net/book/the-book-2nd.html>

2. Richard Bellman. *A Markovian Decision Process*. Journal of Mathematics and Mechanics, 1957.
<https://www.jstor.org/stable/24900506>
3. Ronald J. Williams. *Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning*. Machine Learning, 1992.
<https://doi.org/10.1007/BF00992696>

DQN

4. Volodymyr Mnih et al. *Human-level control through deep reinforcement learning*. Nature, 2015.
<https://doi.org/10.1038/nature14236>
5. Hado van Hasselt, Arthur Guez, David Silver. *Deep Reinforcement Learning with Double Q-learning*. 2015.
<https://arxiv.org/abs/1509.06461>

Actor-Critic、GAE 与 PPO

6. Volodymyr Mnih et al. *Asynchronous Methods for Deep Reinforcement Learning*. 2016.
<https://arxiv.org/abs/1602.01783>
7. John Schulman et al. *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. 2015.
<https://arxiv.org/abs/1506.02438>
8. John Schulman et al. *Proximal Policy Optimization Algorithms*. 2017.
<https://arxiv.org/abs/1707.06347>
9. John Schulman et al. *Trust Region Policy Optimization*. 2015.
<https://arxiv.org/abs/1502.05477>

奖励塑形

10. Andrew Y. Ng, Daishi Harada, Stuart Russell. *Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping*. ICML, 1999.
<https://people.eecs.berkeley.edu/~russell/papers/icml99-shaping.pdf>

行为克隆与数据聚合

11. Stéphane Ross, Geoffrey Gordon, Drew Bagnell. *A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning*. AISTATS, 2011.

<https://proceedings.mlr.press/v15/ross11a.html>

层级强化学习

12. Peter Dayan, Geoffrey E. Hinton. *Feudal Reinforcement Learning*. NeurIPS, 1992.
<https://proceedings.neurips.cc/paper/1992/hash/d14220ee66aeec73c49038385428ec4c-Abstract.html>
13. Alexander Vezhnevets et al. *FeUdal Networks for Hierarchical Reinforcement Learning*. 2017.
<https://arxiv.org/abs/1703.01161>

MCTS 与 AlphaZero

14. Levente Kocsis, Csaba Szepesvári. *Bandit Based Monte-Carlo Planning*. ECML, 2006.
https://doi.org/10.1007/11871842_29
15. David Silver et al. *Mastering the game of Go without human knowledge*. Nature, 2017.
<https://doi.org/10.1038/nature24270>
16. David Silver et al. *A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play*. Science, 2018.
<https://arxiv.org/abs/1712.01815>

无效动作掩码

17. Shengyi Huang, Santiago Ontañón. *A Closer Look at Invalid Action Masking in Policy Gradient Algorithms*. 2020.
<https://arxiv.org/abs/2006.14171>

E.16 框架官方文档

PyTorch

- 张量: <https://pytorch.org/docs/stable/tensors.html>
- 自动求导: <https://pytorch.org/docs/stable/autograd.html>
- `nn.Module`: <https://pytorch.org/docs/stable/generated/torch.nn.Module.html>
- Optimizer: <https://pytorch.org/docs/stable/optim.html>
- Conv2d: <https://pytorch.org/docs/stable/generated/torch.nn.Conv2d.html>

Gymnasium

- Env API: <https://gymnasium.farama.org/api/env/>
- Spaces: <https://gymnasium.farama.org/api/spaces/>
- 处 理 time limits : https://gymnasium.farama.org/tutorials/gymnasium_basics/handling_time_limits/
- Wrapper: <https://gymnasium.farama.org/api/wrappers/>

Gymnasium 官方 API 的 `step` 返回 `observation`、`reward`、`terminated`、`truncated`、`info`。
`terminated` 表示 MDP 终局，`truncated` 表示在 MDP 外部条件下结束采样。

Stable-Baselines3

- 总览: <https://stable-baselines3.readthedocs.io/>
- PPO: <https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html>
- A2C: <https://stable-baselines3.readthedocs.io/en/master/modules/a2c.html>
- DQN: <https://stable-baselines3.readthedocs.io/en/master/modules/dqn.html>
- 自定义策略 / 特征提取 : https://stable-baselines3.readthedocs.io/en/master/guide/custom_policy.html
- VecEnv: https://stable-baselines3.readthedocs.io/en/master/guide/vec_envs.html
- Callback: <https://stable-baselines3.readthedocs.io/en/master/guide/callbacks.html>

sb3-contrib

- MaskablePPO: https://sb3-contrib.readthedocs.io/en/master/modules/ppo_mask.html

官方文档明确要求评估使用 `mask-aware` 接口；使用 `SubprocVecEnv` 时 `action_masks` 必须在环境内部实现。

PettingZoo

- API: <https://pettingzoo.farama.org/api/>
- AEC API: <https://pettingzoo.farama.org/api/aec/>
- Parallel API: <https://pettingzoo.farama.org/api/parallel/>

本项目安装 PettingZoo，但当前主 RL 路径是内部推进对手的 Gymnasium 单智能体环境。

E.17 历史材料与当前事实的分界

仓库中的历史训练评论用于重建研究过程，但引用时遵守：

- 配置文件说明实验意图；
- 日志/结果说明当时观察；
- 成对消融才提供较强因果证据；
- 当前源码定义当前行为；
- 当前本机冒烟定义当前接口是否可运行；
- 大规模能力仍需新的多种子实验。

当四者冲突时，不应把最旧的评论提升为当前事实。

E.18 建议阅读顺序

1. 正文 00–12，建立任务与框架；
2. 根据主线读 15–17；
3. 实际跑 MaskablePPO；
4. 再读 18–22 的进阶方法；
5. 用 23–27 设计实验；
6. 需要核验时查本附录对应符号；
7. 最后阅读原始论文，对照项目取舍。